



République Tunisienne

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

École Supérieure Privée d'ingénierie et de technologie
TEK-UP



RAPPORT DE PROJET DE FIN D'ÉTUDE

**Diplôme National d'Ingénieur en Sciences Appliquées et
Technologiques**

Spécialité : Génie Logiciel et Systèmes d'Information

Réalisé par

Sirine Abdelkhalek

Conception et développement de SUBUL – Plateforme d'accompagnement intelligent pour la préparation aux certifications et le développement des compétences

Encadrant Professionnel :

Mr Abdelkhalek Baccari

Encadrant Académique :

Mme Siwar Zrelli

Année Universitaire : 2025 - 2026

Avis et signature de l'encadrant professionnel

Encadrant professionnel, **M. Abdelkhalek Baccari**

Signature et cachet

Avis et signature de l'encadrant académique

Encadrant académique, **Mme. Siwar Zrelli**

Signature



Dédicace

وكان فضل الله عظيمًا

À mes chers parents,

Vous êtes la source de ma force, de ma persévérance et de mes ambitions. Votre amour inconditionnel et vos sacrifices silencieux m'ont permis d'avancer avec confiance. Merci pour tout.

À ma sœur Malak,

Pour ta présence, ta complicité et ton soutien inconditionnels dans chacun de mes moments. Tu es et tu resteras toujours ma plus belle source d'énergie et de bonheur.

À ma famille,

Pour votre amour, votre compréhension et votre soutien indéfectible. Merci d'avoir toujours cru en moi et de m'avoir encouragée à aller de l'avant.

À mon amie Nourhène,

Pour ta patience, ta générosité et ton amitié sincère. Tu étais ma binôme de stage, et ensemble nous avons affronté chaque obstacle, nous nous sommes soutenues l'une l'autre dans les moments difficiles comme dans les plus beaux. Merci d'avoir été là.

À moi-même,

Pour ne jamais avoir abandonné, même dans les moments les plus difficiles. Pour avoir cru en moi, tenu bon et continué à avancer malgré tout. Ce travail est aussi le fruit de ma propre volonté et de ma détermination.

À tous mes amis et collègues,

Pour les moments partagés, les fous rires et le soutien mutuel tout au long de cette aventure.

Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Je tiens tout d'abord à remercier l'université **TEK-UP** ainsi que toute l'équipe pédagogique pour la qualité de la formation dispensée, leur dévouement et leur engagement tout au long de notre cursus. Grâce à eux, nous avons pu acquérir les compétences nécessaires pour mener à bien ce projet.

Je remercie chaleureusement la startup **Smartovate** ainsi que son CEO, **Mr. Abdelkhalek Baccari**, pour la confiance qu'ils ont placée en moi en m'acceptant au sein de leur équipe pour effectuer ce stage.

J'adresse également mes sincères remerciements à mon encadrante académique, **Mme. Siwar Zrelli**, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses orientations ont été précieuses pour mener ce travail à bien.

Je tiens à exprimer une reconnaissance particulière à **M. Nidhal Jelassi** pour son soutien constant, ses conseils techniques et sa disponibilité tout au long de ce stage. Sa contribution a été d'une grande valeur pour l'avancement de ce projet.

Je tiens également à remercier tous les membres du jury pour avoir bien voulu examiner et juger ce travail.

Table des matières

Introduction Générale	2
1 cadre général du projet	3
Introduction	4
1.1 Présentation de l'organisme d'accueil	4
1.1.1 Mission et domaines d'expertise	4
1.1.2 Organigramme	5
1.2 Cadre du projet	5
1.3 Problématique	5
1.4 Étude de l'existant	6
1.4.1 Coursera	6
1.4.2 Alison	6
1.4.3 Synthèse comparative	7
1.5 Solution proposée	8
1.6 Méthodologie adoptée	8
1.6.1 Définition d'une méthodologie Agile	9
1.6.2 Comparaison entre méthodes classiques et méthodes agiles	9
1.6.3 Synthèse pour le choix de Scrum	10
Conclusion	11
2 Analyse et spécification des besoins	12
Introduction	13
2.1 Spécification des besoins	13
2.1.1 Identification des acteurs	13
2.1.2 Spécification des besoins fonctionnels	13
2.1.3 Spécification des besoins non fonctionnels	15
2.2 Conception	15

2.3	Planification de travail Scrum	18
2.3.1	Équipe et rôles	18
2.3.2	Backlog Produit	18
2.3.3	Planification des releases	21
2.4	Patrons de conception utilisés	22
2.5	Choix technologiques	24
2.5.1	Outils de gestion de projet	25
2.5.2	Outils de conception	25
2.5.3	Outils de développement	26
2.5.4	Frameworks et langages de développement	27
2.6	Architecture de l'application	30
2.6.1	Architecture logique (Microservices)	30
2.6.2	Architecture physique (Déploiement)	32
2.6.3	Communication inter-services (RabbitMQ)	33
2.6.4	Architecture du Service IA	33
	Conclusion	36
3	Release 1 – Fondation de la plateforme	37
	Introduction	38
3.1	Sprint 1 : Infrastructure et authentification	38
3.1.1	Objectifs du sprint 1	38
3.1.2	Backlog du sprint 1	38
3.1.3	Spécification et conception du sprint	39
3.1.4	Réalisation	49
3.2	Sprint 2 : Gestion des cours et progression	53
3.2.1	Objectifs du sprint 2	53
3.2.2	Backlog du sprint 2	54
3.2.3	Spécification et conception du sprint	54
3.2.4	Réalisation	65
	Conclusion	67

4	Release 2 – Fonctionnalités avancées et IA	68
	Introduction	69
4.1	Sprint 3 : Dashboard, Suivi, Objectifs et Administration	69
4.1.1	Objectifs du sprint 3	69
4.1.2	Backlog du sprint 3	70
4.1.3	Spécification et conception du sprint	71
4.1.4	Réalisation	83
4.2	Sprint 4 : Agents IA – TutorAgent et QuizAgent	85
4.2.1	Objectif du sprint	85
4.2.2	Backlog du sprint	86
4.2.3	Spécification et conception du sprint	86
4.2.4	Fondements technologiques	87
4.2.5	Workflow de traitement des données	90
4.2.6	Implémentation — Extraits de code clés	91
4.2.7	Architecture globale des deux agents IA	93
4.2.8	Agent 1 – TutorAgentMCP	95
4.2.9	Agent 2 – QuizAgent	99
4.2.10	Évaluation des agents IA	104
4.2.11	Réalisation	106
	Conclusion	107
5	Release 3 – Tests et monitoring	108
	Introduction	109
5.1	Sprint 5 : Tests et Qualité	109
5.1.1	Objectifs du sprint 5	109
5.1.2	Backlog technique du sprint 5	109
5.1.3	Tests unitaires	110
5.1.4	Tests d’intégration	115
5.1.5	Tests End-to-End et de sécurité	116
5.1.6	Extraits de code — Tests représentatifs	118

5.1.7	Tests de charge	119
5.2	Sprint 6 : Déploiement et Infrastructure	120
5.2.1	Objectifs du sprint 6	120
5.2.2	Backlog technique du sprint 6	120
5.3	L'Architecture Complète : Du Code à la Production	121
5.3.1	Vue d'ensemble du flux de déploiement	121
5.3.2	PHASE 1 : Configuration AWS Infrastructure	122
5.3.3	PHASE 2 : Microservices & Containerisation	126
5.3.4	PHASE 3 : GitHub Actions CI/CD Pipeline	130
5.3.5	PHASE 4 : Amazon ECR Registry	132
5.3.6	PHASE 5 : Kubernetes (EKS) Deployment	133
5.3.7	PHASE 6 : Load Balancers & Networking	134
5.3.8	PHASE 7 : Security & Secrets Management	136
	Conclusion	136
	Conclusion Générale	137
	Bibliographie et Webographie	138
	Bibliographie	138
	Webographie	138

Table des figures

1.1	Logo de l'entreprise Smartovate	4
1.2	Organigramme de l'entreprise	5
1.3	Logo de la plateforme Coursera	6
1.4	Logo de la plateforme Alison	7
1.5	Méthodologie traditionnelle vs Agile	9
2.1	Diagramme des cas d'utilisation global	16
2.2	Diagramme des classes global	17
2.3	Diagramme de planification des releases et sprints	22
2.4	Logo ClickUp	25
2.5	Logo Teams	25
2.6	Logo LucidChart	26
2.7	Architecture logique microservices de Subul Academy	31
2.8	Architecture interne du service IA	35
3.1	Diagramme de cas d'utilisation — Sprint 1	40
3.2	Diagramme de raffinement — UC1 : S'inscrire	41
3.3	Diagramme de raffinement — UC4 : Gérer utilisateurs	43
3.4	Diagramme de raffinement — UC6 : Gérer profil	44
3.5	Diagramme de séquence — Inscription et vérification email	46
3.6	Diagramme de séquence — Authentification JWT	47
3.7	Diagramme de séquence — Gestion des utilisateurs (Admin)	48
3.8	Page d'accueil de la plateforme Subul	49
3.9	Interfaces d'inscription et de connexion	50
3.10	Processus de vérification de l'email	50
3.11	Page de profil de l'utilisateur	51
3.12	Interface d'administration — Liste des utilisateurs	51

3.13	Gestion des comptes utilisateurs par l'administrateur	52
3.14	Gestion des rôles et des statuts des comptes	53
3.15	Diagramme de cas d'utilisation — Sprint 2	55
3.16	Diagramme de raffinement — UC1 : Gérer cours	56
3.17	Diagramme de raffinement — UC3 : S'inscrire / Désinscrire à un cours	57
3.18	Diagramme de raffinement — UC4 : Consulter un cours	59
3.19	Diagramme de raffinement — UC5 : Gérer notes	60
3.20	Diagramme de raffinement — UC6 : Gérer Flashcards	61
3.21	Diagramme de séquence — Sprint 2 (Catalogue, Inscription et Progression)	63
3.22	Diagramme de séquence — Sprint 2 (Notes & Flashcards SM-2)	64
3.23	Catalogue de cours	65
3.24	Consultation des leçons et suivi de progression	66
3.25	Outils d'apprentissage actif : notes et flashcards	67
4.1	Diagramme de cas d'utilisation — Sprint 3	71
4.2	Diagramme de raffinement — UC1 : Dashboard utilisateur	72
4.3	Diagramme de raffinement — UC2 : Suivi apprentissage	74
4.4	Diagramme de raffinement — UC3 : Gestion de notifications	75
4.5	Diagramme de raffinement — UC4 : Gestion des Objectifs	77
4.6	Diagramme de raffinement — UC5 : Dashboard Admin	78
4.7	Diagramme de raffinement — UC6 : Statistiques	79
4.8	Diagramme de séquence — Sprint 3 (Tableau de bord & Gestion des objectifs)	81
4.9	Diagramme de séquence — Sprint 3 (Notifications en temps réel) . . .	82
4.10	Tableau de bord apprenant	83
4.11	Gestion des objectifs	83
4.12	Formulaire de création d'un objectif hebdomadaire	84
4.13	Formulaire de création d'un objectif quotidien	84
4.14	Tableau de bord administrateur	84
4.15	Statistiques d'utilisation de la plateforme	85

4.16	Diagramme de cas d'utilisation — Sprint 4	87
4.17	Architecture du paradigme ReAct (<i>Reasoning and Acting</i>)	88
4.18	Graphe d'état LangGraph ReAct du TutorAgentMCP – états et transitions	88
4.19	Pipeline de traitement des données : de l'acquisition à l'exploitation . .	91
4.20	Position des deux agents dans le AI Service (FastAPI)	94
4.21	Architecture interne du TutorAgentMCP	95
4.22	Diagramme d'activité – TutorAgentMCP	97
4.23	Architecture du QuizAgent et ses 3 sous-agents	100
4.24	Diagramme d'activité – QuizAgent	102
4.25	Interface du tuteur IA (streaming WebSocket, mode texte)	107
4.26	Interface du quiz de module (questions, feedback et misconception) . .	107
5.1	Résultats des tests unitaires — auth-service (13 tests passés)	113
5.2	Résultats des tests unitaires — user-service (28 tests passés)	113
5.3	Résultats des tests unitaires — course-service (43 tests passés)	114
5.4	Résultats des tests unitaires — notification-service (11 tests passés) . .	114
5.5	Résultats des tests unitaires — gateway (27 tests passés)	115
5.6	Vue d'ensemble du flux de déploiement — Infrastructure AWS Subul .	122
5.7	VPC et subnets créés dans AWS Management Console	124
5.8	Vue d'ensemble du cluster EKS dans AWS Management Console	125
5.9	Dockerfile du service Gateway (build multi-stage)	128
5.10	Dockerfile du service AI (Python/FastAPI)	129
5.11	Extrait du fichier Docker Compose (services infrastructure et applicatifs)	130
5.12	Exemple d'exécution d'un workflow GitHub Actions	131
5.13	Images Docker disponibles dans Amazon ECR par service	133
5.14	État des pods en production (<code>kubectl get pods — STATUS : Running</code>)	134

Liste des tableaux

1.1	Comparaison des plateformes e-learning	7
1.2	Comparaison synthétique des approches classiques et agiles	10
2.1	Acteurs de la plateforme Subul	13
2.2	Besoins fonctionnels de la plateforme Subul	14
2.3	Besoins non fonctionnels de la plateforme Subul	15
2.4	Équipe et rôles Scrum du projet Subul	18
2.5	Backlog produit de la plateforme Subul	19
2.6	Planification des releases et des sprints	21
2.7	Patrons de conception utilisés dans Subul Academy	23
2.8	Outils de développement et DevOps	27
2.9	Technologies du Frontend	28
2.10	Technologies du Backend	29
2.11	Technologies de bases de données et de communication	29
2.12	Répartition des microservices et responsabilités fonctionnelles	31
2.13	Infrastructure de déploiement basée sur Docker Compose	32
2.14	Modes de communication inter-services	33
2.15	Architecture interne du service d'intelligence artificielle	34
2.16	Technologies utilisées dans le service d'intelligence artificielle	36
3.1	Backlog du Sprint 1 — Release 1	38
3.2	Description textuelle — UC1 : S'inscrire	41
3.3	Description textuelle — UC4 : Gérer utilisateurs	43
3.4	Description textuelle — UC6 : Gérer profil	45
3.5	Backlog du Sprint 2 — Release 1	54
3.6	Description textuelle — UC1 : Gérer cours	56
3.7	Description textuelle — UC3 : S'inscrire / Désinscrire à un cours	58

3.8	Description textuelle — UC4 : Consulter un cours	59
3.9	Description textuelle — UC5 : Gérer notes	60
3.10	Description textuelle — UC6 : Gérer Flashcards	62
4.1	Backlog du Sprint 3 — Release 2	70
4.2	Description textuelle — UC1 : Dashboard utilisateur	73
4.3	Description textuelle — UC2 : Suivi apprentissage	74
4.4	Description textuelle — UC3 : Gestion de notifications	76
4.5	Description textuelle — UC4 : Gestion des Objectifs	77
4.6	Description textuelle — UC5 : Dashboard Admin	78
4.7	Description textuelle — UC6 : Statistiques	80
4.8	Backlog du Sprint 4 — Release 2	86
4.9	Comparaison HTTP classique vs streaming WebSocket	89
4.10	Stack technologique des deux agents	94
4.11	Composants du TutorAgentMCP	96
4.12	Comportement multilingue du TutorAgentMCP	98
4.13	Les 3 sous-agents du QuizAgent	100
4.14	Modèles Pydantic du QuizAgent	103
4.15	Stratégie de résilience du QuizAgent	104
4.16	Exemples d'évaluation qualitative du TutorAgentMCP	105
4.17	Métriques d'évaluation du QuizAgent (échantillon de 30 questions) . . .	106
5.1	Backlog du Sprint 5 — Release 3	110
5.2	Récapitulatif des tests unitaires et couverture de code	115
5.3	Backlog du Sprint 6 — Release 3	121
5.4	VPC Configuration — Subul Production	123
5.5	AWS EKS Cluster — Subul Production	124
5.6	RDS PostgreSQL Configuration — Subul Production	125
5.7	Redis Cache Configuration — Subul Production	126
5.8	Les 13 Microservices Subul	127

5.9	Workflows GitHub Actions du projet Subul	131
5.10	Étapes de déploiement par service	132
5.11	AWS Elastic Load Balancers — Subul Production	135
5.12	Flux d’une requête de connexion apprenant	135
5.13	Kubernetes Secrets configurés — Subul Production	136

Liste des abréviations

AI	Artificial Intelligence (Intelligence Artificielle)
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
CEO	Chief Executive Officer
CI/CD	Continuous Integration / Continuous Deployment
COO	Chief Operating Officer
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CTO	Chief Technology Officer
DNS	Domain Name System
DTO	Data Transfer Object
E2E	End-to-End
ECR	Elastic Container Registry
EKS	Elastic Kubernetes Service
HA	High Availability (Haute Disponibilité)
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IA	Intelligence Artificielle
IDOR	Insecure Direct Object Reference
IoC	Inversion of Control
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCP	Model Context Protocol
MOOC	Massive Open Online Course

NLP	Natural Language Processing
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
QCM	Questionnaire à Choix Multiples
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RPC	Remote Procedure Call
SEO	Search Engine Optimization
SOLID	Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion
SPA	Single Page Application
SQL	Structured Query Language
SSG	Static Site Generation
SSO	Single Sign-On
SSR	Server-Side Rendering
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
WebSocket	Web Socket Protocol

Introduction Générale

Les plateformes d'apprentissage en ligne occupent aujourd'hui une place centrale dans la formation professionnelle, notamment pour la préparation aux certifications IT reconnues dans des domaines comme le cloud computing, la cybersécurité et l'intelligence artificielle. Dans ce contexte, les récentes avancées en IA, particulièrement les modèles de langage de grande taille et les architectures multi-agents, offrent de nouvelles possibilités pour enrichir l'expérience pédagogique et accompagner intelligemment les apprenants tout au long de leur parcours.

Cependant, concevoir un tel système soulève des défis à deux niveaux. Du côté de l'apprenant, les plateformes existantes proposent des contenus statiques sans accompagnement personnalisé : l'utilisateur se retrouve seul face à ses difficultés, sans guidance adaptée à son profil ni retour intelligent sur ses erreurs. Du côté technique, intégrer plusieurs agents IA spécialisés au sein d'une architecture microservices, garantir des réponses contextualisées en temps réel via une architecture RAG (*Retrieval-Augmented Generation*) et assurer la scalabilité du système en production représente un défi d'ingénierie complexe.

C'est dans ce double contexte que s'inscrit notre Projet de Fin d'Études, réalisé au sein de l'entreprise Smartovate. L'objectif est de concevoir et de développer **Subul**, une plateforme e-learning intelligente dont l'architecture microservices intègre plusieurs agents IA spécialisés : un agent tuteur et un agent de génération de quiz . Le projet a été conduit selon la méthodologie Scrum, organisée en releases et sprints successifs pour garantir une livraison progressive et maîtrisée.

Ce rapport présente l'ensemble des travaux réalisés et s'organise en cinq chapitres. **Le premier chapitre** expose le contexte général du projet, l'organisme d'accueil, la problématique et la méthodologie adoptée. **Le deuxième chapitre** couvre l'analyse des besoins, les choix technologiques et l'architecture globale de la plateforme. **Le troisième chapitre** décrit la réalisation de la première release avec les fonctionnalités de base. **Le quatrième chapitre** présente la deuxième release, axée sur l'intégration des agents intelligents et l'amélioration de l'expérience utilisateur. **Le cinquième chapitre** est consacré à la troisième release, portant sur l'agent tuteur conversationnel, l'agent de génération automatique de quiz, ainsi que les tests et le déploiement en production sur infrastructure AWS.

Chapitre 1

cadre général du projet

Sommaire du chapitre

1. Introduction

2. Présentation de l'organisme d'accueil

- Mission et domaines d'expertise
- Organigramme

3. Cadre général du projet

- État de l'art
- Synthèse comparative
- Problématique
- Solution proposée

4. Méthodologie de gestion de projet à adopter

- Comparaison entre méthodes classiques et méthodes agiles
- Définition d'une méthodologie Agile
- Synthèse pour le choix de Scrum

5. Conclusion

Introduction

Ce chapitre présente le cadre général du projet, l'entreprise d'accueil **Smartovate**, la problématique identifiée ainsi que la solution Subul proposée pour y répondre. Il expose également la méthodologie **Scrum** adoptée pour la gestion du développement.

1.1 Présentation de l'organisme d'accueil

1.1.1 Mission et domaines d'expertise

Smartovate est une entreprise spécialisée dans le développement de solutions logicielles basées sur les technologies de l'intelligence artificielle et du cloud computing.

Elle conçoit des applications adaptées aux besoins métiers de ses clients, en s'appuyant sur des approches modernes orientées automatisation, traitement de données et intelligence artificielle.

L'entreprise intervient principalement dans les domaines suivants :

- Développement web
- Intelligence Artificielle (IA)
- Services cloud
- Internet of Things (IoT)

Grâce à son expertise technique, Smartovate accompagne ses clients tout au long du cycle de développement logiciel, depuis l'analyse des besoins jusqu'à la mise en production des solutions.



FIGURE 1.1 :Logo de l'entreprise Smartovate

1.1.2 Organigramme

La figure 1.2 présente la structure organisationnelle de l'entreprise, articulée autour de trois postes fondateurs.

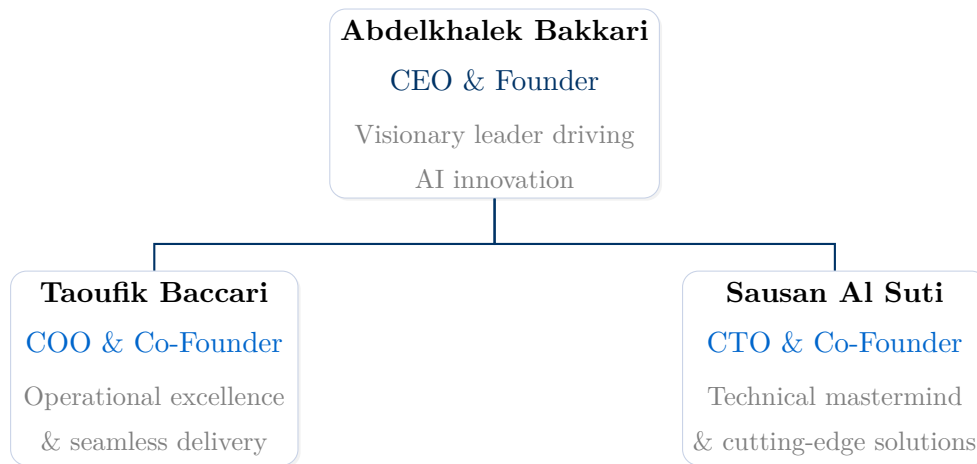


FIGURE 1.2 : Organigramme de l'entreprise

1.2 Cadre du projet

L'e-learning occupe aujourd'hui une place centrale dans les stratégies de formation des entreprises et des institutions éducatives.

Portée par la démocratisation d'Internet et l'essor de l'intelligence artificielle générative, la formation en ligne connaît une transformation profonde [1].

Les plateformes modernes tendent progressivement vers des systèmes d'apprentissage adaptatifs capables de personnaliser les contenus pédagogiques selon le profil et les performances de chaque apprenant.

1.3 Problématique

L'analyse des plateformes existantes révèle une limite commune et fondamentale : elles reproduisent majoritairement un modèle d'apprentissage passif.

L'apprenant consulte des vidéos et répond à des questionnaires, mais il reste souvent seul face à ses difficultés. Lorsqu'une erreur est commise, la plateforme fournit généralement uniquement la bonne réponse, sans explication détaillée ni accompagnement personnalisé.

Par ailleurs, peu de solutions proposent un tuteur conversationnel capable de répondre aux questions de l'apprenant en temps réel à partir du contenu pédagogique étudié.

Dans ce contexte, une problématique centrale se pose :

Comment concevoir une plateforme e-learning exploitant pleinement les capacités de l'intelligence artificielle afin de fournir un accompagnement pédagogique intelligent, adaptatif et personnalisé ?

1.4 Étude de l'existant

Avant de définir notre solution, il est essentiel d'analyser les plateformes e-learning existantes afin d'identifier leurs forces et leurs limites.

1.4.1 Coursera

Coursera est l'une des plateformes de type MOOC (*Massive Open Online Courses*) les plus utilisées au monde [2].

Elle propose des cours certifiants en partenariat avec des universités et des entreprises de renom. La plateforme offre une large bibliothèque de contenus structurés sous forme de vidéos, lectures et quiz.

Cependant, ses évaluations restent principalement statiques et prédéfinies : elles ne s'adaptent pas dynamiquement aux difficultés spécifiques de l'apprenant. De plus, aucun tuteur conversationnel intelligent ne permet d'interagir en langage naturel avec le contenu pédagogique.



FIGURE 1.3 :Logo de la plateforme Coursera

1.4.2 Alison

Alison est une plateforme d'apprentissage en ligne gratuite fondée en 2007, proposant plus de 4 000 cours certifiants dans des domaines variés tels que l'informatique, les

réseaux, la cybersécurité, le développement web et la gestion de projet.

Sa particularité réside dans son accessibilité : tous les cours sont disponibles gratuitement, et les apprenants peuvent obtenir des certificats ou diplômes reconnus moyennant des frais modérés. Alison cible principalement les professionnels souhaitant valider des compétences techniques via des parcours structurés.

Cependant, la plateforme ne propose pas de personnalisation pédagogique avancée : les contenus sont fixes et identiques pour tous les apprenants, sans adaptation aux lacunes individuelles. De plus, aucun mécanisme de tutorat conversationnel intelligent n'est intégré pour accompagner l'apprenant en temps réel.



FIGURE 1.4 :Logo de la plateforme Alison

1.4.3 Synthèse comparative

Le tableau 1.1 présente une comparaison entre les plateformes étudiées et notre solution Subul.

TABLEAU 1.1 :Comparaison des plateformes e-learning

Critère	Coursera	Alison	Subul
Quiz adaptatifs	Statiques	Banque fixe	Générés par IA
Feedback personnalisé	Correction simple	Correction basique	Explication par IA
Tuteur conversationnel	Non	Non	Oui (RAG + LLM)
Suivi des lacunes	Non	Non	Détection automatique
Domaines techniques	Oui	Oui (IT, réseaux, etc.)	Oui (Cloud, IA, etc.)

1.5 Solution proposée

Afin de répondre aux limites des plateformes d'apprentissage actuelles, notamment le manque de personnalisation et de suivi intelligent, nous proposons **Subul Academy**, une plateforme dédiée à la préparation aux certifications informatiques intégrant des agents d'intelligence artificielle.

Notre contribution porte sur le développement de deux agents IA centraux : l'**Agent Tuteur Conversationnel** et l'**Agent Générateur de Quiz Adaptatif**.

L'**Agent Tuteur Conversationnel** accompagne l'apprenant tout au long de son parcours en répondant à ses questions sur les contenus étudiés. Il fournit des réponses contextualisées, en français ou en anglais, à partir des ressources pédagogiques de la plateforme et peut également restituer ses explications sous forme vocale.

L'**Agent Générateur de Quiz Adaptatif** produit automatiquement des évaluations personnalisées selon le contenu étudié et le niveau de l'apprenant. En cas d'erreur, il identifie les notions mal maîtrisées, fournit des explications ciblées et propose de nouvelles questions afin de renforcer l'acquisition des connaissances.

Grâce à ces deux agents, Subul offre un accompagnement pédagogique intelligent, personnalisé et évolutif, favorisant une meilleure préparation aux certifications informatiques.

1.6 Méthodologie adoptée

La complexité d'un projet tel que Subul, mêlant architecture distribuée, intégration de services cloud et développement d'agents intelligents, exige une méthodologie de gestion rigoureuse. En effet, la multiplicité des microservices, l'interdépendance des composants et l'évolution continue des exigences fonctionnelles rendent indispensable l'adoption d'un cadre de travail structuré, capable de s'adapter aux changements tout en maintenant une livraison régulière de valeur.

Il est donc essentiel de choisir une approche méthodologique permettant à la fois une compréhension approfondie des besoins, une conception itérative de l'architecture logicielle et une anticipation efficace des contraintes techniques inhérentes au développement d'une plateforme à base d'intelligence artificielle.

1.6.1 Définition d'une méthodologie Agile

La méthodologie Agile est une approche de gestion de projet fondée sur un développement itératif et incrémental, favorisant la collaboration entre les membres de l'équipe ainsi que l'adaptation continue aux évolutions des besoins [3].

Contrairement aux méthodes traditionnelles, les approches Agile reposent sur des cycles de développement courts appelés *sprints*, permettant de concevoir, développer et tester progressivement les fonctionnalités du système.

Cette méthodologie favorise également la communication continue, la livraison progressive des fonctionnalités ainsi que l'amélioration continue du projet tout au long du processus de développement.

1.6.2 Comparaison entre méthodes classiques et méthodes agiles

Les méthodes classiques, telles que le cycle en V ou le modèle en cascade, reposent sur une planification exhaustive en amont et une exécution séquentielle des phases : analyse, conception, développement puis tests. Cette rigidité les rend peu adaptées aux projets dont les exigences évoluent en cours de route. À l'inverse, les méthodes agiles découpent le projet en itérations courtes et livrent des incréments fonctionnels réguliers, permettant d'intégrer les retours utilisateurs à chaque cycle et de s'adapter rapidement aux changements.

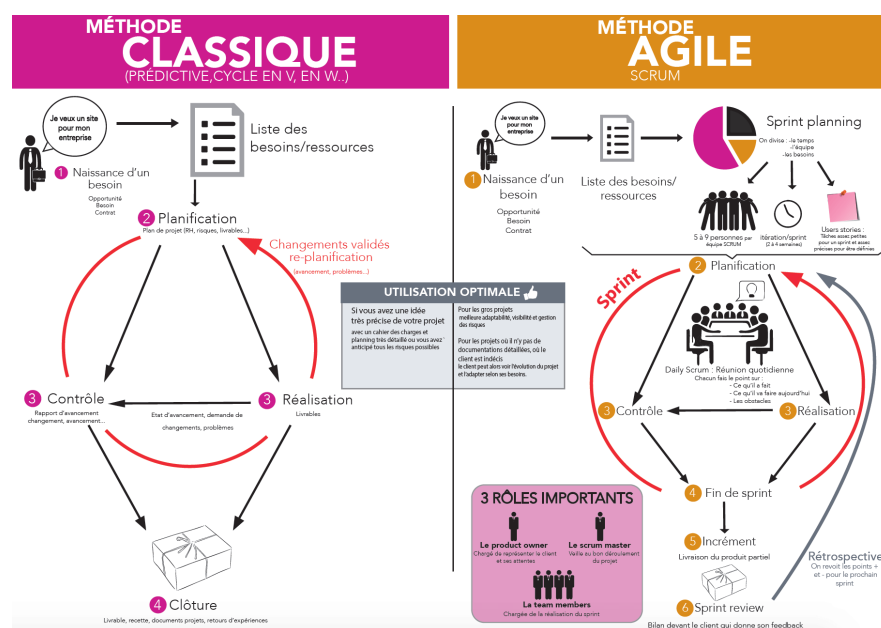


FIGURE 1.5 : Méthodologie traditionnelle vs Agile

La figure 1.5 illustre les différences fondamentales entre l’approche traditionnelle et l’approche agile. Afin de compléter cette comparaison, le tableau 1.2 présente de manière synthétique leurs principales caractéristiques ainsi que leurs points communs.

TABLEAU 1.2 : Comparaison synthétique des approches classiques et agiles

Points communs		
Répondre aux besoins du client		
Garantir la qualité du produit		
Suivre un processus de développement structuré		
Aspect	Classique	Agile
Planification	Fixe	Adaptative
Livraison	Unique	Itérative
Client	Peu impliqué	Très impliqué
Documentation	Complète	Essentielle
Gestion du changement	Rigide	Flexible

1.6.3 Synthèse pour le choix de Scrum

Le développement de la plateforme Subul repose sur une architecture microservices intégrant plusieurs composants indépendants ainsi que des services d’intelligence artificielle évolutifs. Dans ce contexte, il est nécessaire d’adopter une méthodologie de gestion de projet flexible et adaptative.

Parmi les méthodologies Agile existantes, Scrum a été retenu en raison de son approche itérative basée sur des cycles courts appelés *sprints*. Cette méthode permet de développer, tester et valider progressivement les différentes fonctionnalités de la plateforme.

Le choix de Scrum présente plusieurs avantages pour ce projet :

- une meilleure organisation des tâches et des fonctionnalités ;
- une adaptation continue aux évolutions techniques et fonctionnelles ;
- un suivi régulier de l’avancement du projet ;
- une livraison progressive des composants développés.

Ainsi, Scrum constitue une solution adaptée au caractère évolutif et modulaire de la plateforme Subul.

Conclusion

Ce chapitre a permis de présenter le contexte du projet, l'analyse des plateformes existantes et la solution Subul proposée. La méthodologie Scrum a été retenue pour structurer le développement. Le chapitre suivant détaillera l'analyse des besoins et l'architecture technique de la plateforme.

Chapitre 2

Analyse et spécification des besoins

Sommaire du chapitre

1. Introduction
2. Identification des acteurs
3. Spécification des besoins fonctionnels
4. Spécification des besoins non fonctionnels
5. Diagramme des cas d'utilisation global
6. Diagramme de classes global
7. Planification de travail Scrum
 - Équipe et rôles
 - Backlog Produit
 - Planification des releases
 - Diagramme de Gantt
8. Choix technologiques
 - Outils de gestion de projet
 - Outils de conception
 - Outils de développement
 - Frameworks et langages de développement
9. Architecture de l'application
 - Architecture logique (Microservices)
 - Architecture physique (Déploiement)
 - Communication inter-services (RabbitMQ)
 - Patrons de conception utilisés
 - Architecture du service IA
10. Conclusion

Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels de la plateforme Subul, les diagrammes de conception, la planification Scrum ainsi que les choix technologiques et l'architecture microservices adoptés.

2.1 Spécification des besoins

2.1.1 Identification des acteurs

Cette section présente les principaux intervenants de la plateforme Subul et décrit leur rôle au sein du système.

TABLEAU 2.1 :Acteurs de la plateforme Subul

Acteur	Description
Visiteur	Utilisateur non authentifié qui peut consulter la page d'accueil, parcourir le catalogue de cours publics et s'inscrire pour créer un compte.
Apprenant	Utilisateur authentifié qui suit des cours, passe des quiz et interagit avec l'agent tuteur pour approfondir ses connaissances.
Administrateur	Gère les comptes utilisateurs, la configuration globale de la plateforme ainsi que les cours, les modules et leur contenu pédagogique.

2.1.2 Spécification des besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir à ses différents acteurs. Le tableau 2.2 recense les principaux besoins identifiés.

TABLEAU 2.2 :Besoins fonctionnels de la plateforme Subul

ID	Fonctionnalité	Description
BF1	Gestion des comptes	Le système doit permettre à un visiteur de s'inscrire, de vérifier son email et de se connecter via un mécanisme JWT.
BF2	Gestion des cours	Le système doit permettre à un administrateur de créer, modifier et supprimer des cours, modules et leçons.
BF3	Inscription et progression	Le système doit permettre à un apprenant de s'inscrire à des cours et de suivre sa progression au fil des leçons.
BF4	Notes et flashcards	Le système doit offrir à l'apprenant la possibilité de prendre des notes et de générer des flashcards à partir du contenu pédagogique.
BF5	Agent tuteur IA	Le système doit proposer un agent conversationnel capable de répondre aux questions de l'apprenant à partir du contenu des cours (RAG).
BF6	Génération de quiz	Le système doit générer automatiquement des quiz adaptés au niveau de l'apprenant et fournir un feedback personnalisé.
BF7	Tableaux de bord	Le système doit proposer des tableaux de bord personnalisés pour l'apprenant et l'administrateur.
BF8	Notifications	Le système doit envoyer des notifications en temps réel aux utilisateurs via Redis pub/sub.
BF9	Objectifs d'apprentissage	Le système doit permettre à l'apprenant de définir et de suivre des objectifs journaliers et hebdomadaires.

2.1.3 Spécification des besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et de performance que le système doit respecter. Le tableau 2.3 présente les principales exigences identifiées.

TABLEAU 2.3 :Besoins non fonctionnels de la plateforme Subul

Critère	Exigence
Performance	Les pages de l’interface doivent se charger en moins de 2 secondes pour garantir une expérience utilisateur fluide.
Sécurité	Les données sensibles doivent être chiffrées. L’accès aux ressources est contrôlé par des tokens JWT à durée de vie limitée.
Scalabilité	L’architecture microservices doit permettre une montée en charge horizontale en déployant plusieurs instances d’un même service.
Disponibilité	La conteneurisation via Docker garantit la portabilité et la reproductibilité des environnements, assurant une haute disponibilité.
Maintenabilité	Le code doit être typé (TypeScript/Python), modulaire et respecter les principes SOLID afin de faciliter la maintenance et l’évolution.
Compatibilité	L’application doit être accessible sur tous les navigateurs modernes (Chrome, Firefox, Safari, Edge) sans dégradation de l’expérience.

2.2 Conception

Diagramme de cas d’utilisation global

La figure 2.1 présente le diagramme de cas d’utilisation global de la plateforme **Subul Academy**, illustrant l’ensemble des interactions entre les acteurs et les fonctionnalités offertes.

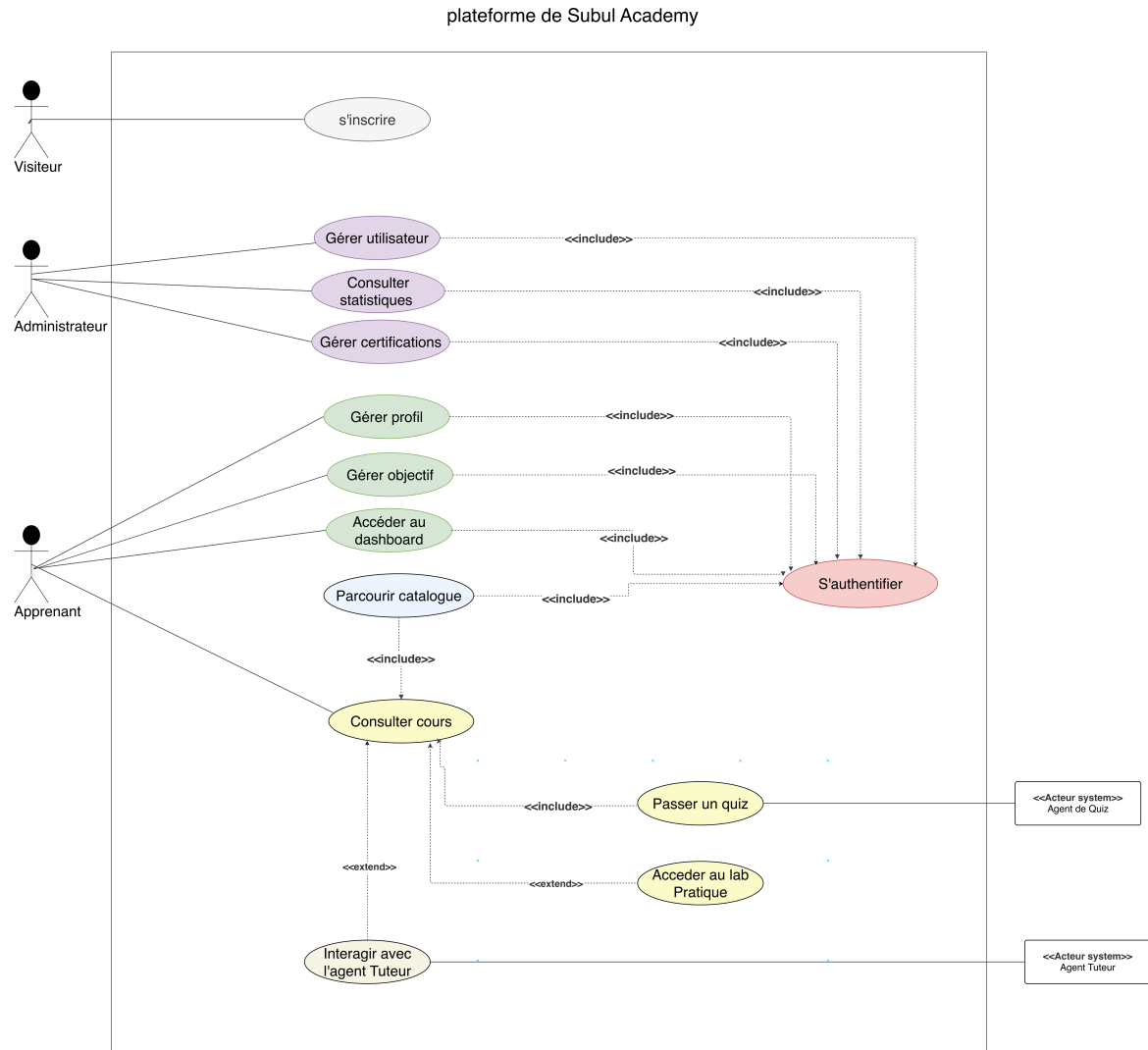


FIGURE 2.1 :Diagramme des cas d'utilisation global

Trois acteurs principaux ont été identifiés : le **Visiteur**, utilisateur non authentifié qui peut explorer la plateforme et s'inscrire, l'**Administrateur**, responsable de la gestion des contenus et des utilisateurs, et l'**Apprenant**, qui consomme les ressources pédagogiques. Deux acteurs systèmes complètent ce modèle : l'**Agent de Quiz**, qui génère et évalue automatiquement les quiz associés aux cours, et l'**Agent Tuteur**, un agent conversationnel intelligent qui accompagne l'apprenant dans sa compréhension du contenu pédagogique.

Diagramme de classes global

La figure 2.2 présente le diagramme de classes global de la plateforme **Subul Academy**, représentant la structure statique du système et les relations entre les entités métier.

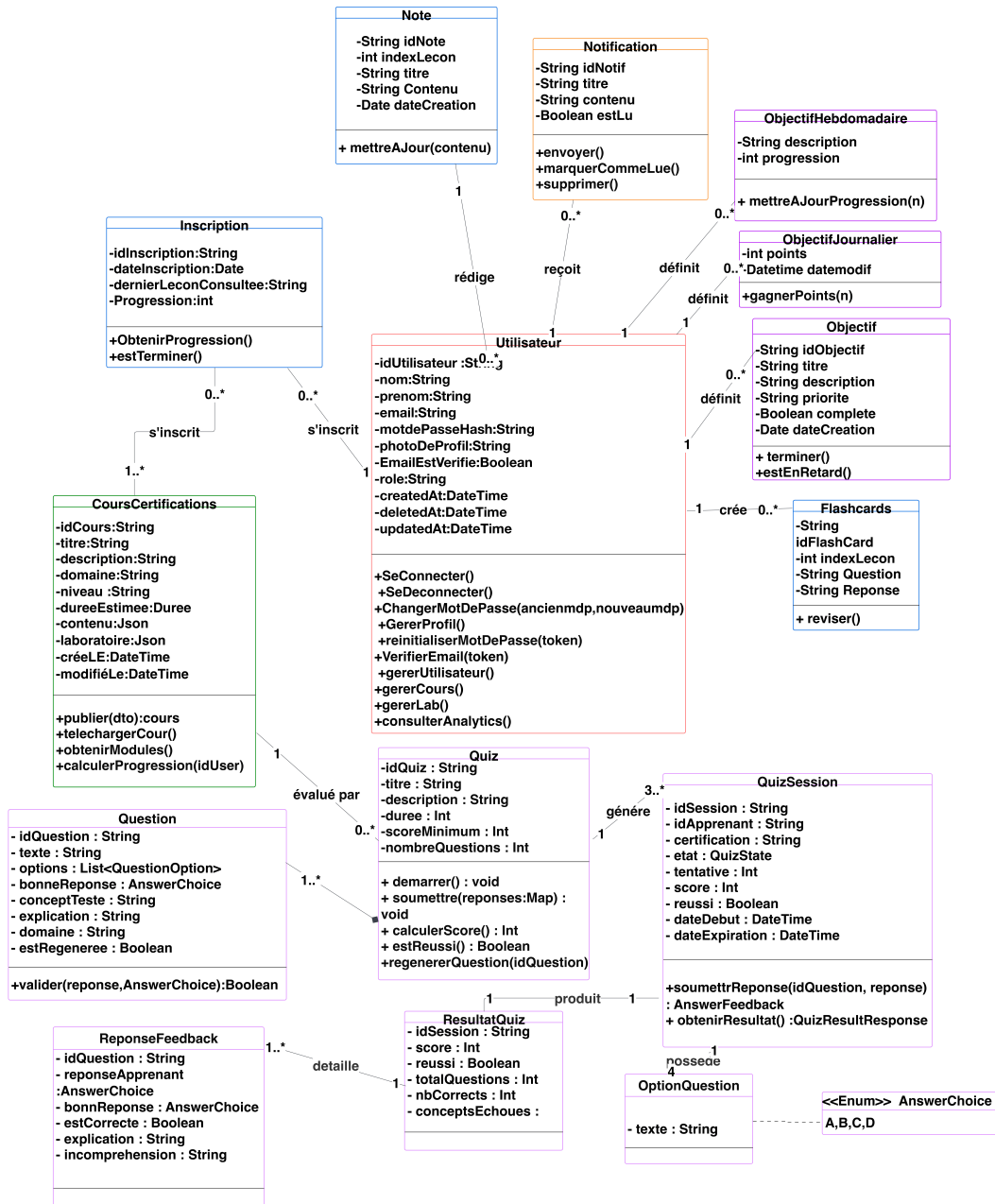


FIGURE 2.2 :Diagramme des classes global

La classe centrale est l'**Utilisateur**, qui s'inscrit à des **CoursCertifications** — contenant du contenu JSON, des laboratoires et des modules — via la classe **Inscription** qui trace sa progression. Il rédige des **Notes**, crée des **Flashcards** révisables, reçoit des **Notifications** et définit des **Objectifs** déclinés en **ObjectifJournalier** et

ObjectifHebdomadaire. Le module IA évalue les cours via le **Quiz**, qui génère des **QuizSessions** composées de **Questions** à choix multiples (**AnswerChoice** : A, B, C, D), produisant un **ResultatQuiz** et des **ReponseFeedback** détaillés.

2.3 Planification de travail Scrum

La gestion du projet **Subul Academy** s'appuie sur la méthodologie agile **Scrum**, un cadre de développement itératif permettant de livrer de la valeur de manière régulière et contrôlée sur une période de **5 mois**, soit **20 semaines**.

2.3.1 Équipe et rôles

Le tableau 2.4 présente la composition de l'équipe Scrum du projet et les responsabilités associées à chaque rôle.

TABLEAU 2.4 :Équipe et rôles Scrum du projet Subul

Rôle	Responsable	Responsabilités
Scrum Master	Taoufik Baccari	Facilitation des cérémonies Scrum, levée des obstacles et garantie du respect du processus agile.
Product Owner	Abdelkhalek Baccari	Définition et priorisation du backlog produit, validation des fonctionnalités livrées.
Équipe de développement	Sirine Abdelkhalek, Nourhene Ferchichi	Conception, implémentation et validation des fonctionnalités selon les priorités du sprint.

2.3.2 Backlog Produit

Le backlog produit regroupe l'ensemble des user stories du système, organisées par **sprint** et priorisées selon la valeur métier.

TABLEAU 2.5 :Backlog produit de la plateforme Subul

Sprint	ID	Module	User Story	Priorité
Sprint 1 — Infrastructure et authentification				
1	US-01	Landing Page	En tant que visiteur, je veux consulter la page d'accueil de la plateforme	Must
	US-02	Authentification	En tant que visiteur, je veux créer un compte avec vérification d'email	Must
	US-03		En tant qu'utilisateur, je veux me connecter via JWT	Must
	US-04		En tant qu'utilisateur, je veux réinitialiser mon mot de passe	Must
	US-05	Gestion utilisateurs	En tant qu'admin, je veux gérer les comptes et les rôles utilisateurs	Must
Sprint 2 — Gestion des cours et progression				
2	US-06	Cours	En tant qu'apprenant, je veux parcourir le catalogue de cours	Must
	US-06b		En tant qu'apprenant, je veux m'inscrire à un cours	Must
	US-07		En tant qu'apprenant, je veux me désinscrire d'un cours	Must
	US-08	Progression	En tant qu'apprenant, je veux suivre ma progression leçon par leçon	Must
	US-09	Notes	En tant qu'apprenant, je veux prendre des notes personnelles par leçon	Should
	US-10	Flashcards	En tant qu'apprenant, je veux générer des flashcards (SM-2)	Should
Sprint 3 — Dashboard et notifications				

Suite page suivante...

Suite Tableau 2.5

Sprint	ID	Module	User Story	Priorité
3	US-11	Dashboard	En tant qu'apprenant, je veux accéder à mon tableau de bord personnalisé	Must
	US-12	Suivi	En tant qu'apprenant, je veux visualiser mes statistiques d'apprentissage	Must
	US-13	Notifications	En tant qu'apprenant, je veux recevoir des notifications en temps réel	Must
	US-14	Administration	En tant qu'admin, je veux accéder aux statistiques globales de la plateforme	Should
Sprint 4 — Objectifs et Intelligence Artificielle				
4	US-15	Objectifs	En tant qu'apprenant, je veux définir des objectifs journaliers	Must
	US-16	Agent Tuteur IA	En tant qu'apprenant, je veux interagir avec un tuteur IA (RAG + LLM)	Must
	US-17	Agent Quiz IA	En tant qu'apprenant, je veux passer des quiz générés automatiquement par l'IA	Must
	US-18	Feedback IA	En tant qu'apprenant, je veux recevoir un feedback IA détaillé sur mes erreurs	Must
Sprint 5 — Tests et validation				
5	US-19	Tests unitaires	En tant que développeur, je veux des tests unitaires pour chaque microservice	Must
	US-20	Tests d'intégration	En tant que développeur, je veux des tests d'intégration inter-services	Must
	US-21	Tests E2E	En tant que développeur, je veux des tests End-to-End des parcours utilisateurs	Should
Sprint 6 — Déploiement et monitoring				

Suite page suivante...

Suite Tableau 2.5

Sprint	ID	Module	User Story	Priorité
6	US-22	Déploiement	Conteneurisation Docker Compose, pipeline CI/CD, déploiement final	Must

2.3.3 Planification des releases

La gestion du projet a été organisée selon une approche Agile Scrum basée sur des releases incrémentales. Chaque release regroupe plusieurs sprints orientés vers un objectif fonctionnel précis de la plateforme. Le tableau 2.6 présente la répartition des sprints par release.

TABLEAU 2.6 :Planification des releases et des sprints

Release	Sprint	Objectif principal
Release 1 : Fondation de la plateforme	Sprint 1 : Infrastructure et authentification	Mise en place de l'API Gateway, authentification JWT, landing page et gestion des utilisateurs.
	Sprint 2 : Gestion des cours et progression	Gestion des cours, inscription des apprenants, suivi de progression, notes et flashcards.
Release 2 : Fonctionnalités avancées et IA	Sprint 3 : Tableaux de bord et notifications	Dashboard utilisateur et administrateur, suivi d'apprentissage, gestion des notifications et statistiques.
	Sprint 4 : Agents IA et objectifs	Gestion des objectifs, intégration de l'Agent Tuteur IA (RAG), Agent Quiz IA, suivi avancé.
Release 3 : Tests et monitoring	Sprint 5 : Tests et validation	Tests unitaires, tests d'intégration inter-services (RabbitMQ), tests End-to-End.

Suite page suivante...

Suite Tableau 2.6

Release	Sprint	Objectif principal
	Sprint 6 : Déploiement et monitoring	Docker Compose, pipeline CI/CD, monitoring Prometheus et Grafana, déploiement final.

La figure 2.3 illustre leur répartition temporelle.

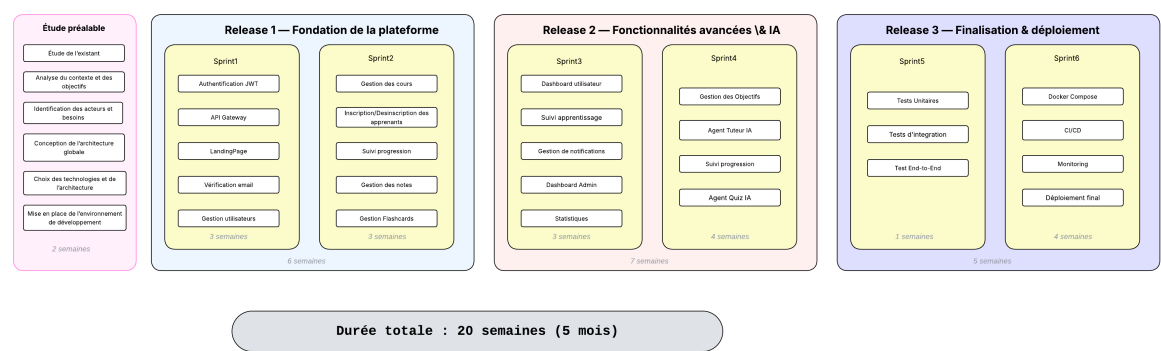


FIGURE 2.3 :Diagramme de planification des releases et sprints

2.4 Patrons de conception utilisés

L'architecture microservices de **Subul Academy** s'appuie sur un ensemble de patrons de conception (*design patterns*) éprouvés, répartis en trois catégories : création, structure et comportement. Le tableau 2.7 présente ces patrons, leur catégorie, leur application concrète dans le projet ainsi que le bénéfice apporté.

TABLEAU 2.7 :Patrons de conception utilisés dans Subul Academy

Patron	Catégorie	Application dans le projet	Bénéfice
Factory Pattern	Création	Instanciation dynamique des clients de services via <code>JwtModule.registerAsync()</code> et <code>ClientsModule.register()</code> selon la configuration d'environnement.	Découple la création des objets de leur utilisation; facilite les tests et le changement de configuration.
Singleton Pattern		Les services NestJS utilisent le scope par défaut, garantissant une instance unique de <code>AuthService</code> , <code>PrismaService</code> et des clients RabbitMQ par processus.	Évite la multiplication des connexions à la base de données et au broker de messages.
API Gateway Pattern	Structurel	Un point d'entrée unique reçoit toutes les requêtes HTTP, les authentifie via les gardes JWT et les route vers le microservice compétent.	Centralise la sécurité et le routage; simplifie le client en exposant une API unifiée.
Repository Pattern		Prisma ORM abstrait l'accès aux données pour chaque microservice, découplant la logique métier du moteur de base de données.	Facilite les tests unitaires (mock du repository) et le remplacement de la couche de persistance.
DTO Pattern		Les <i>Data Transfer Objects</i> définis dans <code>@repo/shared-contracts</code> assurent la validation et la sérialisation des données à chaque frontière de service.	Garantit la cohérence des contrats d'interface entre frontend et microservices.

Suite page suivante...

Suite Tableau 2.7

Patron	Catégorie	Application dans le projet	Bénéfice
Observer Pattern	Comportemental	Le service de notifications utilise Redis pub/sub pour diffuser les événements en temps réel aux clients connectés, sans couplage entre les services émetteurs et récepteurs.	Faible couplage entre producteurs et consommateurs d'événements ; extensible sans modifier les émetteurs.
Chain of Responsibility		Dans l'API Gateway, les requêtes traversent une chaîne de gardes et d'intercepteurs NestJS (authentification JWT, validation des rôles, transformation des réponses) avant d'atteindre le contrôleur.	Sépare chaque préoccupation de sécurité et de transformation en un maillon indépendant.
Strategy Pattern		Le service d'authentification utilise Passport.js avec plusieurs stratégies interchangeables (JWT, locale) pour valider les identités selon le contexte.	Permet d'ajouter de nouvelles stratégies d'authentification sans modifier le service existant.

2.5 Choix technologiques

Le choix des technologies repose sur trois critères principaux : la **scalabilité** (capacité à supporter la croissance du nombre d'utilisateurs), la **maintenabilité** (code structuré, typé et documenté) et l'**adéquation** avec l'architecture microservices adoptée.

2.5.1 Outils de gestion de projet

ClickUp

ClickUp a été utilisé comme outil principal de gestion des tâches. Il nous a permis d'organiser le travail de manière structurée, de répartir les responsabilités au sein de l'équipe et de suivre l'avancement du projet selon une approche agile.



FIGURE 2.4 :Logo ClickUp

Microsoft Teams

Microsoft Teams a été adopté comme outil de communication au sein de l'équipe. Il a facilité l'organisation des réunions, les échanges en temps réel ainsi que le partage de documents et d'informations liées au projet.



FIGURE 2.5 :Logo Teams

2.5.2 Outils de conception

Lucidchart est la plateforme de modélisation retenue pour la conception des diagrammes du projet.

Lucidchart

Lucidchart est une plateforme en ligne dédiée à la modélisation et à la conception de diagrammes. Elle permet de réaliser différents types de représentations, notamment des diagrammes UML, des organigrammes, des architectures systèmes ainsi que divers schémas de processus.

Son interface intuitive, basée sur le glisser-déposer, facilite la création et l'organisation des éléments graphiques, même pour des modèles complexes.

L'outil se distingue également par ses fonctionnalités de collaboration en temps réel, ce qui permet à plusieurs membres d'une équipe de travailler simultanément sur les mêmes diagrammes, ce qui est particulièrement adapté aux projets académiques et professionnels.



FIGURE 2.6 :Logo LucidChart

Lucidchart est largement utilisé durant les phases de conception, car il permet de clarifier les besoins du système et de représenter de manière visuelle son architecture globale.

2.5.3 Outils de développement

Le tableau 2.8 recense les outils transversaux utilisés tout au long du cycle de développement.

TABLEAU 2.8 :Outils de développement et DevOps

Outil	Logo	Rôle
VS Code		Éditeur de code principal utilisé pour le développement, offrant un environnement léger, extensible et adapté au développement web et backend.
Git / GitHub		Gestion du code source, versioning, gestion des branches et collaboration via Pull Requests.
Docker		Conteneurisation des services afin d'assurer la portabilité et la cohérence des environnements.
Turborepo		Système de monorepo permettant l'optimisation des builds grâce à un cache intelligent.
Jest		Framework de tests unitaires et d'intégration pour le backend.
Playwright		Tests end-to-end (E2E) pour valider les fonctionnalités du frontend.
Postman		Outil de test et de validation des API REST, permettant d'envoyer des requêtes, analyser les réponses et documenter les endpoints.

2.5.4 Frameworks et langages de développement

Le choix des technologies utilisées dans le cadre du projet Subul Academy a été guidé par plusieurs critères, notamment la maturité de l'écosystème, la compatibilité avec l'architecture microservices adoptée ainsi que la productivité offerte en termes de développement, de maintenabilité et de performance.

Frontend

Le tableau 2.9 présente les technologies retenues pour le développement du frontend de la plateforme.

TABLEAU 2.9 :Technologies du Frontend

Logo	Technologie	Description et justification
	Next.js	Framework React full-stack développé par Vercel. Il intègre le rendu côté serveur (SSR), la génération statique (SSG) ainsi que l'App Router. Il a été retenu pour ses performances élevées, son optimisation SEO et son architecture modulaire.
	TypeScript	Sur-ensemble typé de JavaScript apportant un système de types statiques. Son utilisation garantit la cohérence des contrats entre le frontend et les microservices backend tout en réduisant les erreurs d'intégration.
	Tailwind CSS	Framework CSS utilitaire permettant de construire rapidement des interfaces modernes et cohérentes grâce à des classes prédéfinies et un système de design personnalisable.

Backend

Le tableau 2.10 présente les technologies retenues pour le développement des microservices backend.

TABLEAU 2.10 :Technologies du Backend

Logo	Technologie	Description et justification
	NestJS	Framework Node.js progressif basé sur TypeScript. Son architecture modulaire, son système d'injection de dépendances et son support natif des microservices facilitent le développement et la maintenance des services métier.
	Prisma ORM	ORM moderne pour TypeScript permettant la génération automatique d'un client typé ainsi que la gestion des migrations. Chaque microservice dispose de son propre schéma Prisma afin de respecter l'isolation des données.

Bases de données et communication

Le tableau 2.11 présente les technologies retenues pour le stockage des données et la communication inter-services.

TABLEAU 2.11 :Technologies de bases de données et de communication

Logo	Technologie	Description et justification
	PostgreSQL	Système de gestion de bases de données relationnelles open source reconnu pour sa robustesse et sa conformité SQL. Chaque microservice possède sa propre base afin d'assurer l'indépendance des données et la scalabilité du système.

Tableau 2.11 — suite

Logo	Technologie	Description et justification
	RabbitMQ	Broker de messages basé sur le protocole AMQP permettant la communication asynchrone entre les microservices. Il assure un faible couplage entre les services et améliore la résilience globale de l'architecture.

Service d'Intelligence Artificielle

Le service IA repose principalement sur **FastAPI** (Python), **LangChain** / **LangGraph** pour l'orchestration des agents, et **OpenAI GPT-4** pour la génération de contenu. Le détail complet de ce service est présenté dans le **Chapitre 4**.

2.6 Architecture de l'application

L'architecture de **Subul Academy** a été conçue selon un principe strict de **séparation des responsabilités**, garantissant une modularité élevée, une scalabilité horizontale et une maintenance facilitée. Chaque composant est indépendant, faiblement couplé et déployable séparément.

2.6.1 Architecture logique (Microservices)

La plateforme repose sur une architecture microservices [4] où chaque domaine métier est encapsulé dans un service autonome. Les échanges inter-services sont gérés via un broker de messages centralisé (**RabbitMQ**).

Le tableau 2.12 présente la répartition des microservices et leurs responsabilités fonctionnelles.

TABLEAU 2.12 :Répartition des microservices et responsabilités fonctionnelles

Service	Responsabilités
API Gateway	Point d'entrée unique, authentification JWT, validation des requêtes et routage.
Authentification	Gestion des comptes, JWT (access/refresh), vérification email.
Utilisateur	Gestion des profils, informations personnelles et avatars.
Cours	Gestion des cours, modules, leçons, progression et flashcards.
Objectifs	Suivi des objectifs journaliers et hebdomadaires.
Apprenant	Tableau de bord et suivi personnalisé de progression.
Notifications	Notifications temps réel via Redis pub/sub.
Intelligence Artificielle	Agent tuteur (RAG + GPT-4) et génération automatique de quiz.

La figure 2.7 illustre l'architecture logique complète de la plateforme et les interactions entre les différents microservices.

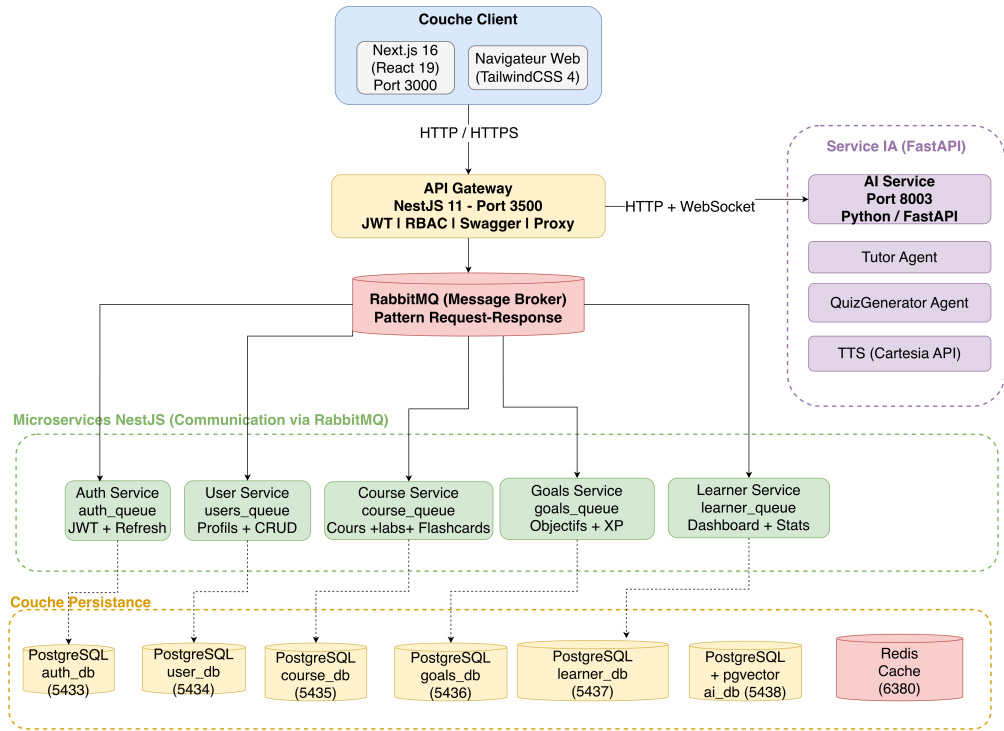


FIGURE 2.7 :Architecture logique microservices de Subul Academy

Les requêtes des clients transitent exclusivement par l'**API Gateway**, qui constitue le seul point d'entrée du système et assure l'authentification JWT, la validation et le routage vers le microservice approprié. Chaque microservice est **isolé** et **indépendant** : il possède sa propre base de données PostgreSQL, garantissant qu'aucune donnée n'est partagée directement entre services. La coordination entre ces services repose sur **RabbitMQ** comme broker de messages central, permettant une communication asynchrone fiable via des files dédiées. Le **Service IA**, développé en Python avec FastAPI, se distingue des autres services par son recours à des technologies spécialisées — LangGraph, pgvector [5] et l'API OpenAI GPT-4 — pour offrir un accompagnement pédagogique intelligent. L'ensemble de l'infrastructure est conteneurisé via **Docker** et orchestré avec **Docker Compose**, tandis que **Prometheus** et **Grafana** assurent la surveillance continue des performances du système.

2.6.2 Architecture physique (Déploiement)

Chaque microservice est conteneurisé via **Docker** et orchestré avec **Docker Compose**, assurant portabilité et isolation.

Le tableau 2.13 récapitule l'ensemble des conteneurs déployés dans l'infrastructure.

TABLEAU 2.13 :Infrastructure de déploiement basée sur Docker Compose

Conteneur	Image	Rôle
subul-rabbitmq	rabbitmq :3-management	Broker de messages central (AMQP)
subul-postgres-auth	postgres :15	Base de données Authentification
subul-postgres-user	postgres :15	Base de données Utilisateur
subul-postgres-course	postgres :15	Base de données Cours
subul-postgres-goals	postgres :15	Base de données Objectifs
subul-postgres-ai	postgres :15 + pgvector	Base vectorielle du service IA
subul-redis-ai	redis :7-alpine	Cache des sessions IA

Suite page suivante...

Suite Tableau 2.13

Conteneur	Image	Rôle
subul-redis-notifications	redis :7-alpine	Pub/Sub notifications temps réel
subul-prometheus	prom/prometheus :v2.55.1	Monitoring des métriques
subul-grafana	grafana/grafana :11.3.0	Visualisation des dashboards

2.6.3 Communication inter-services (RabbitMQ)

Les microservices communiquent via **AMQP (RabbitMQ)**, permettant un modèle asynchrone et résilient.

Le tableau 2.14 décrit les deux modes de communication utilisés entre les services.

TABLEAU 2.14 :Modes de communication inter-services

Mode	Description
RPC (Request/Response)	Communication synchrone avec réponse immédiate (ex : login JWT).
Event (Fire & Forget)	Communication asynchrone basée sur événements (ex : notification email).

Chaque service possède sa file dédiée : `auth_queue`, `user_queue`, etc.

2.6.4 Architecture du Service IA

Vue d'ensemble

Le service d'intelligence artificielle constitue le **cœur différenciateur** de la plateforme **Subul Academy**. Développé en **Python** avec **FastAPI** et déployé dans un conteneur Docker indépendant, il s'intègre à l'écosystème microservices via deux canaux : des appels **HTTP synchrones** depuis l'API Gateway, et une communication **asynchrone via RabbitMQ** pour les événements système. Il expose deux endpoints spécialisés :

- POST /ai/chat : dialogue pédagogique de l’**Agent Tuteur**, qui répond aux questions de l’apprenant en s’appuyant sur le contenu des cours (système RAG) et l’historique de conversation.
- POST /ai/quiz : génération automatique de quiz par l’**Agent de Quiz**, qui produit des questions adaptées au niveau de l’apprenant et analyse ses réponses pour fournir un feedback personnalisé.

Architecture interne (Agents, LLM, base vectorielle)

L’architecture interne du service IA repose sur trois couches complémentaires, chacune jouant un rôle précis dans le traitement des requêtes pédagogiques.

Le tableau 2.15 décrit ces trois couches ainsi que les technologies associées.

TABLEAU 2.15 :Architecture interne du service d’intelligence artificielle

Couche	Technologie	Description
LLM	OpenAI GPT-4	Génération de contenu pédagogique et de réponses contextualisées à partir des informations fournies par le système RAG et l’historique de conversation.
Orchestration des agents	LangChain + LangGraph	Modélisation de l’Agent Tuteur et de l’Agent Quiz sous forme de graphes d’états permettant l’enchaînement des étapes de raisonnement, de recherche et de génération.
Recherche vectorielle (RAG)	PostgreSQL + pgvector	Stockage des embeddings des cours et récupération des passages les plus pertinents via une recherche par similarité cosinus.

La figure 2.8 illustre l’articulation entre les trois couches du service IA et le flux de traitement des requêtes pédagogiques.

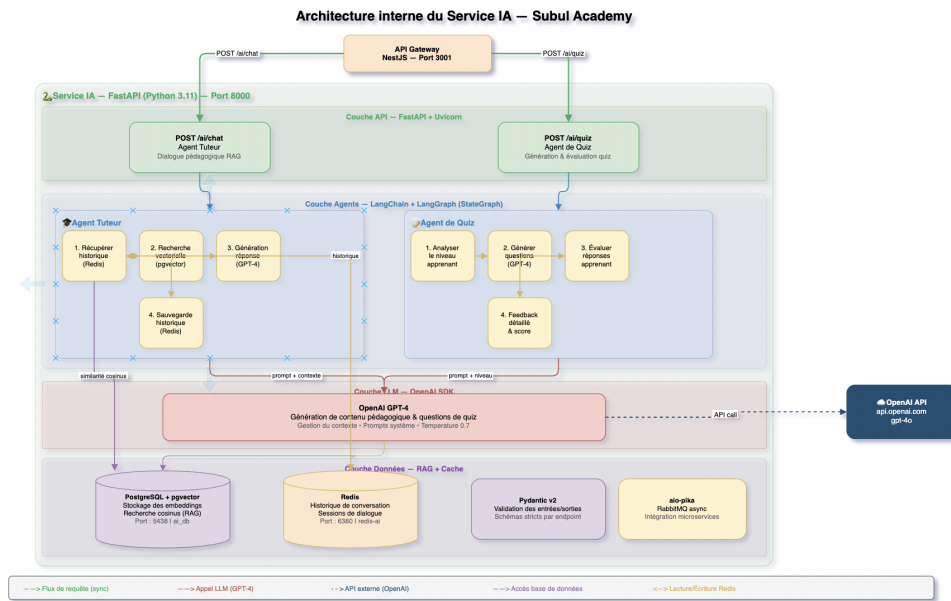


FIGURE 2.8 :Architecture interne du service IA

L'Agent Tuteur maintient un historique de conversation persisté dans **Redis** et sélectionne dynamiquement les outils à invoquer selon la nature de la requête. L'Agent de Quiz, quant à lui, opère de manière autonome en générant des séries de questions structurées et en évaluant les réponses de l'apprenant pour produire un retour pédagogique détaillé.

Flux de traitement d'une requête IA

Le flux de traitement d'une question soumise par l'apprenant suit les étapes suivantes :

1. L'apprenant soumet une question via l'interface web (**Frontend Next.js**).
2. L'**API Gateway** authentifie la requête (JWT) et la transmet au **Service IA** via HTTP.
3. Le service récupère l'**historique de conversation** depuis **Redis** afin de maintenir la continuité du dialogue.
4. L'**agent LangGraph** déclenche une recherche par similarité cosinus dans **pgvector** afin d'extraire les passages du cours les plus pertinents (*Retrieval-Augmented Generation*).
5. Le contexte enrichi (question, historique et extraits du cours) est transmis au modèle **GPT-4**, qui génère la réponse pédagogique.

6. La réponse est mise en cache dans **Redis**, enregistrée dans l'historique de conversation puis renvoyée à l'apprenant via l'API Gateway.

Technologies spécifiques au service IA

Le tableau 2.16 détaille les technologies utilisées au sein du service d'intelligence artificielle et leur rôle respectif.

TABLEAU 2.16 :Technologies utilisées dans le service d'intelligence artificielle

Technologie	Rôle dans le service IA
FastAPI + Uvicorn	Exposition des endpoints REST du service IA avec prise en charge asynchrone et documentation OpenAPI automatique.
LangChain	Gestion des interactions avec les modèles de langage, des prompts et des chaînes de traitement.
LangGraph	Orchestration des agents intelligents sous forme de graphes d'états et gestion des flux décisionnels.
OpenAI GPT-4	Génération de contenu pédagogique, réponses conversationnelles et création automatique de quiz.
pgvector	Stockage vectoriel et recherche sémantique des contenus pédagogiques dans le cadre du système RAG.
Redis	Persistance des sessions utilisateur, historique des conversations et mécanisme de cache.
Pydantic	Validation et sérialisation des données échangées entre les composants du service.
aio-pika	Communication asynchrone avec RabbitMQ pour l'intégration dans l'architecture microservices.

Conclusion

Ce chapitre a défini les besoins, la planification Scrum, les choix technologiques et l'architecture microservices de la plateforme Subul.

Chapitre 3

Release 1 – Fondation de la plateforme

Sommaire du chapitre

1. Introduction
2. Sprint 1 : Infrastructure et authentification
 - Objectifs du sprint 1
 - Backlog du sprint 1
 - Spécification et conception du sprint
 - Diagramme de séquence
 - Réalisation
3. Sprint 2 : Gestion des cours et progression
 - Objectifs du sprint 2
 - Backlog du sprint 2
 - Spécification et conception du sprint
 - Diagramme de séquence
 - Réalisation
4. Conclusion

Introduction

Ce chapitre présente la réalisation de la **Release 1 — Fondation de la plateforme**, composée du Sprint 1 (infrastructure et authentification) et du Sprint 2 (gestion des cours et progression). Pour chaque sprint, nous détaillons les objectifs, le backlog, les diagrammes et les interfaces réalisées.

3.1 Sprint 1 : Infrastructure et authentification

3.1.1 Objectifs du sprint 1

Le Sprint 1 a pour objectif de mettre en place les fondations techniques de la plateforme Subul. Il s’agit de la première itération du projet, dont les livraisons constituent des prérequis indispensables au bon fonctionnement de l’ensemble des services.

Les principaux objectifs de ce sprint sont :

- Mise en place de l’**API Gateway** comme point d’entrée unique pour tous les microservices [4] ;
- Implémentation du système d’**authentification JWT** [6] (inscription, connexion, refresh token, déconnexion) ;
- Intégration de la **vérification d’email** à l’inscription ;
- Développement de la **gestion des utilisateurs** (profils, rôles, administration) ;
- Création de la **Landing Page** présentant la plateforme.

3.1.2 Backlog du sprint 1

Le tableau 3.1 présente l’ensemble des user stories réalisées durant le Sprint 1.

TABLEAU 3.1 :Backlog du Sprint 1 — Release 1

UC	Cas d’usage	Description	Est.
UC1	Authentification JWT	En tant qu’utilisateur, je veux me connecter et obtenir un access token et un refresh token sécurisés	5j

UC	Cas d'usage	Description	Est.
UC2	API Gateway	En tant que système, je veux que toutes les requêtes transitent par une passerelle centralisée gérant le routage et la sécurité	4j
UC3	Landing Page	En tant que visiteur, je veux consulter la page d'accueil de la plateforme	2j
UC4	Vérification email	En tant que visiteur, je veux créer un compte avec vérification d'email avant l'activation	3j
UC5a	Gestion utilisateurs	En tant qu'admin, je veux gérer les comptes des utilisateurs de la plateforme	2j
UC5b		En tant qu'admin, je veux gérer les rôles des utilisateurs de la plateforme	2j
UC6a	Gestion du profil	En tant qu'apprenant, je veux consulter mes informations personnelles après authentification	1j
UC6b		En tant qu'apprenant, je veux modifier mes informations personnelles après authentification	1j

3.1.3 Spécification et conception du sprint

Diagramme de cas d'utilisation

La figure 3.1 présente le diagramme de cas d'utilisation du Sprint 1, illustrant les interactions entre les trois acteurs (Visiteur, Apprenant, Administrateur) et les fonctionnalités développées durant cette itération.

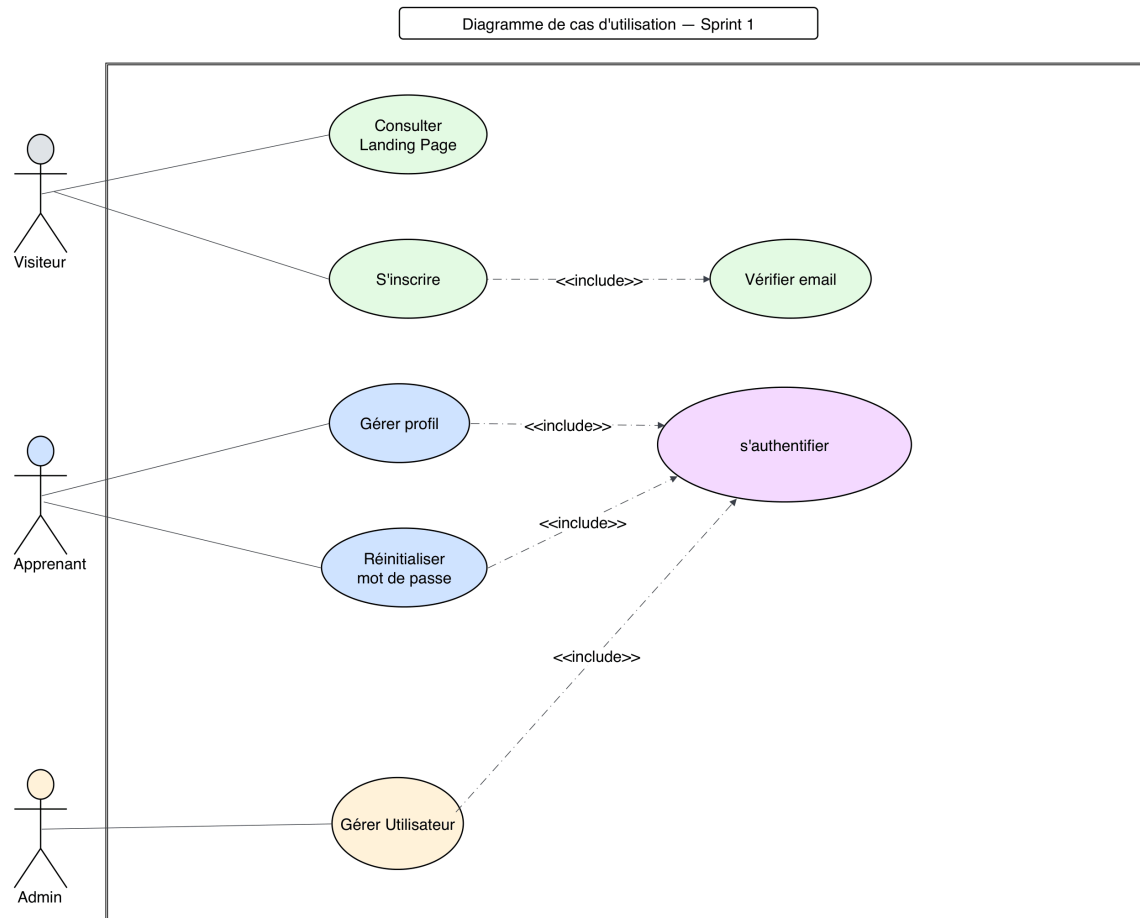


FIGURE 3.1 :Diagramme de cas d'utilisation — Sprint 1

Visiteur

- **Consulter la Landing Page** : le visiteur accède à la page d'accueil de la plateforme sans authentification.
- **S'inscrire** : le visiteur crée un compte. Ce cas inclut obligatoirement («*include*») la **vérification d'email** afin de valider l'adresse fournie avant l'activation du compte.

Apprenant L'apprenant dispose des fonctionnalités du Visiteur et de deux cas supplémentaires nécessitant une authentification préalable («*include*») :

- **Gérer son profil** : consulter et modifier ses informations personnelles (photo, nom, email).

Administrateur L'administrateur dispose d'un accès dédié à la gestion des utilisateurs, conditionné par une authentification préalable («*include*») :

- **Gérer les utilisateurs** : ce cas d'utilisation regroupe plusieurs extensions («*extend*») : lister, ajouter, modifier et supprimer des utilisateurs, modifier leur rôle, ainsi que **changer leur statut** (activer ou désactiver le compte).

Diagrammes de raffinement des cas d'utilisation

Les figures suivantes présentent les diagrammes de raffinement des trois cas d'utilisation principaux du Sprint 1, détaillant leurs sous-cas et les relations entre acteurs.

Diagramme de raffinement du cas d'utilisation « UC1 — S'inscrire »

La figure 3.2 détaille le cas d'utilisation **S'inscrire**. Lorsque le visiteur soumet le formulaire d'inscription, le système déclenche obligatoirement («*include*») la **vérification email**, qui elle-même inclut l'envoi d'un email de vérification via le **Service Email** (acteur système). Une fois le lien cliqué, le sous-cas **Activer le compte** est exécuté.

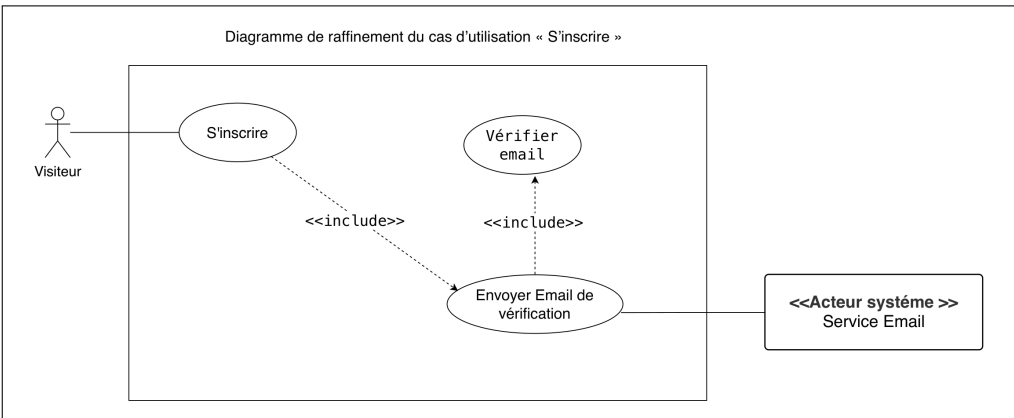


FIGURE 3.2 :Diagramme de raffinement — UC1 : S'inscrire

Le tableau 3.2 décrit le cas d'utilisation **S'inscrire**, détaillant les acteurs impliqués, les préconditions, le scénario principal ainsi que les scénarios alternatifs.

TABLEAU 3.2 :Description textuelle — UC1 : S'inscrire

Champ	Détail
Intitulé	S'inscrire

Suite page suivante...

Suite Tableau 3.2

Champ	Détail
Acteurs	Visiteur (principal), Service Email (système)
Préconditions	Le visiteur n'est pas connecté et ne possède pas de compte
Postconditions	Compte créé et activé, l'utilisateur peut se connecter
Scénario principal	1. Le visiteur remplit le formulaire (nom, email, mot de passe) 2. Le système valide les données saisies 3. Le système envoie un email de vérification («include») 4. Le visiteur clique sur le lien reçu 5. Le système active le compte
Scénario alternatif	3a. Email déjà utilisé → message d'erreur 3b. Mot de passe trop faible → message d'erreur 4a. Lien expiré → option de renvoi d'un nouvel email

Diagramme de raffinement du cas d'utilisation « UC4 — Gérer utilisateurs »

La figure 3.3 détaille le cas d'utilisation **Gérer utilisateurs** réservé à l'Administrateur. Ce cas central est étendu («*extend*») par six sous-cas : lister, ajouter, modifier et supprimer un utilisateur, modifier son rôle, ainsi que **Changer le statut**, qui se décompose à son tour en deux alternatives : **Activer le compte** et **Désactiver le compte**.

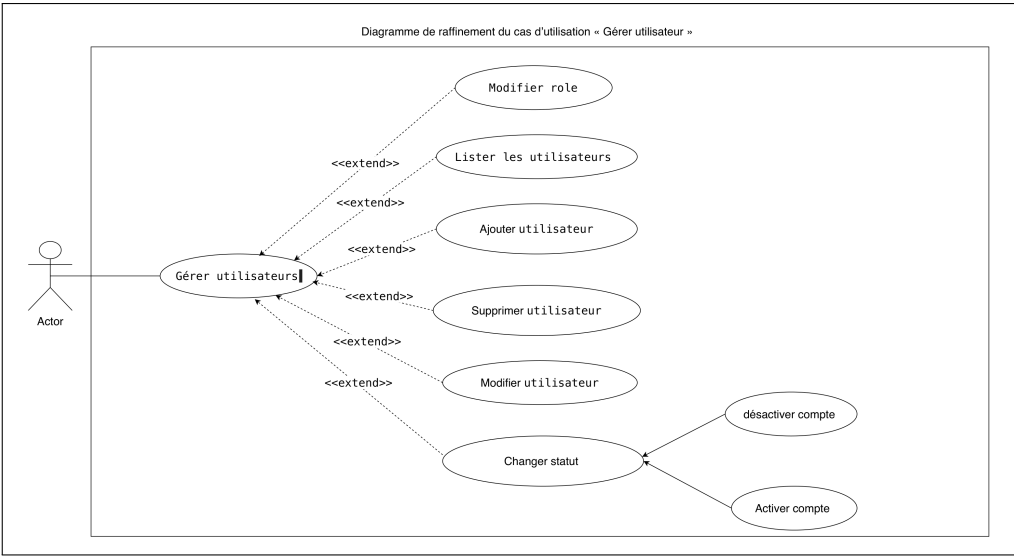


FIGURE 3.3 :Diagramme de raffinement — UC4 : Gérer utilisateurs

Le tableau 3.3 décrit le cas d'utilisation **Gérer utilisateurs**, en présentant l'ensemble des actions disponibles pour l'administrateur et les scénarios alternatifs associés.

TABEAU 3.3 :Description textuelle — UC4 : Gérer utilisateurs

Champ	Détail
Intitulé	Gérer utilisateurs
Acteurs	Administrateur (principal)
Préconditions	L'administrateur est authentifié
Postconditions	L'action choisie est effectuée et les données sont mises à jour
Scénario principal	1. L'admin accède au panneau de gestion des utilisateurs 2. Il consulte la liste des utilisateurs («extend») 3. Il sélectionne une action : ajouter, modifier, supprimer, changer rôle ou statut 4. Le système exécute l'action et confirme

Suite page suivante...

Suite Tableau 3.3

Champ	Détail
Scénario alternatif	3a. Suppression d'un compte actif → confirmation requise 3b. Changement de statut → choix entre Activer ou Désactiver le compte

Diagramme de raffinement du cas d'utilisation « UC6 — Gérer profil »

La figure 3.4 détaille le cas d'utilisation **Gérer profil** destiné à l'Apprenant. Ce cas inclut obligatoirement («*include*») la **consultation du profil** ainsi que l'authentification préalable. La modification de la photo de profil constitue une extension optionnelle («*extend*»).

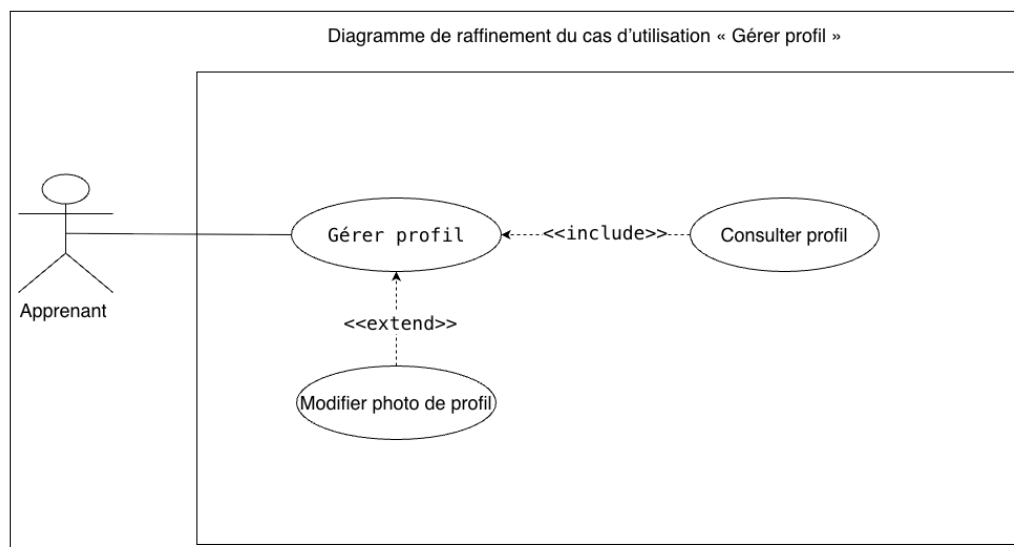


FIGURE 3.4 :Diagramme de raffinement — UC6 : Gérer profil

Le tableau 3.4 décrit le cas d'utilisation **Gérer profil**, précisant les interactions de l'apprenant avec son espace personnel ainsi que les extensions optionnelles.

TABLEAU 3.4 :Description textuelle — UC6 : Gérer profil

Champ	Détail
Intitulé	Gérer profil
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est authentifié
Postconditions	Les informations du profil sont mises à jour
Scénario principal	1. L'apprenant accède à son profil («include» Consulter profil) 2. Il modifie ses informations personnelles (nom, email) 3. Il sauvegarde les modifications 4. Le système confirme la mise à jour
Scénario alternatif	2a. L'apprenant choisit de modifier sa photo de profil («extend») 2b. Email déjà utilisé → message d'erreur

Diagramme de séquence

Dans le cadre du Sprint 1, trois diagrammes de séquence ont été élaborés afin de détailler les flux principaux du système d'authentification et de gestion des utilisateurs.

Diagramme de séquence : Inscription et vérification email

La figure 3.5 illustre le flux d'inscription d'un nouveau visiteur sur la plateforme Subul.

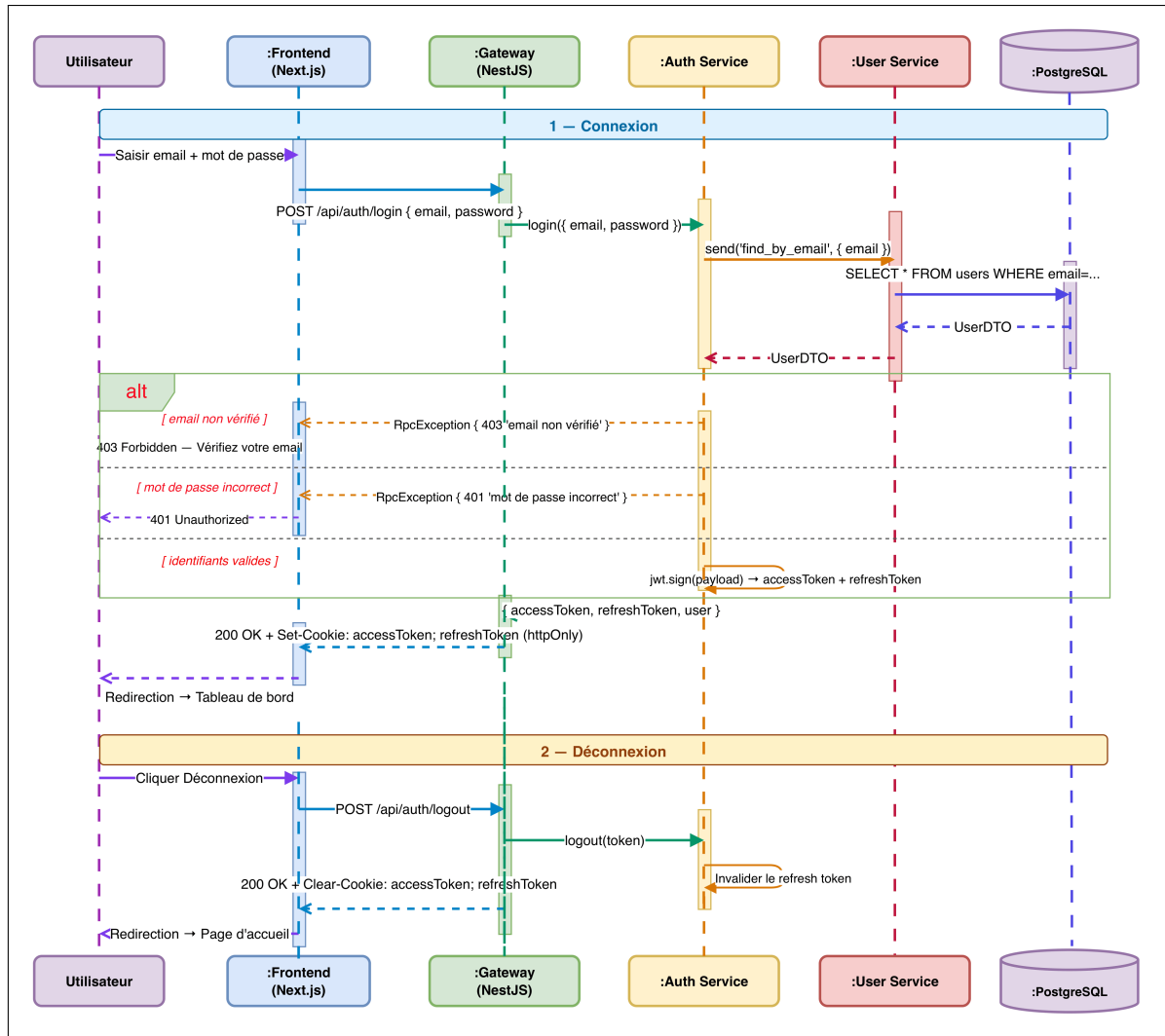


FIGURE 3.5 :Diagramme de séquence — Inscription et vérification email

Le diagramme montre que lors de la soumission du formulaire, le service d'authentification vérifie d'abord la disponibilité de l'email dans la base de données. Si l'email est disponible (*alt [email disponible]*), le mot de passe est haché, le compte est créé avec le statut `isVerified=false`, et un email de confirmation est envoyé. Dans le cas contraire (*alt [email déjà utilisé]*), une erreur **409 Conflict** est retournée. Une fois le lien cliqué, le token est validé et le compte est activé (`isVerified=true`).

Diagramme de séquence : Authentification JWT

La figure 3.6 présente le flux d'authentification JWT, incluant la connexion, le renouvellement du token d'accès et la déconnexion.

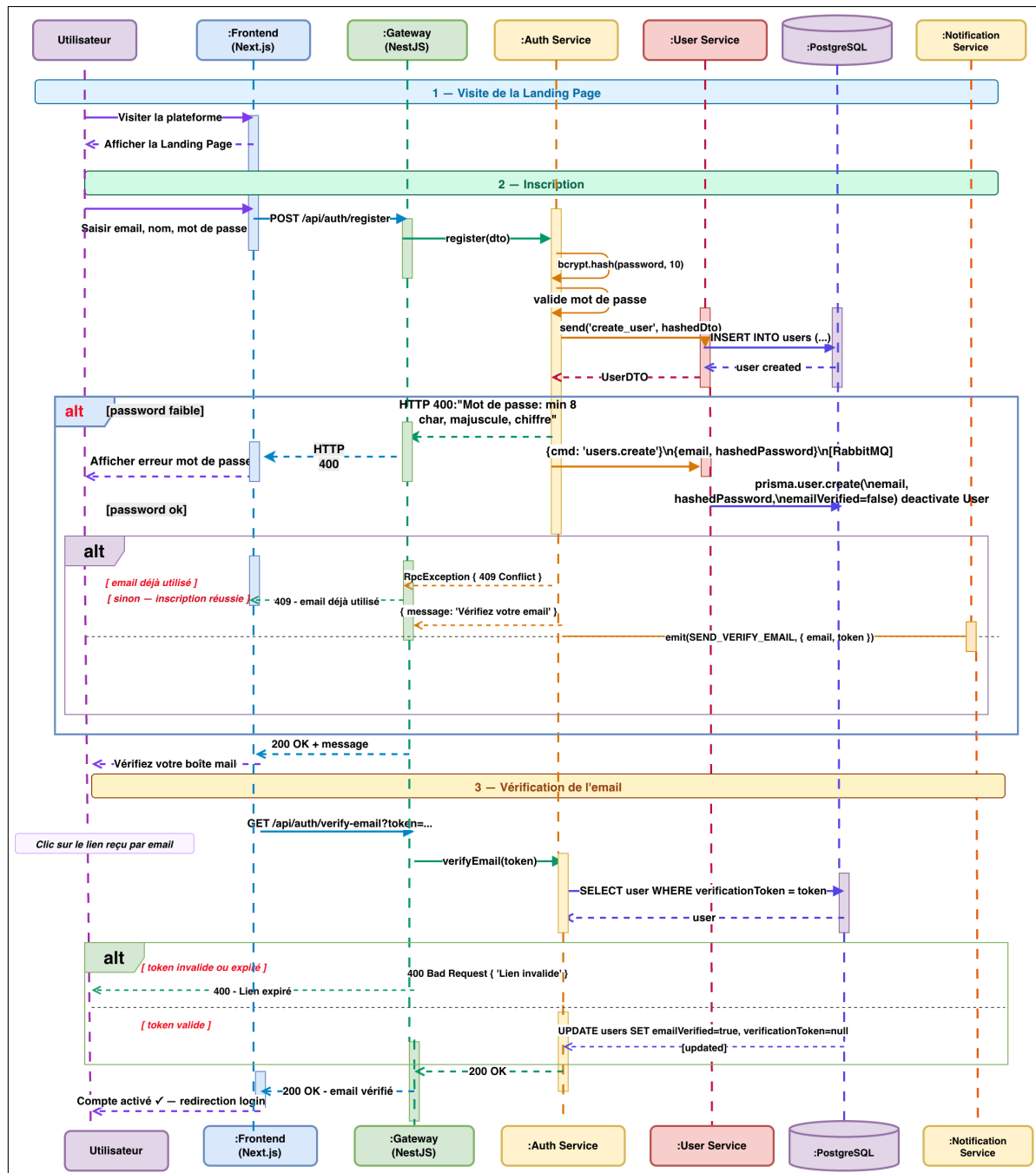


FIGURE 3.6 :Diagramme de séquence — Authentification JWT

Lors de la connexion, le service Auth vérifie les identifiants via la base de données. Si le mot de passe est correct (*alt [password correct]*), un **access token** (durée de vie : 15 min) et un **refresh token** (durée de vie : 7 jours) sont générés, ce dernier étant stocké dans Redis. En cas d'échec (*alt [password incorrect]*), une erreur 401 **Unauthorized** est retournée. Lorsque l'access token expire, le client peut le renouveler via le refresh token (*alt [token valide]*) ou être redirigé vers la page de connexion si ce dernier est expiré (*alt [token expiré ou invalide]*). La déconnexion supprime le refresh token de Redis.

Diagramme de séquence : Gestion des utilisateurs (Admin)

La figure 3.7 illustre le diagramme de séquence de la gestion des utilisateurs par l'administrateur.

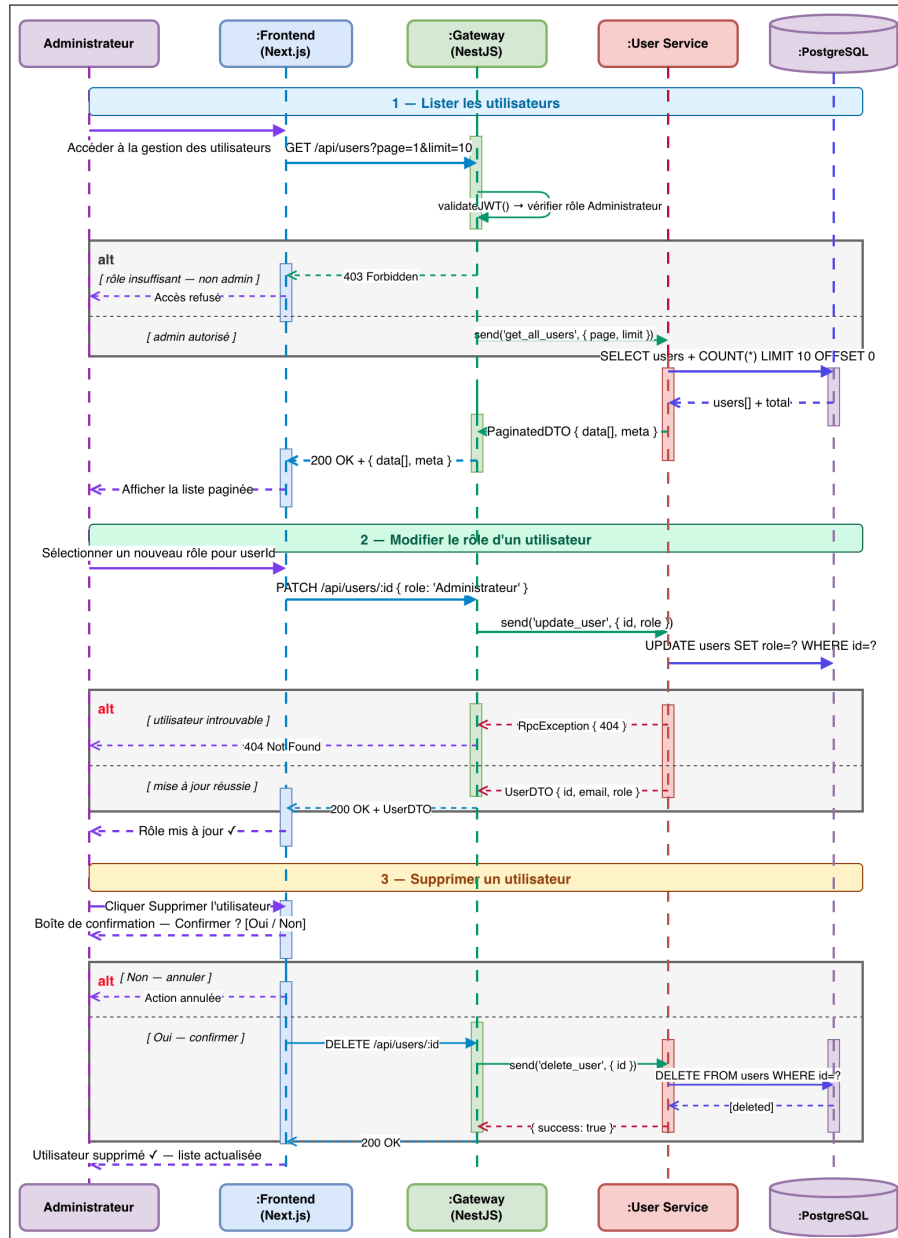


FIGURE 3.7 :Diagramme de séquence — Gestion des utilisateurs (Admin)

Toutes les opérations de gestion des utilisateurs nécessitent une authentification préalable via JWT, vérifiée par l'API Gateway avant tout routage vers le service utilisateur. L'administrateur peut **recupérer la liste** des utilisateurs (GET /users), **ajouter** un nouvel utilisateur (POST /users — réponse 201 Created), et **changer le statut** d'un compte via PATCH /users/:id/status, ce qui permet d'activer ou de désactiver un compte. Chaque opération implique une interaction directe avec la base

de données via le service utilisateur.

3.1.4 Réalisation

Cette section présente les interfaces développées durant le Sprint 1, couvrant le parcours complet du visiteur depuis la découverte de la plateforme jusqu'à l'authentification et la gestion du profil.

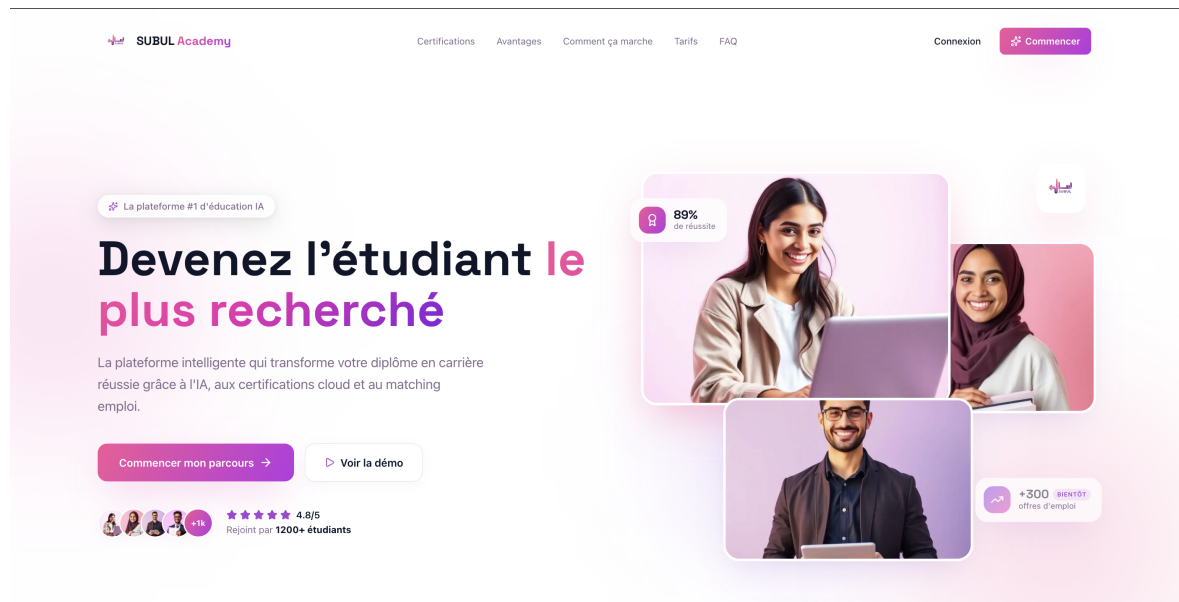


FIGURE 3.8 :Page d'accueil de la plateforme Subul

La page d'accueil présente la plateforme Subul au visiteur, mettant en avant ses fonctionnalités principales et invitant à s'inscrire ou à se connecter.

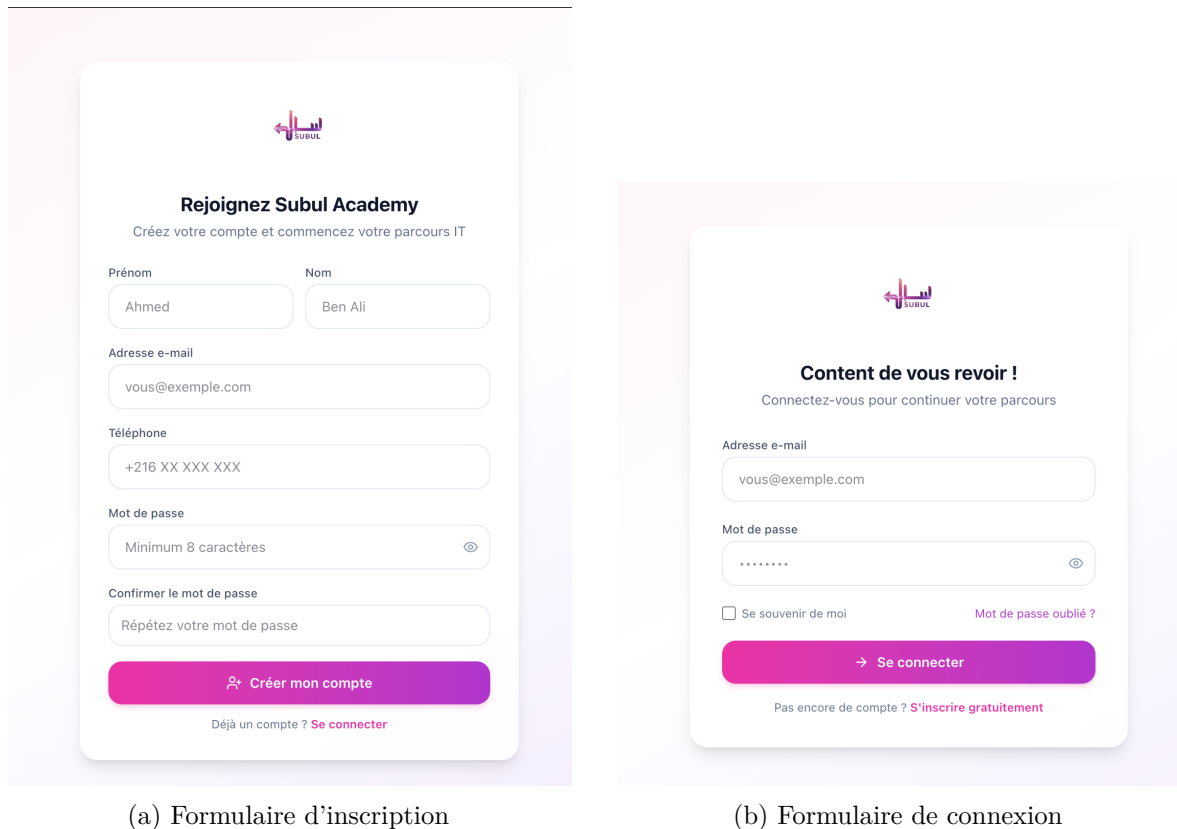


FIGURE 3.9 :Interfaces d'inscription et de connexion

Le formulaire d'inscription permet au visiteur de créer un compte en renseignant son nom, son email et son mot de passe. Le formulaire de connexion permet à l'utilisateur de s'authentifier via ses identifiants pour accéder à son espace personnel.

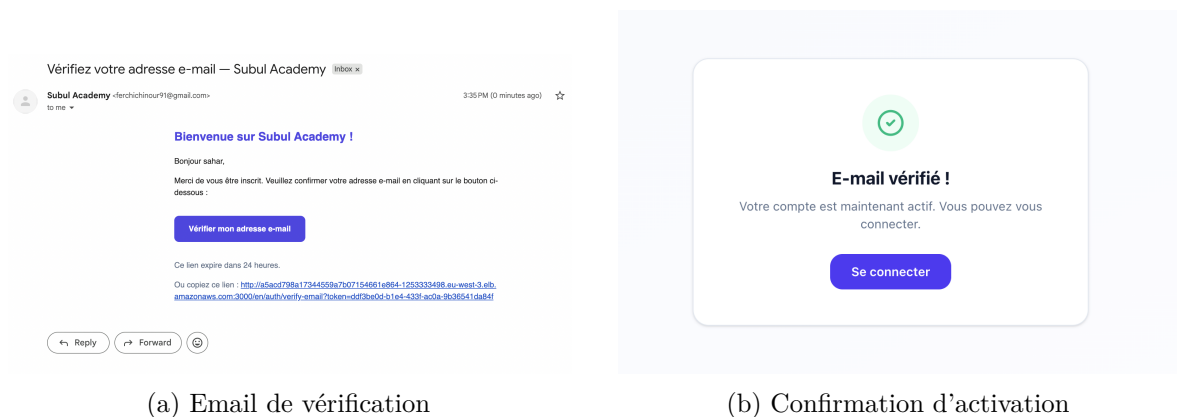


FIGURE 3.10 :Processus de vérification de l'email

Après l'inscription, un email contenant un lien de vérification est envoyé à l'utilisateur. Une fois le lien cliqué, la page de confirmation indique que le compte a été activé avec succès et invite l'utilisateur à se connecter.

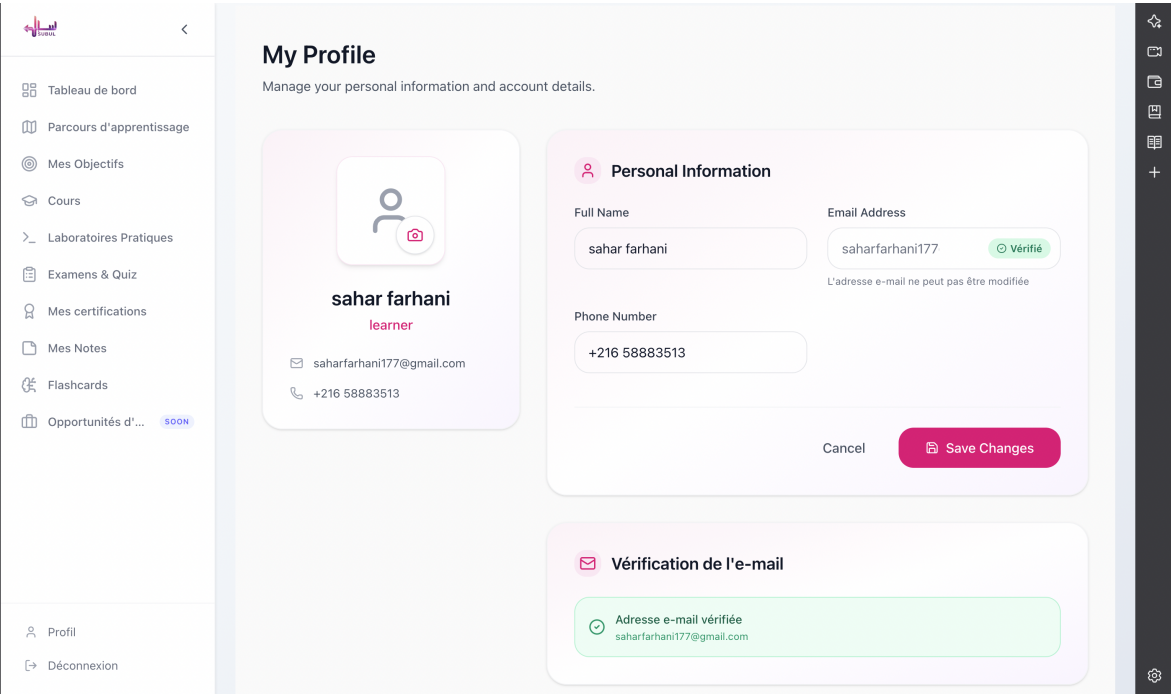


FIGURE 3.11 :Page de profil de l'utilisateur

La page de profil permet à l'utilisateur de consulter et de modifier ses informations personnelles (nom, email, photo de profil), offrant ainsi une gestion complète de son compte sur la plateforme.

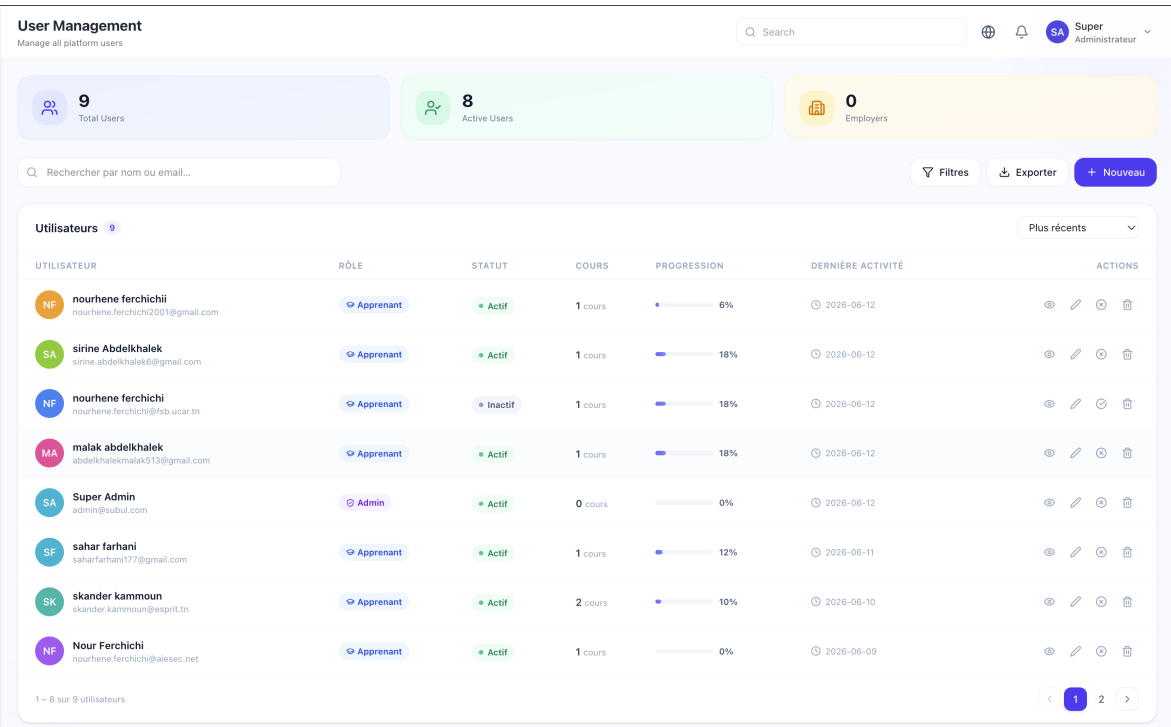


FIGURE 3.12 :Interface d'administration — Liste des utilisateurs

Le tableau de bord administrateur affiche la liste complète des utilisateurs enregistrés

sur la plateforme, avec leurs informations essentielles (nom, email, rôle, statut du compte). L'administrateur dispose d'un accès centralisé pour gérer l'ensemble des comptes depuis cette interface.

Ajouter un utilisateur

Nom complet

Nom complet

Email

Please fill out this field.

email@example.com

Téléphone

+216 XX XXX XXX

Rôle

Apprenant

Mot de passe

.....

Annuler

Créer

ACTIONS

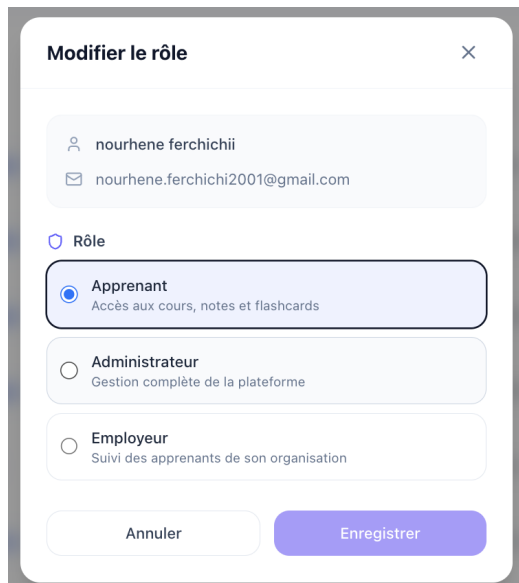
Modifier

(a) Formulaire d'ajout d'un utilisateur (b) Modification des informations d'un utilisateur

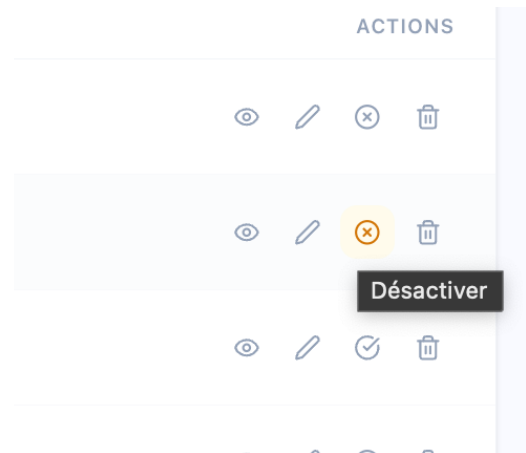
FIGURE 3.13 :Gestion des comptes utilisateurs par l'administrateur

L'administrateur peut créer manuellement un nouveau compte utilisateur en renseignant les informations requises (nom, email, rôle). Il peut également modifier les informations d'un compte existant, notamment le rôle attribué (*Apprenant* ou *Administrateur*).

52



(a) Attribution des rôles



(b) Activation et désactivation d'un compte

FIGURE 3.14 :Gestion des rôles et des statuts des comptes

La gestion des rôles permet à l'administrateur de définir les droits d'accès de chaque utilisateur au sein de la plateforme. Le contrôle du statut des comptes lui offre la possibilité d'activer ou de désactiver un compte utilisateur, assurant ainsi une administration sécurisée et maîtrisée de la communauté.

3.2 Sprint 2 : Gestion des cours et progression

3.2.1 Objectifs du sprint 2

Le Sprint 2 s'appuie sur les fondations établies lors du Sprint 1 pour introduire les fonctionnalités pédagogiques essentielles de la plateforme Subul. Ce sprint constitue le cœur du parcours apprenant, en lui permettant d'accéder aux contenus de formation et de suivre son évolution.

Les principaux objectifs de ce sprint sont :

- Mise en place du **catalogue de cours** avec recherche et filtres par domaine et niveau ;
- Développement du système d'**inscription et de désinscription** aux cours ;
- Implémentation du **suivi de progression** leçon par leçon ;
- Développement de la fonctionnalité de **prise de notes** personnelles par leçon ;
- Intégration des **flashcards** avec l'algorithme de répétition espacée SM-2.

3.2.2 Backlog du sprint 2

Le tableau 3.5 présente l'ensemble des user stories réalisées durant le Sprint 2.

TABLEAU 3.5 :Backlog du Sprint 2 — Release 1

UC	Cas d'usage	Description	Est.
UC1	Gestion des cours	En tant qu'administrateur, je veux créer, modifier et publier des cours avec leurs leçons et contenus	5j
UC2	Inscription des apprenants	En tant qu'apprenant, je veux m'inscrire à un cours et accéder à son contenu	3j
UC3	Désinscription des apprenants	En tant qu'apprenant, je veux me désinscrire d'un cours que je suis	2j
UC4	Suivi de progression	En tant qu'apprenant, je veux suivre ma progression leçon par leçon au sein d'un cours	4j
UC5	Gestion des notes	En tant qu'apprenant, je veux prendre et consulter des notes personnelles sur chaque leçon	3j
UC6	Gestion Flashcards	En tant qu'apprenant, je veux générer et réviser des flashcards à partir du contenu d'un cours	4j

3.2.3 Spécification et conception du sprint

Diagramme de cas d'utilisation

La figure 3.15 présente le diagramme de cas d'utilisation du Sprint 2, illustrant les interactions entre l'apprenant, l'administrateur et les fonctionnalités pédagogiques développées durant cette itération.

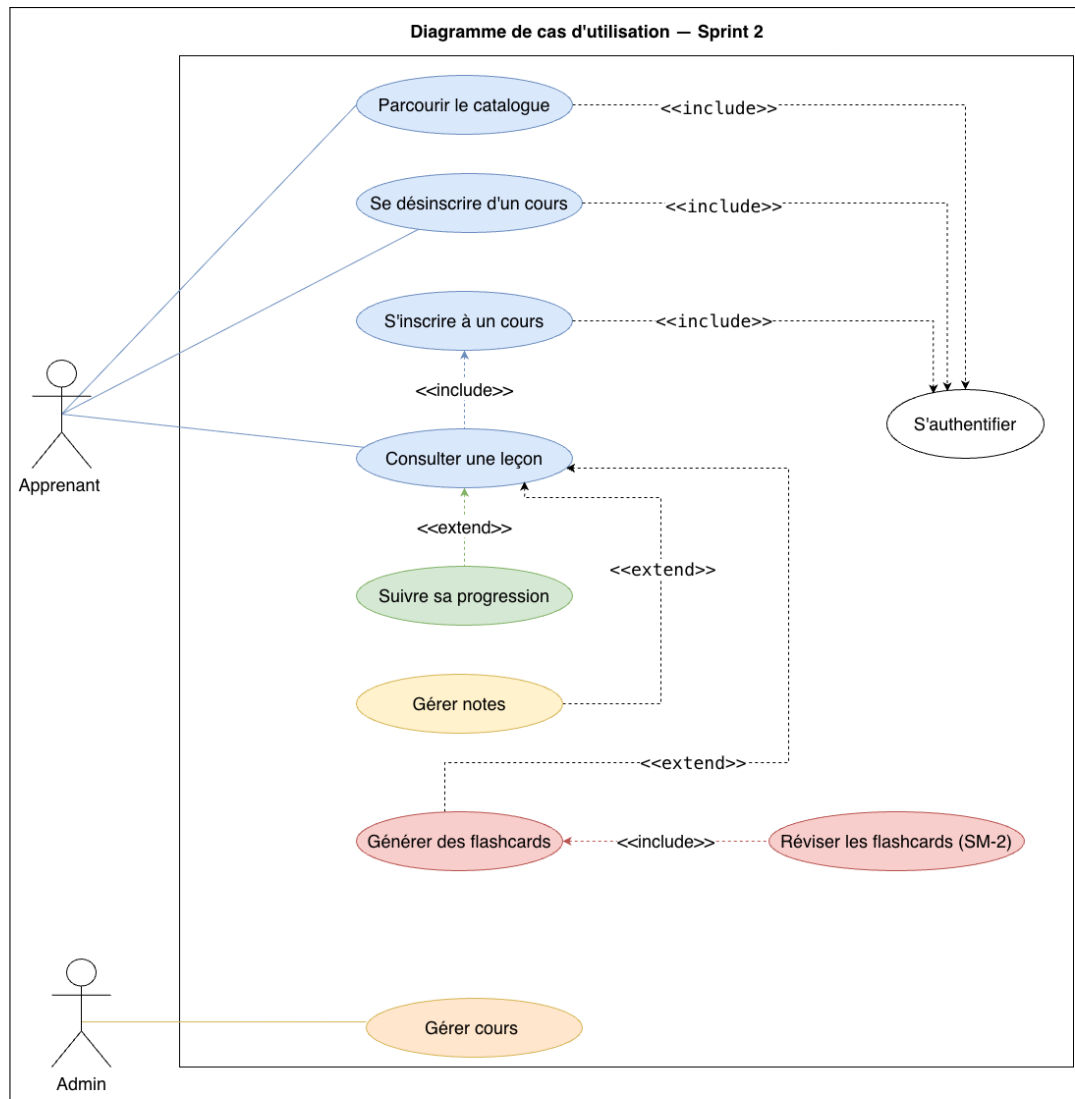


FIGURE 3.15 :Diagramme de cas d'utilisation — Sprint 2

L'**Apprenant** peut parcourir le catalogue de cours avec des filtres, s'inscrire et se désinscrire d'un cours, consulter les leçons et suivre sa progression, prendre des notes personnelles sauvegardées automatiquement, ainsi que générer et réviser des flashcards via l'algorithme SM-2. L'**Administrateur** dispose quant à lui des fonctionnalités de création et de publication de cours, ainsi que de gestion des modules et leçons.

Diagrammes de raffinement des cas d'utilisation

Les figures suivantes présentent les diagrammes de raffinement des cinq cas d'utilisation principaux du Sprint 2.

Diagramme de raffinement du cas d'utilisation « UC1 — Gérer cours »

La figure 3.16 détaille le cas d'utilisation **Gérer cours** réservé à l'Administrateur. Ce cas central est étendu («*extend*») par les sous-cas : créer, modifier, supprimer et publier un cours. La publication inclut l'ajout de leçons. Toutes ces actions nécessitent une authentification préalable («*include*»).

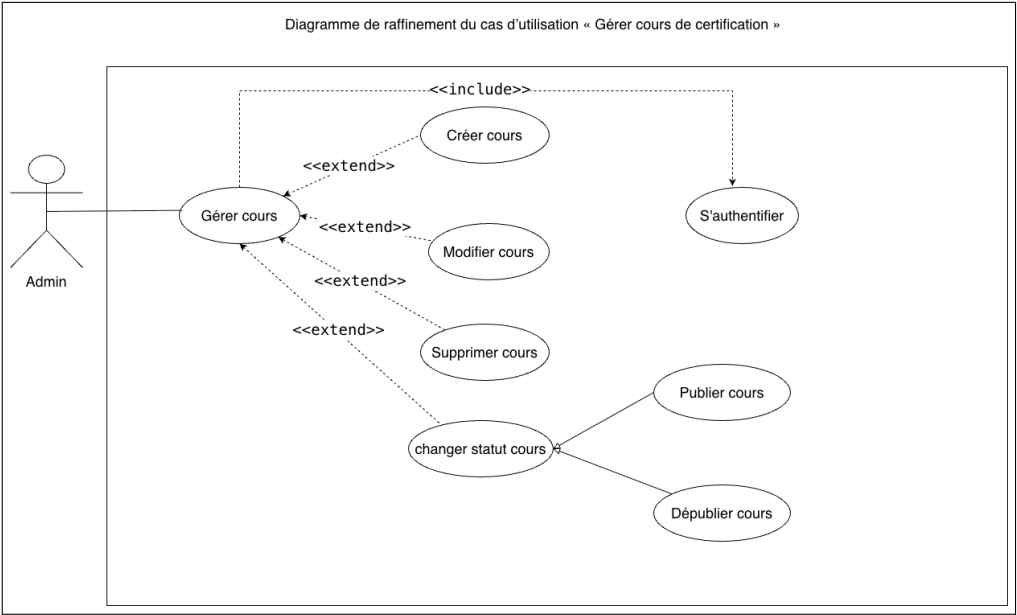


FIGURE 3.16 :Diagramme de raffinement — UC1 : Gérer cours

Le tableau 3.6 décrit le cas d'utilisation **Gérer cours**, présentant les actions disponibles pour l'administrateur pour créer, modifier et publier des cours, ainsi que les préconditions d'authentification requises.

TABEAU 3.6 :Description textuelle — UC1 : Gérer cours

Champ	Détail
Intitulé	Gérer cours
Acteurs	Administrateur (principal)
Préconditions	L'administrateur est authentifié
Postconditions	Le cours est créé, modifié ou publié sur la plateforme

Suite page suivante...

Suite Tableau 3.6

Champ	Détail
Scénario principal	1. L'admin accède à l'espace de gestion des cours 2. Il choisit une action : créer, modifier, supprimer ou publier 3. Pour la publication, il ajoute les leçons et le contenu 4. Le système enregistre et confirme les modifications
Scénario alternatif	2a. Suppression d'un cours publié → confirmation requise 3a. Leçon sans contenu → message d'erreur de validation

Diagramme de raffinement du cas d'utilisation « UC3 — S'inscrire / Désinscrire à un cours »

La figure 3.17 détaille les cas d'utilisation **S'inscrire** et **Se désinscrire** d'un cours. L'inscription inclut obligatoirement («*include*») le parcours du catalogue et donne accès au contenu. Les deux actions nécessitent une authentification.

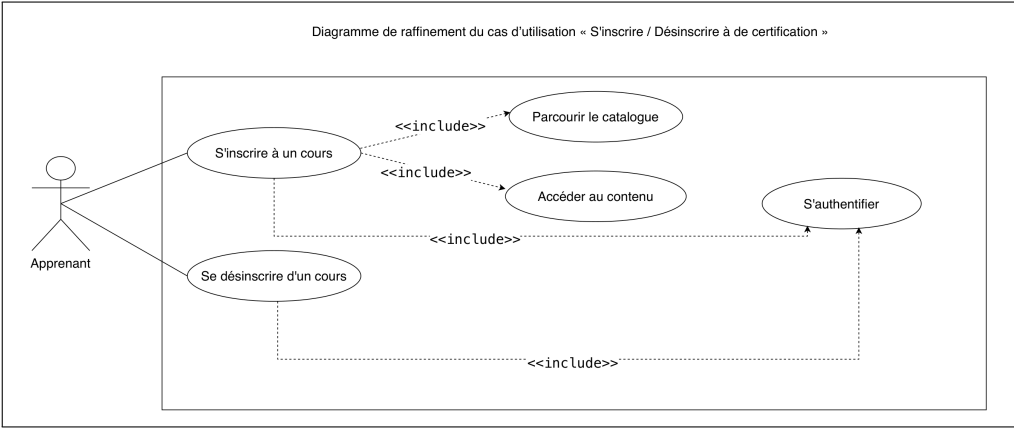


FIGURE 3.17 :Diagramme de raffinement — UC3 : S'inscrire / Désinscrire à un cours

Le tableau 3.7 décrit le cas d'utilisation **S'inscrire et se désinscrire à un cours**, détaillant les interactions de l'apprenant avec le catalogue et les conditions d'accès au contenu pédagogique.

TABLEAU 3.7 :Description textuelle — UC3 : S’inscrire / Désinscrire à un cours

Champ	Détail
Intitulé	S’inscrire / Se désinscrire à un cours
Acteurs	Apprenant (principal)
Préconditions	L’apprenant est authentifié
Postconditions	L’apprenant est inscrit ou désinscrit du cours
Scénario principal	1. L’apprenant parcourt le catalogue («include») 2. Il sélectionne un cours et clique sur « S’inscrire » 3. Le système enregistre l’inscription et donne accès au contenu 4. Pour la désinscription : l’apprenant confirme et perd l’accès
Scénario alternatif	2a. Cours déjà suivi → bouton « Se désinscrire » affiché 4a. Désinscription annulée → aucune modification

Diagramme de raffinement du cas d’utilisation « UC4 — Consulter un cours »

La figure 3.18 détaille le cas d’utilisation **Consulter un cours**. La consultation d’une leçon déclenche automatiquement («*include*») le **suivi de progression**, qui inclut lui-même le marquage de la leçon comme terminée.

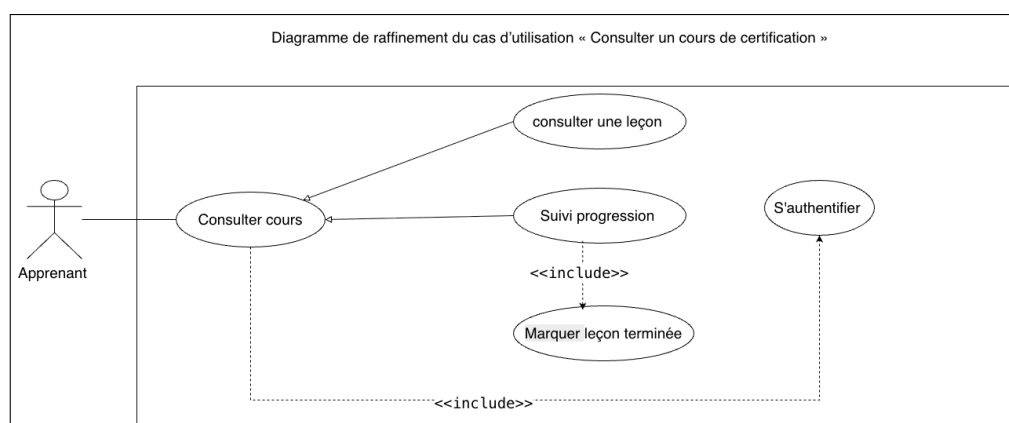


FIGURE 3.18 :Diagramme de raffinement — UC4 : Consulter un cours

Le tableau 3.8 décrit le cas d'utilisation **Consulter un cours**, incluant le suivi de progression leçon par leçon déclenché automatiquement lors de la consultation du contenu.

TABLEAU 3.8 :Description textuelle — UC4 : Consulter un cours

Champ	Détail
Intitulé	Consulter un cours
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est inscrit au cours et authentifié
Postconditions	La leçon est consultée et la progression est mise à jour
Scénario principal	1. L'apprenant accède à son cours depuis le dashboard 2. Il sélectionne une leçon à consulter 3. Le système enregistre la progression («include») 4. L'apprenant marque la leçon comme terminée
Scénario alternatif	2a. Leçon verrouillée (précédente non terminée) → accès refusé

Diagramme de raffinement du cas d'utilisation « UC5 — Gérer notes »

La figure 3.19 détaille le cas d'utilisation **Gérer notes**. L'apprenant peut créer, consulter, modifier et supprimer des notes associées à chaque leçon, toutes ces actions étant des extensions («*extend*») du cas central. Le tableau 3.8 décrit le cas d'utilisation **Consulter un cours**, incluant le suivi de progression leçon par leçon déclenché automatiquement lors de la consultation du contenu.

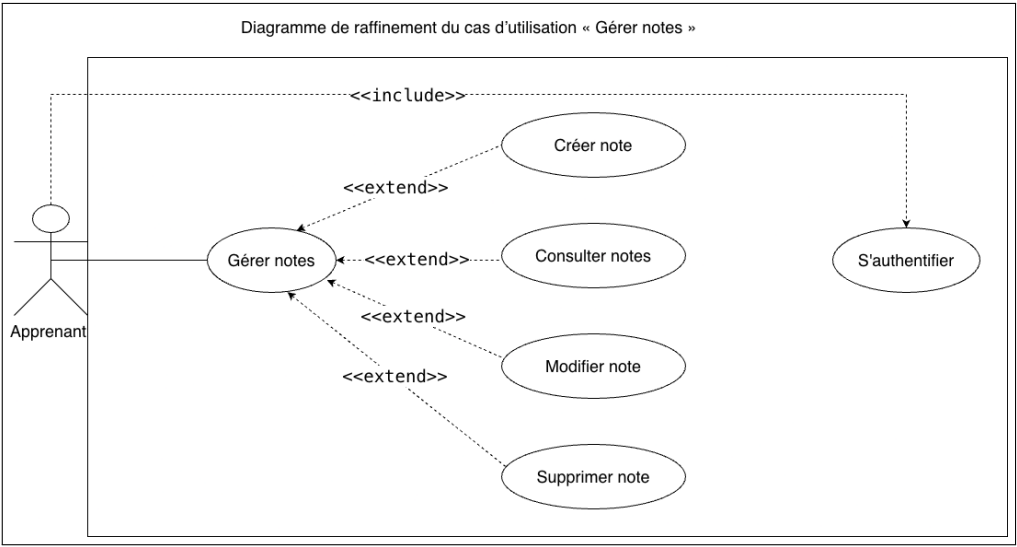


FIGURE 3.19 :Diagramme de raffinement — UC5 : Gérer notes

Le tableau 3.8 décrit le cas d'utilisation **Consulter un cours**, incluant le suivi de progression leçon par leçon déclenché automatiquement lors de la consultation du contenu.

TABLEAU 3.9 :Description textuelle — UC5 : Gérer notes

Champ	Détail
Intitulé	Gérer notes
Acteurs	Apprenant (principal)
Préconditions	L'apprenant consulte une leçon et est authentifié
Postconditions	La note est créée, modifiée ou supprimée

Suite page suivante...

Suite Tableau 3.9

Champ	Détail
Scénario principal	1. L'apprenant ouvre le panneau de notes d'une leçon 2. Il crée une nouvelle note («extend») 3. Il peut modifier ou supprimer une note existante 4. Les notes sont sauvegardées automatiquement
Scénario alternatif	3a. Suppression → confirmation requise 2a. Note vide → sauvegarde bloquée

Diagramme de raffinement du cas d'utilisation « UC6 — Gérer Flashcards »

La figure 3.20 détaille le cas d'utilisation **Gérer Flashcards**. La génération de flashcards est incluse obligatoirement («*include*») à partir du contenu du cours. La révision est une extension optionnelle («*extend*»).

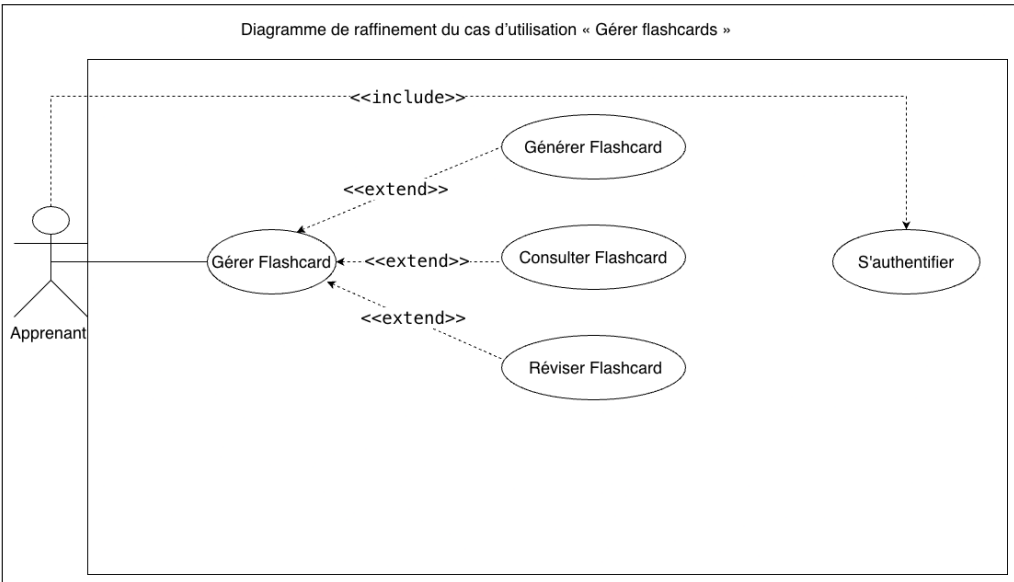


FIGURE 3.20 :Diagramme de raffinement — UC6 : Gérer Flashcards

Le tableau 3.10 décrit le cas d'utilisation **Gérer Flashcards**, détaillant la génération automatique de flashcards à partir du contenu d'un cours et leur révision selon l'algorithme SM-2.

TABLEAU 3.10 :Description textuelle — UC6 : Gérer Flashcards

Champ	Détail
Intitulé	Gérer Flashcards
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est inscrit au cours et authentifié
Postconditions	Les flashcards sont générées et/ou révisées
Scénario principal	1. L'apprenant demande la génération de flashcards pour un cours 2. Le système génère les flashcards («include») 3. L'apprenant consulte ses flashcards 4. Il lance une session de révision («extend») via l'algorithme SM-2
Scénario alternatif	2a. Contenu insuffisant → message d'information 4a. Toutes les cartes maîtrisées → fin de session affichée

Diagramme de séquence

Les diagrammes de séquence du Sprint 2 sont répartis en deux figures complémentaires couvrant l'ensemble des flux du sprint.

La figure 3.21 illustre les trois premiers flux du Sprint 2 : la consultation du catalogue de cours, l'inscription à un cours et le suivi de progression leçon par leçon.

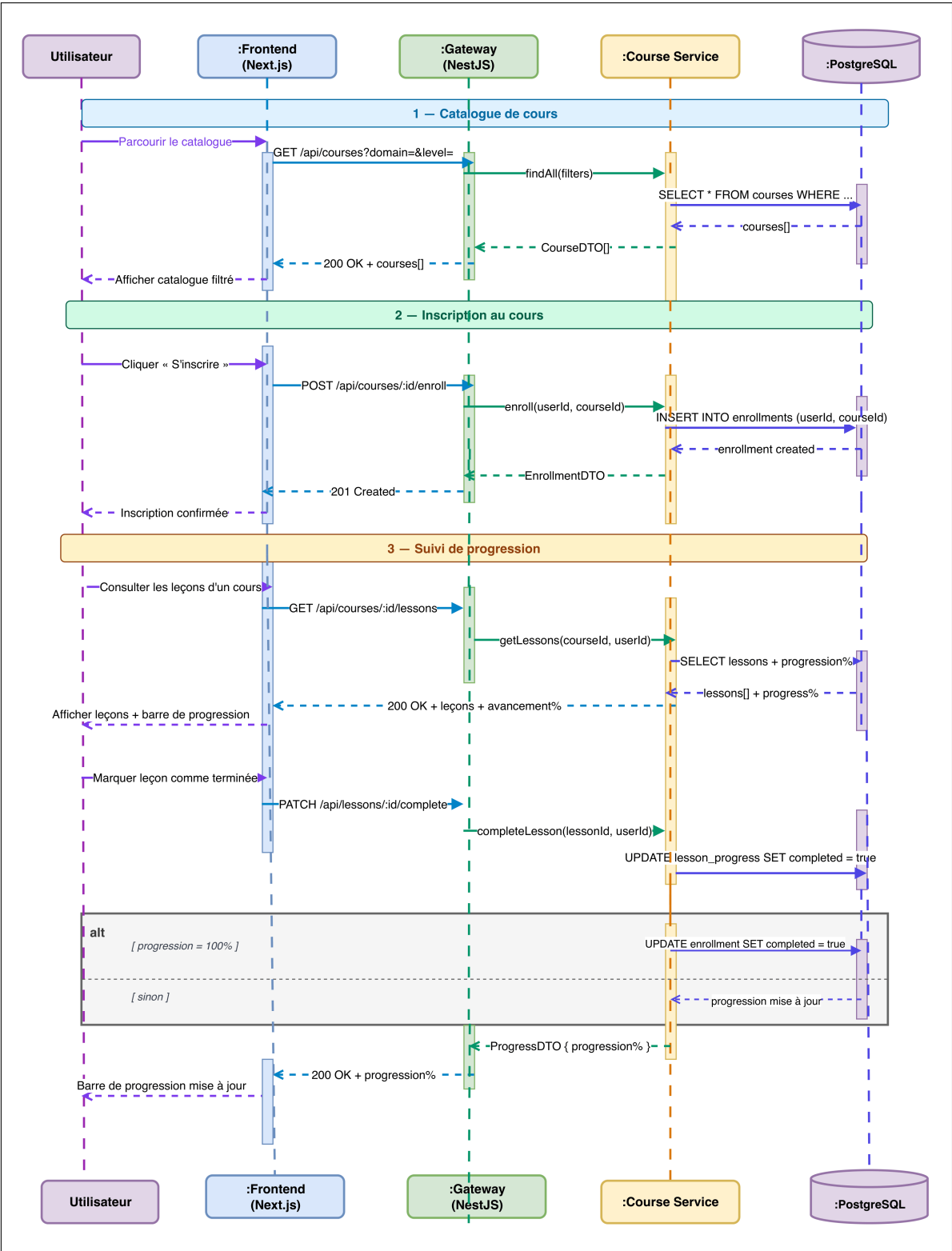


FIGURE 3.21 :Diagramme de séquence — Sprint 2 (Catalogue, Inscription et Progression)

Le **flux du catalogue** permet à l'apprenant de parcourir les cours disponibles via `GET /api/courses` avec filtres par domaine et niveau, et de recevoir la liste paginée en réponse.

Le **flux d'inscription** déclenche un POST `/api/courses/:id/enroll` qui insère une entrée dans la table `enrollments` et retourne une confirmation 201 `Created`.

Le **flux de progression** permet à l'apprenant de consulter les leçons avec le pourcentage d'avancement, puis de marquer chaque leçon comme terminée. Un fragment *alt* gère le cas où la progression atteint 100%, déclenchant la mise à jour du statut de complétion du cours.

La figure 3.22 illustre le quatrième flux du Sprint 2 : la prise de notes et la gestion des flashcards avec l'algorithme SM-2.

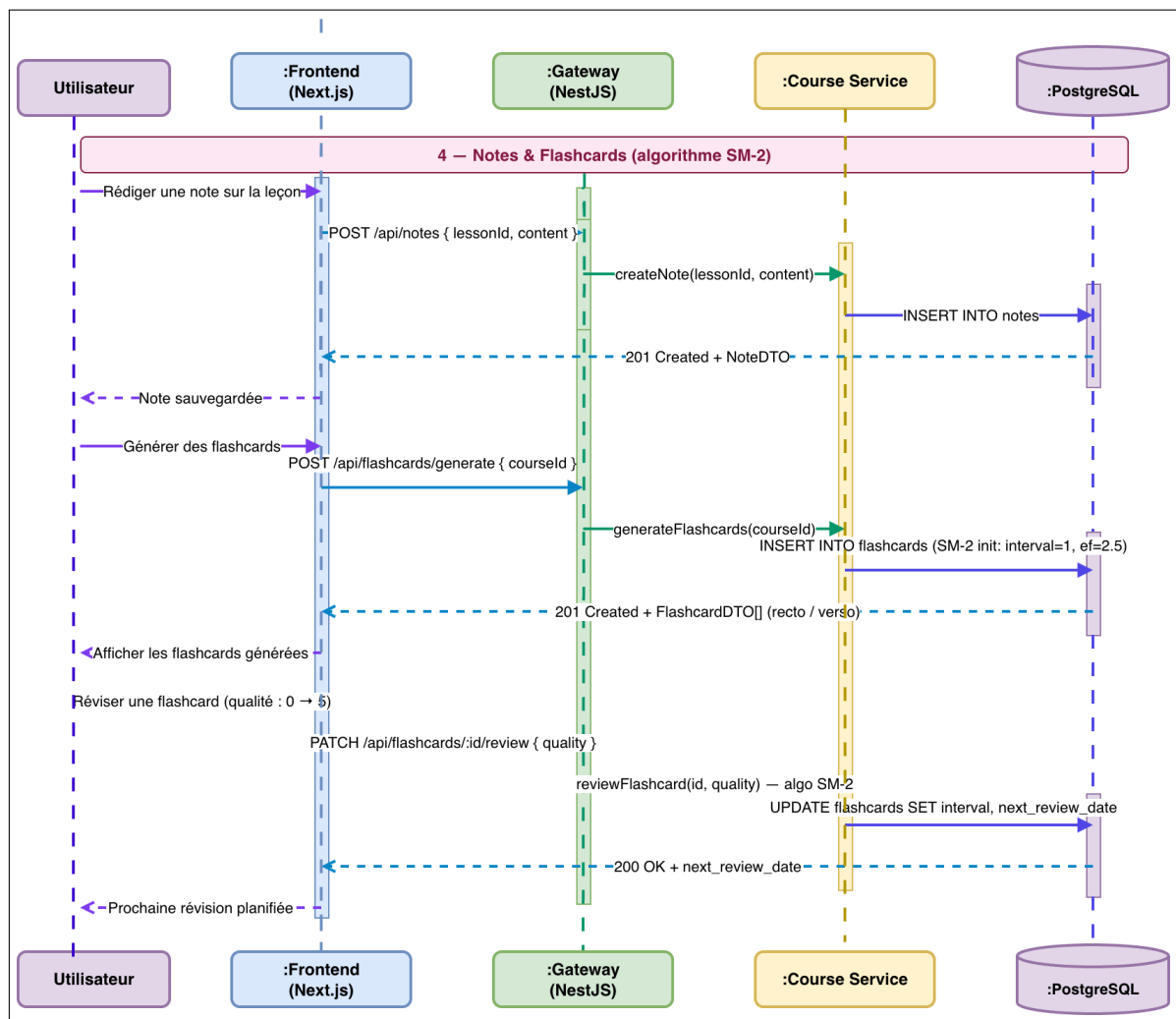


FIGURE 3.22 :Diagramme de séquence — Sprint 2 (Notes & Flashcards SM-2)

Le **flux notes et flashcards** couvre la création de notes personnelles par leçon (POST `/api/notes`), la génération automatique de flashcards depuis le contenu du cours avec initialisation des paramètres SM-2 (`interval=1`, `ef=2.5`), et leur révision adaptative via PATCH `/api/flashcards/:id/review` qui recalcule l'intervalle et la prochaine date de révision selon la qualité de la réponse de l'apprenant.

3.2.4 Réalisation

Cette section présente les interfaces développées durant le Sprint 2, couvrant le catalogue de cours, le suivi de progression, la prise de notes et les flashcards.

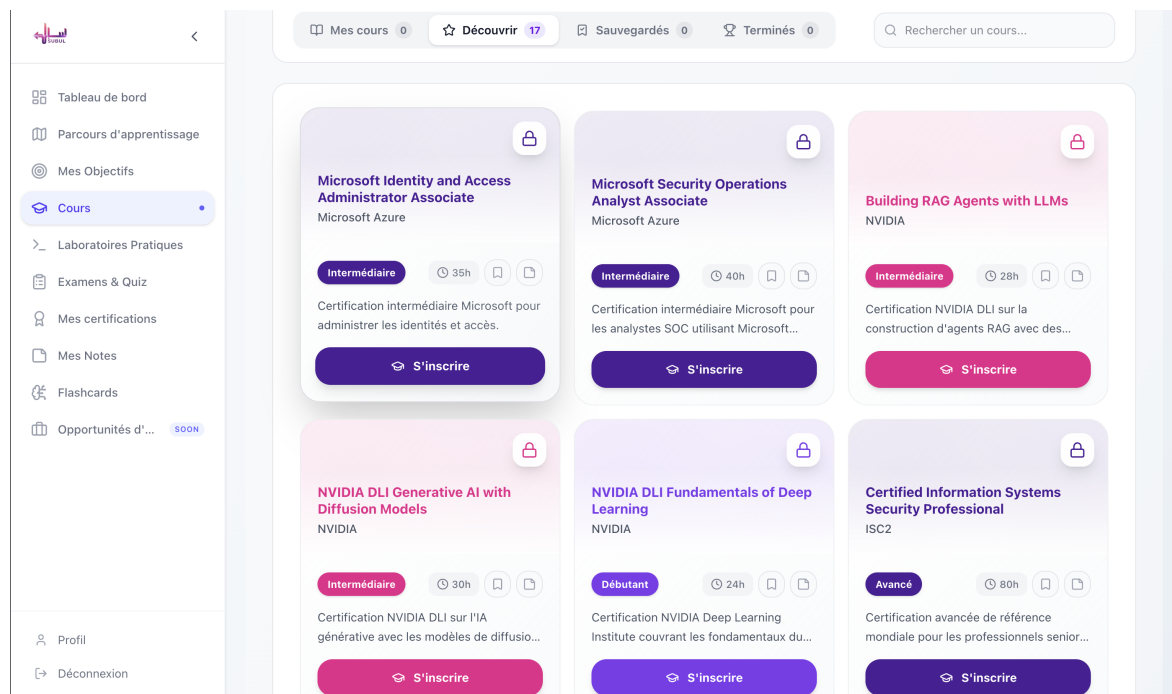
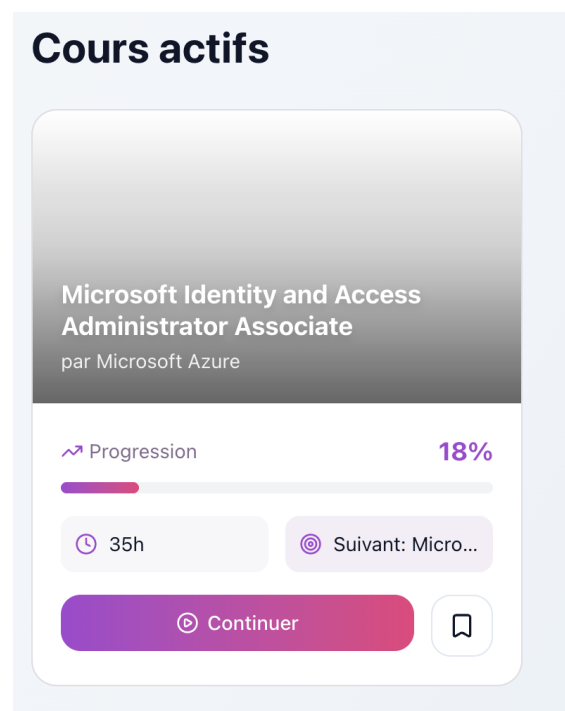


FIGURE 3.23 :Catalogue de cours

Le catalogue de cours permet à l'apprenant de parcourir l'ensemble des formations disponibles sur la plateforme, avec des filtres par domaine et par niveau afin de trouver rapidement une formation adaptée à ses besoins.



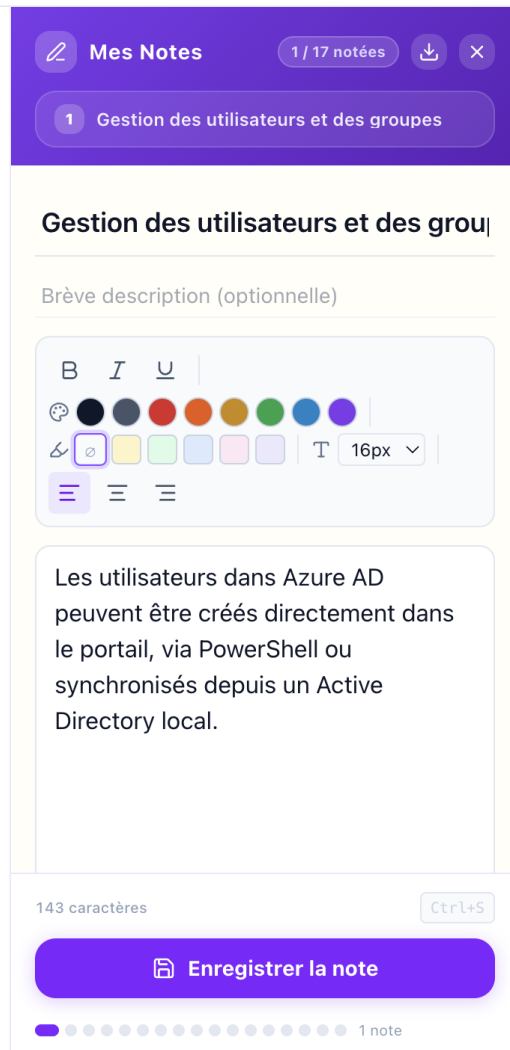
(a) Lecteur de leçon



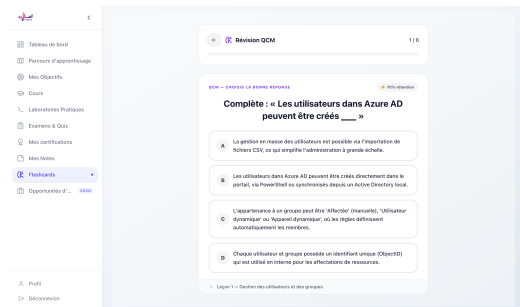
(b) Suivi de progression

FIGURE 3.24 : Consultation des leçons et suivi de progression

Le lecteur de leçon permet à l'apprenant de consulter le contenu pédagogique et de marquer chaque leçon comme terminée. Le suivi de progression affiche le pourcentage d'avancement global dans le cours, mis à jour en temps réel après chaque leçon complétée.



(a) Prise de notes



(b) Révision par flashcards

FIGURE 3.25 :Outils d'apprentissage actif : notes et flashcards

La fonctionnalité de prise de notes permet à l'apprenant de rédiger et sauvegarder des notes personnelles pour chaque leçon. Les flashcards, générées automatiquement depuis le contenu du cours, sont révisées selon l'algorithme SM-2 qui adapte les intervalles de révision en fonction des performances de l'apprenant.

Conclusion

Ce chapitre a présenté la réalisation de la **Release 1** : le Sprint 1 a mis en place l'infrastructure (API Gateway, JWT, gestion des utilisateurs) et le Sprint 2 a introduit les fonctionnalités pédagogiques (cours, progression, notes, flashcards). Le chapitre suivant détaillera la Release 2 avec les agents IA et le tableau de bord.

Chapitre 4

Release 2 – Fonctionnalités avancées et IA

Sommaire du chapitre

1. Introduction

2. Sprint 3 : Dashboard, Suivi, Objectifs et Administration

- Objectifs du sprint 3
- Backlog du sprint 3
- Spécification et conception du sprint
- Diagramme de séquence
- Réalisation

3. Sprint 4 : Agents IA – TutorAgent et QuizAgent

- Objectif du sprint 4
- Backlog du sprint 4
- Spécification et conception du sprint
- Fondements technologiques (LangGraph, MCP, Index, WebSocket)
- Workflow de traitement des données
- Agent 1 – TutorAgentMCP
- Agent 2 – QuizAgent
- Réalisation

4. Conclusion

Introduction

Ce chapitre présente la réalisation de la **Release 2 — Fonctionnalités avancées & IA**, composée du Sprint 3 (dashboard utilisateur, suivi d'apprentissage, gestion des notifications, gestion des objectifs, dashboard administrateur et statistiques) et du Sprint 4 (agents IA pédagogiques). Chaque sprint est planifié sur une durée de **4 semaines**, représentant un total de **8 semaines** pour cette release. Pour chaque sprint, nous détaillons les objectifs, le backlog, les diagrammes et les interfaces réalisées.

4.1 Sprint 3 : Dashboard, Suivi, Objectifs et Administration

4.1.1 Objectifs du sprint 3

Le Sprint 3 s'appuie sur les fondations établies lors de la Release 1 pour introduire les fonctionnalités de pilotage et de suivi propres à chaque profil d'utilisateur de la plateforme Subul. Ce sprint représente une étape clé dans l'expérience utilisateur, en centralisant toutes les informations pertinentes au sein de tableaux de bord dédiés.

Les principaux objectifs de ce sprint sont :

- Développement du **tableau de bord apprenant** (*Dashboard utilisateur*) centralisant les cours en cours, la progression globale et les statistiques d'apprentissage ;
- Mise en place du **suivi d'apprentissage** avec des métriques de progression détaillées par cours et par domaine ;
- Implémentation du système de **gestion des notifications** pour informer les apprenants des événements importants en temps réel ;
- Mise en place de la **gestion des objectifs** permettant à l'apprenant de définir, suivre et atteindre ses objectifs journaliers, hebdomadaires et généraux ;
- Développement du **tableau de bord administrateur** (*Dashboard Admin*) offrant une vue d'ensemble sur les utilisateurs et les cours de la plateforme ;
- Développement des **statistiques** d'utilisation accessibles à l'administrateur pour piloter la plateforme ;
- Mise en place du **suivi de progression avancé** permettant à l'apprenant de visualiser son évolution détaillée par domaine et par certification.

4.1.2 Backlog du sprint 3

Le tableau 4.1 présente l'ensemble des user stories réalisées durant le Sprint 3.

TABLEAU 4.1 :Backlog du Sprint 3 — Release 2

UC	Cas d'usage	Description	Est.
UC1	Dashboard utilisateur	En tant qu'apprenant, je veux accéder à un tableau de bord centralisant mes cours en cours, ma progression globale et mes statistiques d'apprentissage	5j
UC2	Suivi apprentissage	En tant qu'apprenant, je veux suivre ma progression détaillée par cours et consulter l'historique de mes activités	4j
UC3	Gestion de notifications	En tant qu'apprenant, je veux recevoir des notifications en temps réel pour les rappels et nouveaux cours	3j
UC4	Gestion des Objectifs	En tant qu'apprenant, je veux définir et suivre des objectifs journaliers, hebdomadaires et généraux d'apprentissage	4j
UC5	Dashboard Admin	En tant qu'administrateur, je veux accéder à un tableau de bord pour superviser les utilisateurs et les cours de la plateforme	5j
UC6	Statistiques	En tant qu'administrateur, je veux consulter les statistiques d'utilisation de la plateforme	3j

4.1.3 Spécification et conception du sprint

Diagramme de cas d'utilisation

La figure 4.1 présente le diagramme de cas d'utilisation du Sprint 3, illustrant les interactions entre l'apprenant, l'administrateur et les fonctionnalités de tableau de bord développées durant cette itération.

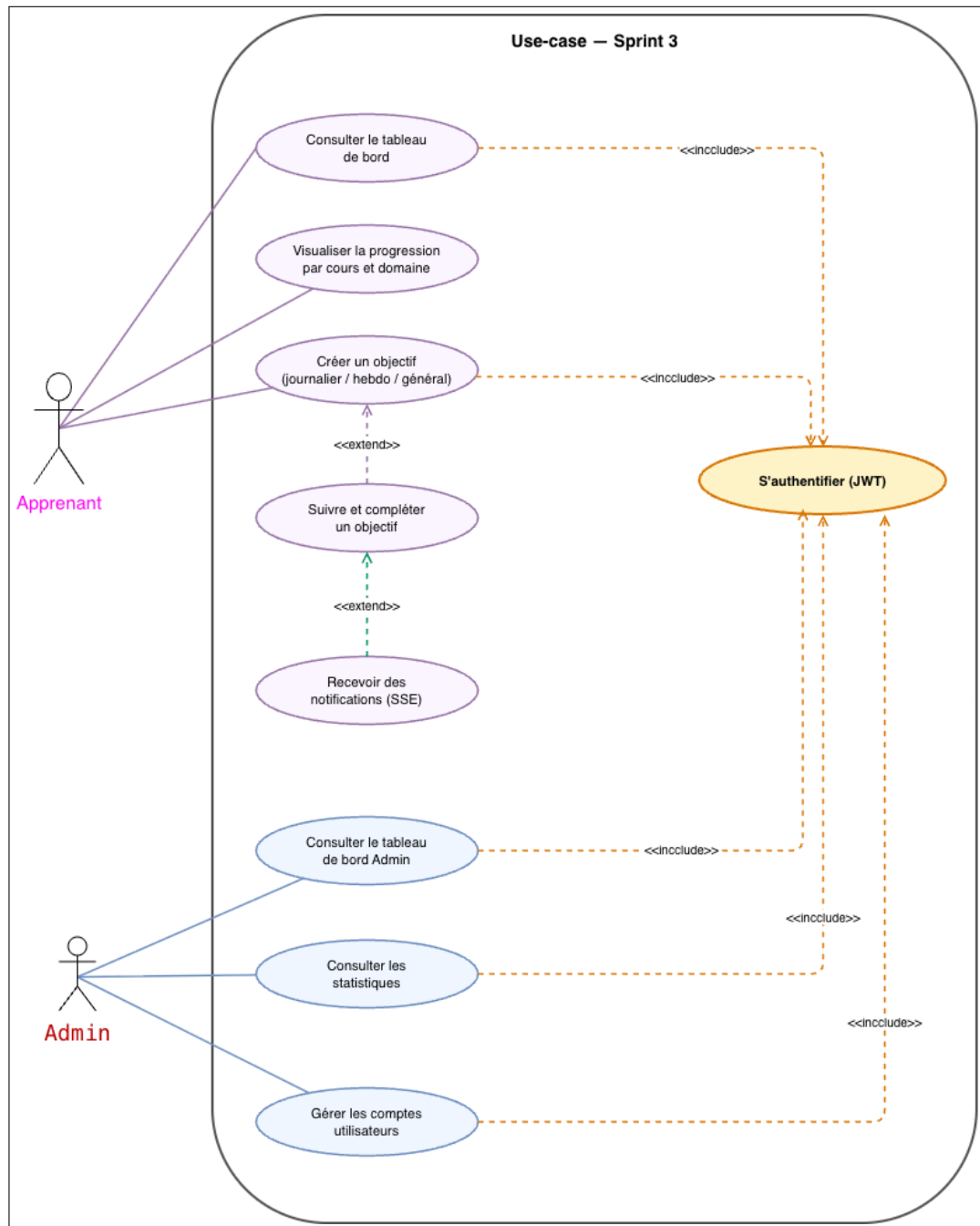


FIGURE 4.1 :Diagramme de cas d'utilisation — Sprint 3

L'**Apprenant** accède à son tableau de bord personnalisé regroupant ses cours actifs, sa progression globale et ses statistiques d'apprentissage. Il peut configurer et suivre ses objectifs journaliers, hebdomadaires et généraux (*<<include>>*), et recevoir des notifications pour rester informé des événements de la plateforme. L'**Administrateur** dispose d'un tableau de bord dédié lui permettant de superviser l'ensemble de la plateforme, de gérer les comptes utilisateurs et de consulter les statistiques d'utilisation (*<<include>>*).

Diagrammes de raffinement des cas d'utilisation

Les figures suivantes présentent les diagrammes de raffinement des six cas d'utilisation principaux du Sprint 3, détaillant leurs sous-cas et les relations entre acteurs.

Diagramme de raffinement du cas d'utilisation « UC1 — Dashboard utilisateur »

La figure 4.2 détaille le cas d'utilisation **Dashboard utilisateur**. L'apprenant accède à son tableau de bord qui inclut obligatoirement (*«include»*) la vue des cours en cours et de la progression globale. La consultation des statistiques d'apprentissage constitue une extension optionnelle (*«extend»*).

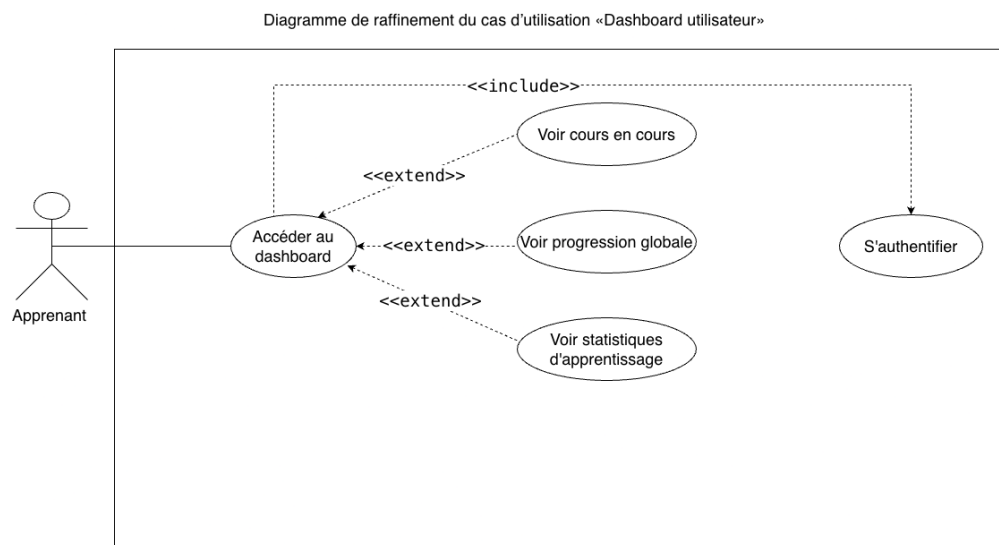


FIGURE 4.2 :Diagramme de raffinement — UC1 : Dashboard utilisateur

Le tableau 4.2 décrit le cas d'utilisation **Dashboard utilisateur**, détaillant les acteurs impliqués, les préconditions et les scénarios associés.

TABLEAU 4.2 :Description textuelle — UC1 : Dashboard utilisateur

Champ	Détail
Intitulé	Dashboard utilisateur
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est authentifié
Postconditions	Le tableau de bord est affiché avec les données à jour
Scénario principal	1. L'apprenant accède à son espace personnel 2. Le système charge les cours en cours («include») 3. Le système affiche la progression globale («include») 4. L'apprenant consulte ses statistiques d'apprentissage («extend»)
Scénario alternatif	2a. Aucun cours en cours → message d'invitation à s'inscrire 3a. Aucune progression → barre à 0% affichée

Diagramme de raffinement du cas d'utilisation « UC2 — Suivi apprentissage »

La figure 4.3 détaille le cas d'utilisation **Suivi apprentissage**. L'apprenant peut suivre sa progression détaillée par cours et par domaine. La consultation de l'historique des activités est une extension optionnelle («*extend*»).

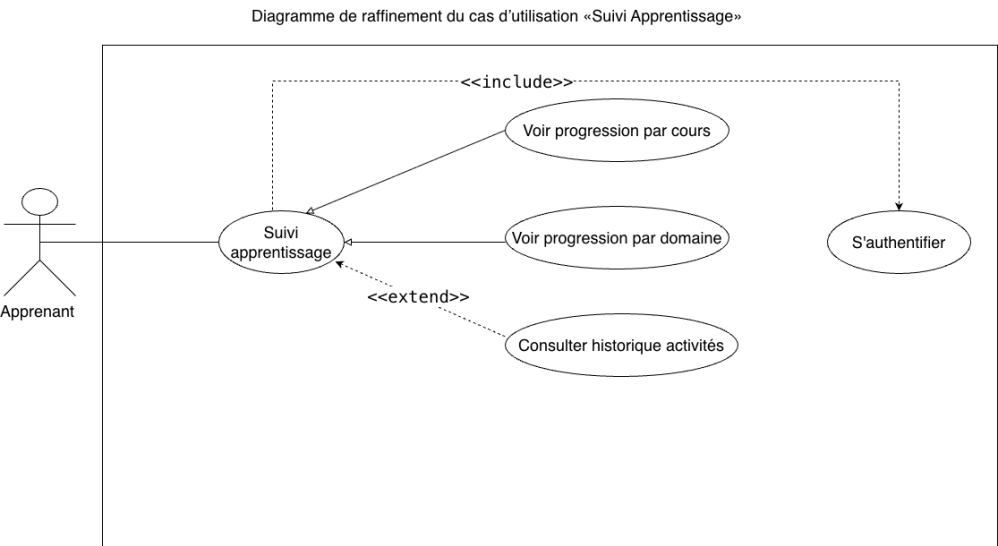


FIGURE 4.3 :Diagramme de raffinement — UC2 : Suivi apprentissage

Le tableau 4.3 décrit le cas d’utilisation **Suivi apprentissage**, détaillant les métriques de progression disponibles et les scénarios associés.

TABEAU 4.3 :Description textuelle — UC2 : Suivi apprentissage

Champ	Détail
Intitulé	Suivi apprentissage
Acteurs	Apprenant (principal)
Préconditions	L’apprenant est authentifié et inscrit à au moins un cours
Postconditions	Les métriques de progression sont affichées
Scénario principal	1. L’apprenant accède à la section suivi 2. Le système affiche la progression par cours («include») 3. Le système affiche la progression par domaine («include») 4. L’apprenant consulte l’historique de ses activités («extend»)

Suite page suivante...

Suite Tableau 4.3

Champ	Détail
Scénario alternatif	2a. Cours non débuté → progression à 0% 4a. Aucune activité récente → historique vide affiché

Diagramme de raffinement du cas d'utilisation « UC3 — Gestion de notifications »

La figure 4.4 détaille le cas d'utilisation **Gestion de notifications**. L'apprenant reçoit et consulte ses notifications en temps réel. Le marquage comme lue est une extension optionnelle («*extend*»).

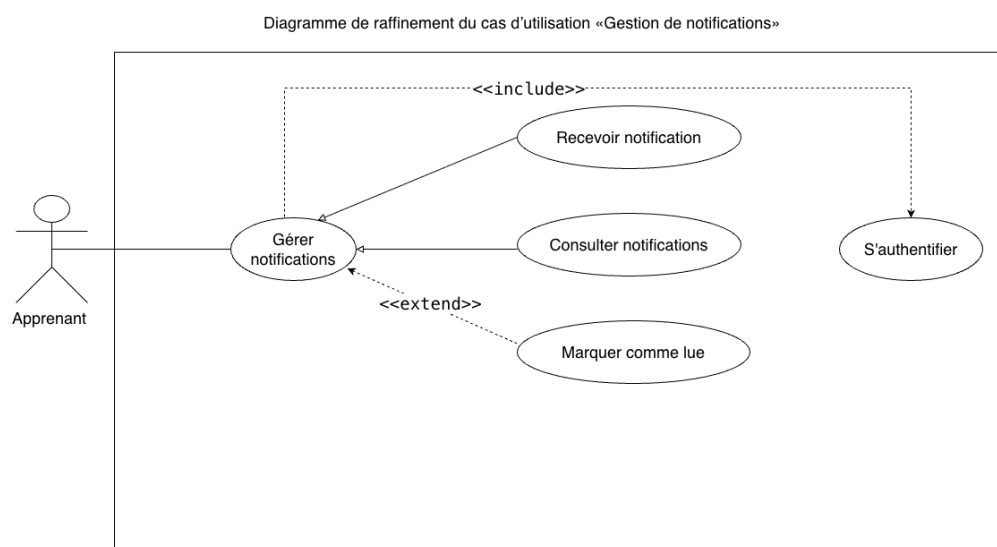


FIGURE 4.4 :Diagramme de raffinement — UC3 : Gestion de notifications

Le tableau 4.4 décrit le cas d'utilisation **Gestion de notifications**, précisant les flux de réception et de traitement des notifications en temps réel.

TABLEAU 4.4 :Description textuelle — UC3 : Gestion de notifications

Champ	Détail
Intitulé	Gestion de notifications
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est authentifié
Postconditions	Les notifications sont reçues et/ou marquées comme lues
Scénario principal	1. Le système envoie une notification en temps réel («include») 2. L'apprenant consulte ses notifications («include») 3. Il marque une ou plusieurs notifications comme lues («extend»)
Scénario alternatif	2a. Aucune notification → message « Aucune notification » 3a. Marquer toutes comme lues → action groupée disponible

Diagramme de raffinement du cas d'utilisation « UC4 — Gestion des Objectifs »

La figure 4.5 détaille le cas d'utilisation **Gestion des Objectifs**. L'apprenant peut définir des objectifs journaliers, hebdomadaires et généraux via des extensions («*extend*»). Le suivi de la complétion est inclus obligatoirement («*include*»).

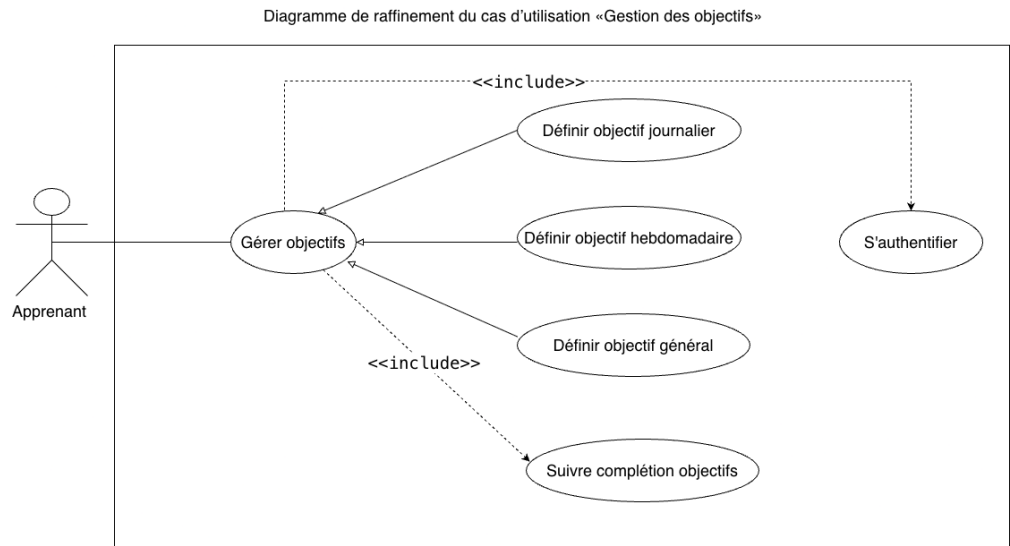


FIGURE 4.5 :Diagramme de raffinement — UC4 : Gestion des Objectifs

Le tableau 4.5 décrit le cas d’utilisation **Gestion des Objectifs**, détaillant la définition et le suivi automatique des objectifs d’apprentissage.

TABEAU 4.5 :Description textuelle — UC4 : Gestion des Objectifs

Champ	Détail
Intitulé	Gestion des Objectifs
Acteurs	Apprenant (principal)
Préconditions	L'apprenant est authentifié
Postconditions	Les objectifs sont créés et leur complétion est suivie
Scénario principal	1. L'apprenant accède à la section objectifs 2. Il définit un objectif journalier, hebdomadaire ou général («extend») 3. Le système enregistre l'objectif 4. Le système suit automatiquement la complétion («include»)
Scénario alternatif	2a. Objectif déjà atteint → badge de réussite affiché 3a. Valeur invalide → message d'erreur de validation

Diagramme de raffinement du cas d'utilisation « UC5 — Dashboard Admin »

La figure 4.6 détaille le cas d'utilisation **Dashboard Admin**. L’administrateur accède à une vue d’ensemble des utilisateurs et des cours. La gestion des comptes utilisateurs est une extension optionnelle («*extend*»).

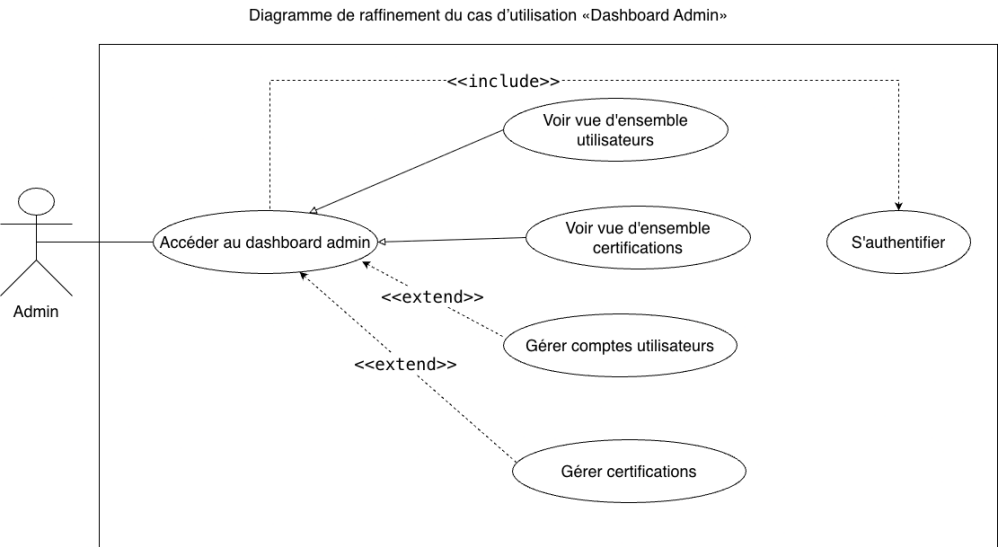


FIGURE 4.6 :Diagramme de raffinement — UC5 : Dashboard Admin

Le tableau 4.6 décrit le cas d'utilisation **Dashboard Admin**, présentant les fonctionnalités de supervision disponibles pour l’administrateur.

TABEAU 4.6 :Description textuelle — UC5 : Dashboard Admin

Champ	Détail
Intitulé	Dashboard Admin
Acteurs	Administrateur (principal)
Préconditions	L’administrateur est authentifié
Postconditions	Le tableau de bord admin est affiché avec les données à jour

Suite page suivante...

Suite Tableau 4.6

Champ	Détail
Scénario principal	1. L'administrateur accède à son espace d'administra- tion 2. Le système affiche la vue d'ensemble des utilisateurs («include») 3. Le système affiche la vue d'ensemble des cours («in- clude») 4. L'admin peut accéder à la gestion des comptes («extend»)
Scénario alternatif	2a. Aucun utilisateur inscrit → tableau vide affiché 3a. Aucun cours publié → liste vide avec invite de création

Diagramme de raffinement du cas d'utilisation « UC6 — Statistiques »

La figure 4.7 détaille le cas d'utilisation **Statistiques**. L'administrateur consulte les statistiques d'utilisation de la plateforme incluant les métriques utilisateurs, cours et taux de complétion.

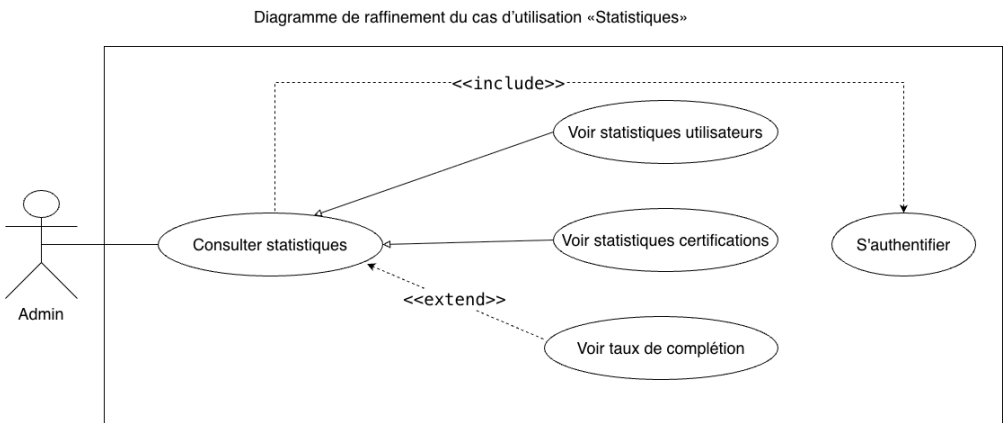


FIGURE 4.7 :Diagramme de raffinement — UC6 : Statistiques

Le tableau 4.7 décrit le cas d'utilisation **Statistiques**, détaillant les métriques d'utilisation consultables par l'administrateur.

TABLEAU 4.7 :Description textuelle — UC6 : Statistiques

Champ	Détail
Intitulé	Statistiques
Acteurs	Administrateur (principal)
Préconditions	L'administrateur est authentifié
Postconditions	Les statistiques d'utilisation sont affichées
Scénario principal	1. L'administrateur accède à la section statistiques 2. Le système affiche les statistiques des utilisateurs («include») 3. Le système affiche les statistiques des cours («include») 4. L'admin consulte le taux de complétion global («extend»)
Scénario alternatif	2a. Données insuffisantes → graphiques vides avec message explicatif 4a. Taux de complétion à 0% → alerte de suivi affichée

Diagramme de séquence

Les diagrammes de séquence du Sprint 3 sont répartis en deux figures complémentaires couvrant l'ensemble des flux du sprint.

La figure 4.8 illustre les deux premiers flux du Sprint 3 : l'agrégation des données du tableau de bord et la gestion des objectifs d'apprentissage.

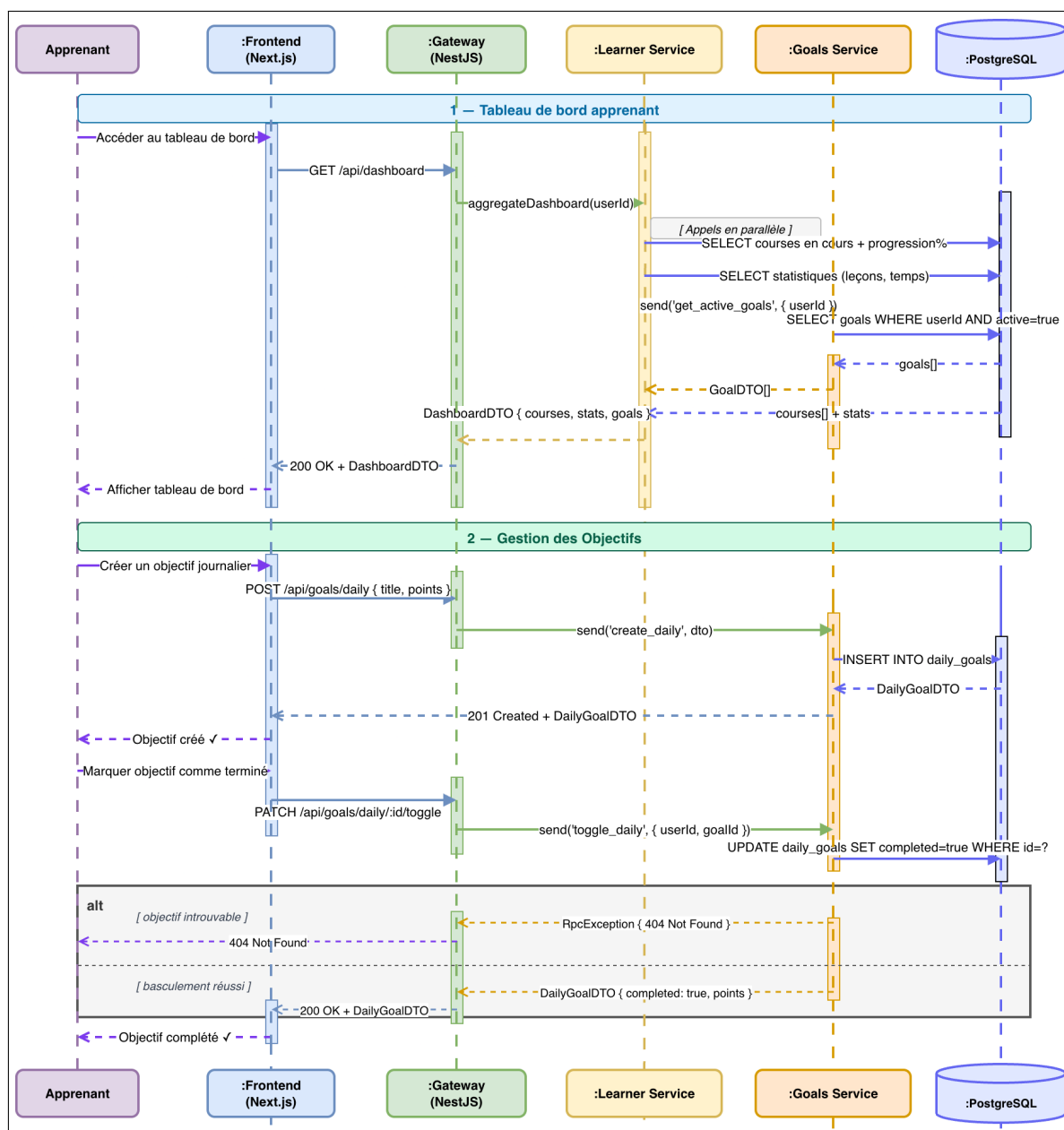


FIGURE 4.8 :Diagramme de séquence — Sprint 3 (Tableau de bord & Gestion des objectifs)

Le **flux du tableau de bord** permet à l'apprenant de charger en parallèle ses cours en cours, ses statistiques globales et ses objectifs actifs via un appel agrégé au Learner Service, qui coordonne les réponses du Goals Service et de la base de données.

Le **flux de gestion des objectifs** couvre la création d'un objectif journalier via `POST /api/goals/daily` (inséré en base avec les points associés), puis son basculement via `PATCH /api/goals/daily/:id/toggle`. Un fragment *alt* distingue le cas « objectif introuvable » (404) du cas de succès (200 + `completed: true`).

La figure 4.9 illustre le flux de notifications en temps réel, depuis l'abonnement SSE jusqu'à la réception d'un événement métier.

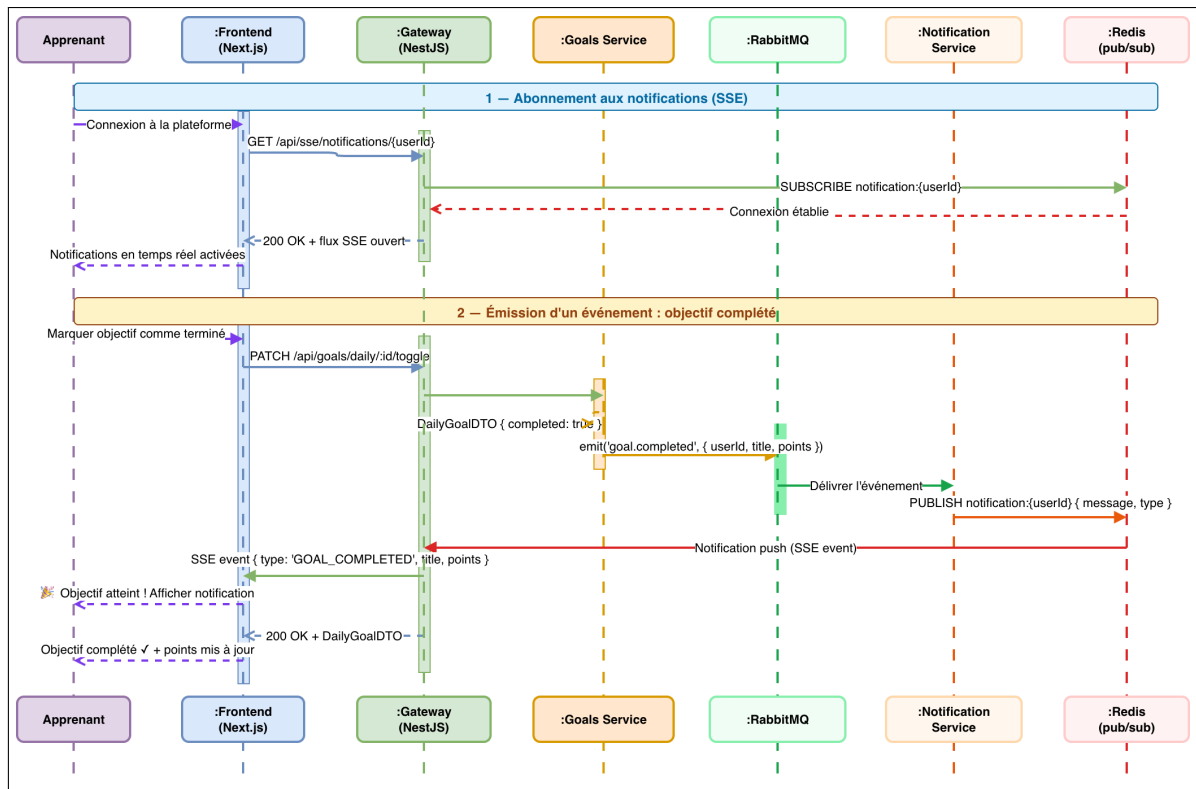


FIGURE 4.9 :Diagramme de séquence — Sprint 3 (Notifications en temps réel)

Le **flux d'abonnement SSE** établit une connexion persistante entre le frontend et la Gateway via `GET /api/sse/notifications/{userId}`, qui s'abonne au canal Redis `notification:{userId}` pour recevoir les événements en temps réel.

Le **flux d'émission** décrit la chaîne complète déclenchée lors de la complétion d'un objectif : le Goals Service émet l'événement `goal.completed` sur RabbitMQ [7], le Notification Service le consomme et publie sur Redis, la Gateway reçoit le push et le transfère au client sous forme d'événement SSE, et l'apprenant voit apparaître la notification en temps réel.

4.1.4 Réalisation

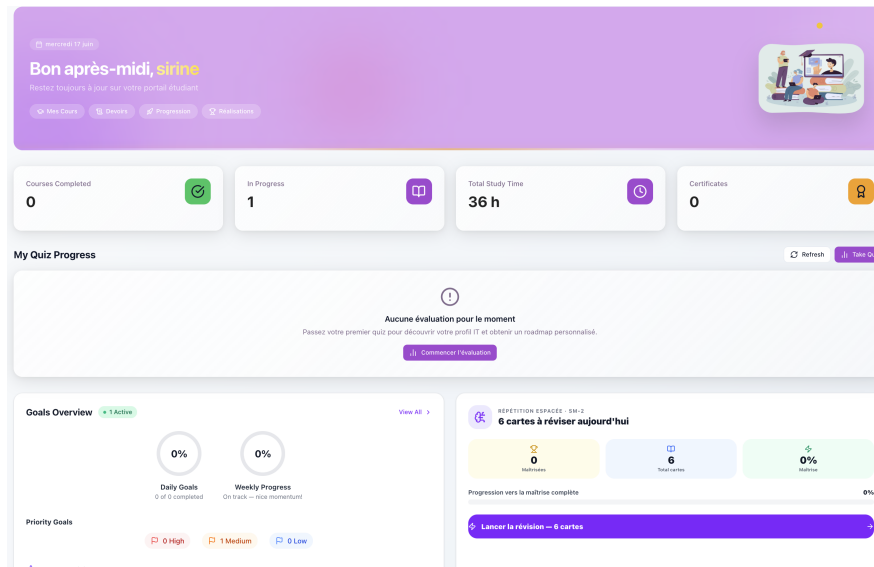


FIGURE 4.10 :Tableau de bord apprenant

Le tableau de bord apprenant centralise toutes les informations clés : cours en cours avec leur progression, objectifs du jour, statistiques d'apprentissage et activités récentes.

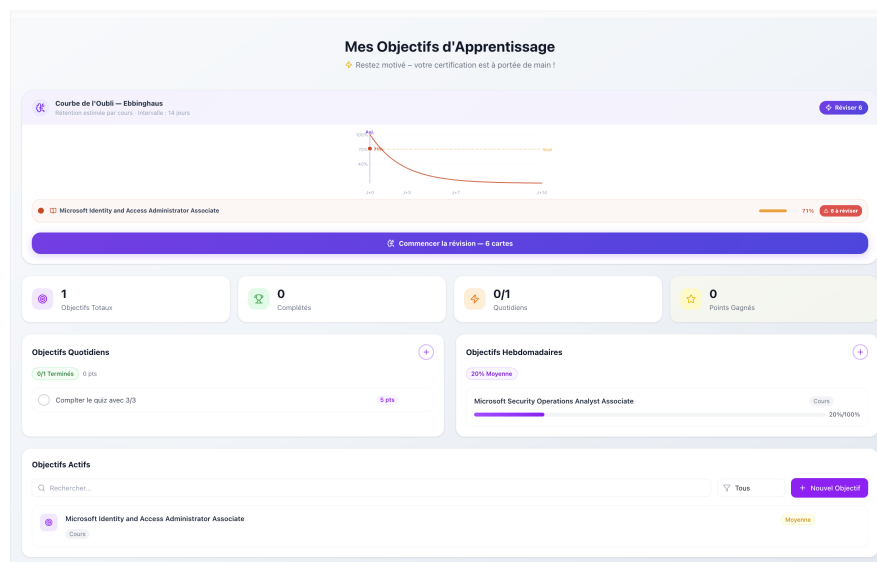


FIGURE 4.11 :Gestion des objectifs

La fonctionnalité de gestion des objectifs permet à l'apprenant de définir des objectifs journaliers, hebdomadaires et généraux, d'en suivre la progression en temps réel et de visualiser ses accomplissements.

Nouvel Objectif Hebdomadaire

Planifiez votre progression pour la semaine.

Lier à un cours / certification (optionnel)

Microsoft Security Operations Analyst Associ...

Titre

Microsoft Security Operations Analyst Associate

Description

Catégorie

Cours

Progression (%)

20

Cible (%)

100

Annuler

Ajouter

FIGURE 4.12 :Formulaire de création d'un objectif hebdomadaire

Nouvel Objectif Quotidien

Boostez votre journée avec des tâches motivantes.

Titre

Ex: Compléter un module

Points

5

Annuler

Ajouter

FIGURE 4.13 :Formulaire de création d'un objectif quotidien

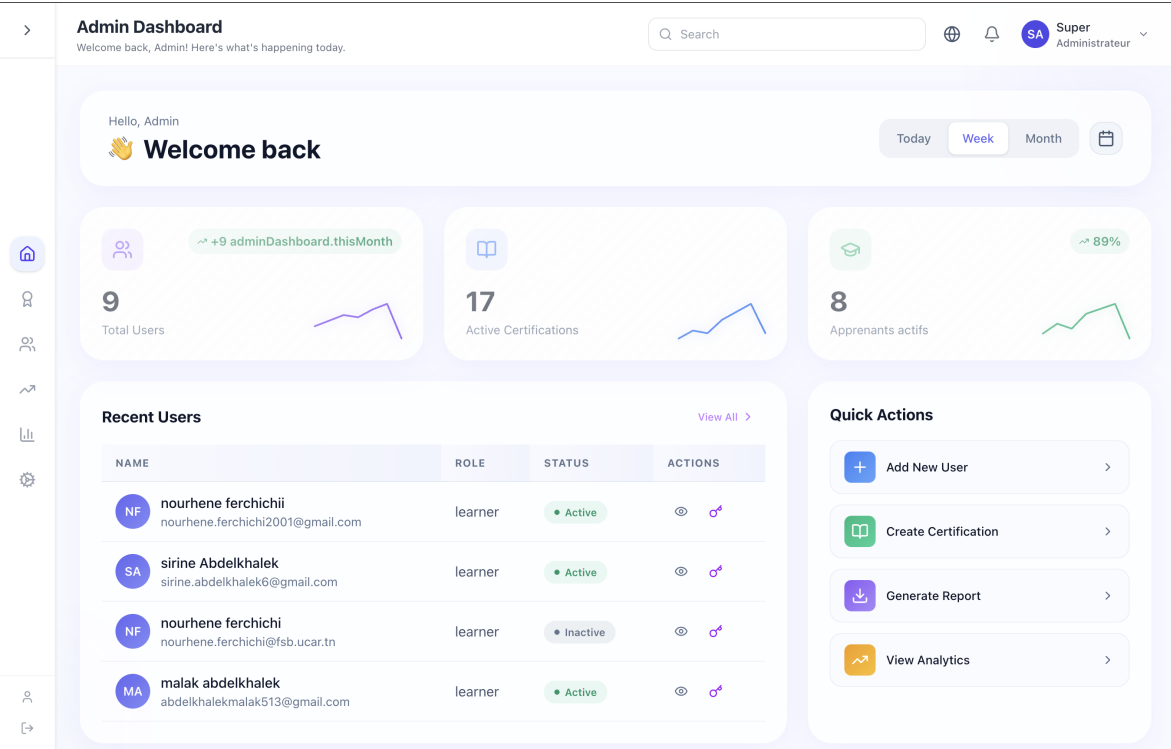


FIGURE 4.14 :Tableau de bord administrateur

Le tableau de bord administrateur offre une vue d'ensemble de la plateforme :

nombre total d'utilisateurs, cours publiés, taux de complétion moyen et alertes système.

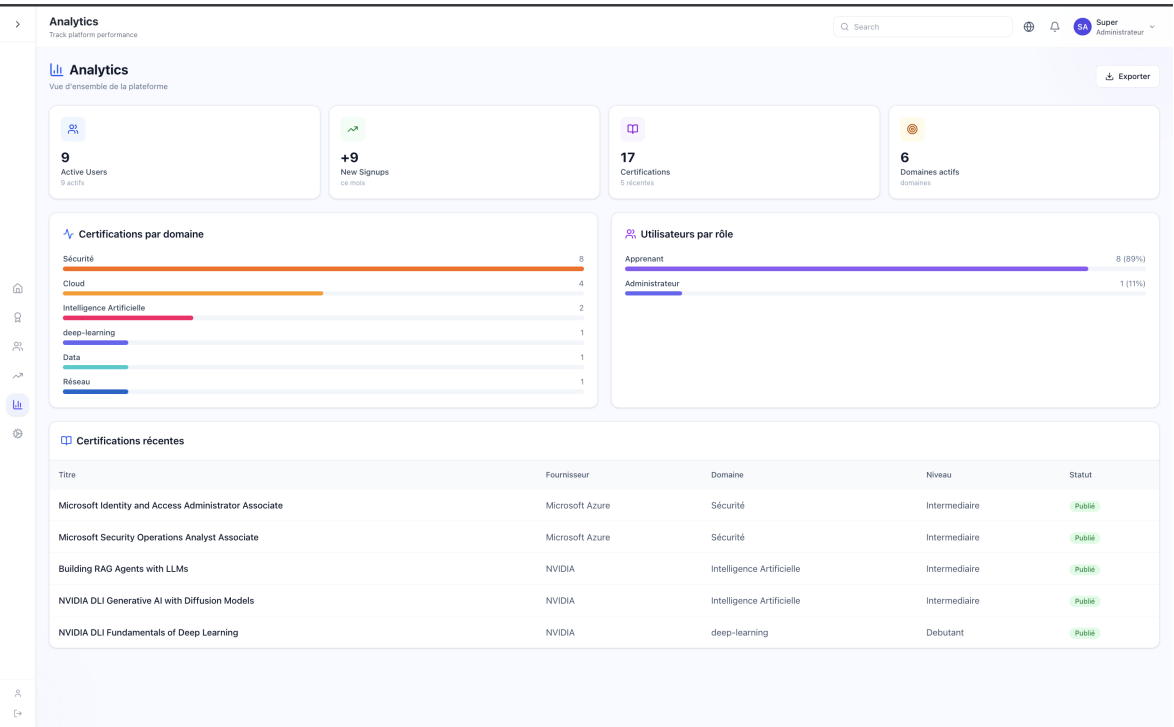


FIGURE 4.15 :Statistiques d'utilisation de la plateforme

La page de statistiques fournit à l'administrateur des métriques détaillées sur l'utilisation de la plateforme : évolution du nombre d'inscrits, taux de progression par cours et indicateurs de performance globaux.

4.2 Sprint 4 : Agents IA – TutorAgent et QuizAgent

4.2.1 Objectif du sprint

Ce sprint, d'une durée de **4 semaines**, vise à concevoir et implémenter les deux agents pédagogiques du *AI Service* de SUBUL :

- Le **TutorAgentMCP** : agent conversationnel temps réel combinant la recherche augmentée par récupération (**RAG**) [8] sur Azure AI Search et la recherche web dynamique via le protocole **MCP** (Model Context Protocol) avec Tavily, orchestré par un graphe d'état **LangGraph ReAct**, avec diffusion en continu (*streaming*) WebSocket bimode (texte et audio).
- Le **QuizAgent** : agent d'évaluation modulaire composé de trois sous-agents spécialisés (**QuestionGenerator**, **QuestionEvaluator**, **QuestionRegenerator**),

avec régénération adaptative ciblant les concepts insuffisamment maîtrisés.

4.2.2 Backlog du sprint

Le tableau 4.8 présente les user stories réalisées au cours de ce sprint.

TABLEAU 4.8 :Backlog du Sprint 4 — Release 2

UC	Cas d’usage	Description	Est.
UC1	Agent Tuteur IA	En tant qu’apprenant, je veux poser des questions à un tuteur IA qui répond en temps réel à partir des cours et du web	7j
UC2	Suivi de progression	En tant qu’apprenant, je veux que le tuteur IA adapte ses réponses selon ma progression et mon niveau dans le cours	4j
UC3	Agent Quiz IA	En tant qu’apprenant, je veux recevoir des questions QCM contextuelles avec feedback immédiat et régénération adaptative	9j

4.2.3 Spécification et conception du sprint

Diagramme de cas d’utilisation du Sprint 4

La figure 4.16 présente les cas d’utilisation du Sprint 4. L’apprenant interagit avec le TutorAgent pour formuler des questions pédagogiques et avec le QuizAgent pour s’auto-évaluer à l’issue de chaque module.

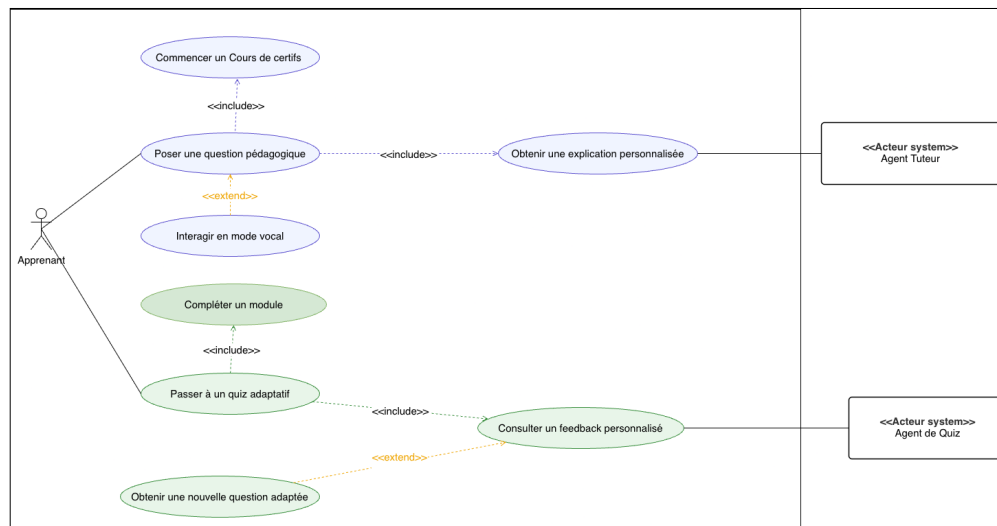


FIGURE 4.16 :Diagramme de cas d'utilisation — Sprint 4

4.2.4 Fondements technologiques

Le *AI Service* de SUBUL repose sur un ensemble de briques technologiques choisies pour leur complémentarité et leur adéquation aux contraintes pédagogiques du projet. Cette section présente chaque brique de façon synthétique, en mettant en évidence son rôle concret dans l'architecture que j'ai conçue.

Python 3.11 et bibliothèques IA

J'ai choisi **Python 3.11** comme langage du service IA pour son support natif de l'asynchronisme (**asyncio**), indispensable pour gérer simultanément de multiples sessions d'apprenants sans blocage. Les quatre bibliothèques clés utilisées sont : **LangChain** pour l'intégration des LLMs, **LangGraph** pour l'orchestration des agents, **Pydantic v2** pour la validation stricte des sorties du QuizAgent, et **asyncpg** pour les accès asynchrones à PostgreSQL.

LangGraph – Orchestration par graphe d'état

J'ai choisi **LangGraph** pour orchestrer les deux agents car, contrairement aux chaînes séquentielles de LangChain, il permet de modéliser des boucles de raisonnement conditionnelles via un graphe d'état (*StateGraph*). L'**AgentState** — le dictionnaire partagé et enrichi à chaque nœud — contient l'historique de conversation, les passages RAG récupérés et les résultats des outils invoqués.

Le paradigme ReAct (Reasoning and Acting, Yao et al., 2022), implémenté dans

LangGraph, repose sur trois états cycliques : Think (analyse du contexte et décision), Act (invocation de Tavily MCP si nécessaire) et Observe (réinjection du résultat pour l'itération suivante). La condition d'arrêt est déterminée automatiquement par le LLM. La figure 4.17 illustre l'architecture générale du paradigme ReAct.

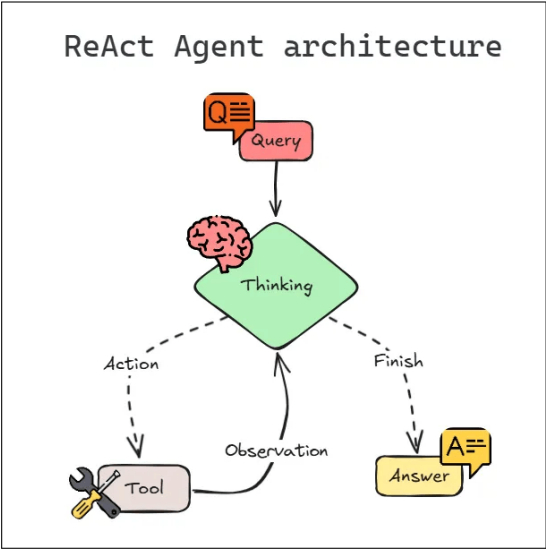


FIGURE 4.17 :Architecture du paradigme ReAct (*Reasoning and Acting*)

La figure 4.18 présente le graphe d'état LangGraph.

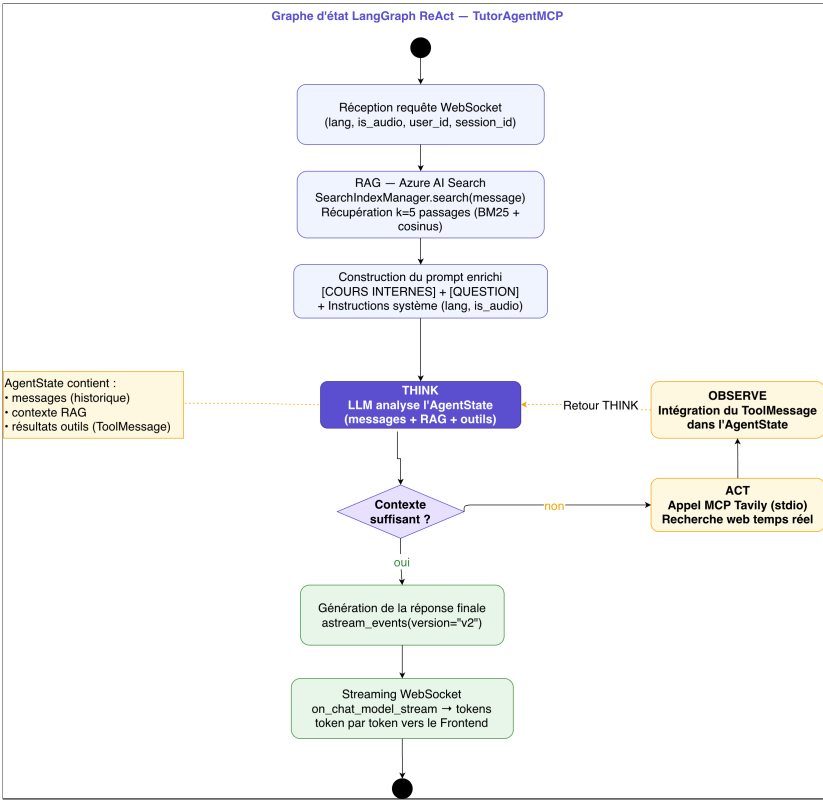


FIGURE 4.18 :Graphe d'état LangGraph ReAct du TutorAgentMCP – états et transitions

Protocole MCP et Azure AI Search

Le **Model Context Protocol (MCP)**, proposé par Anthropic en 2024, est le protocole que j’ai utilisé pour connecter l’agent LangGraph à **Tavily Search** en tant qu’outil de recherche web temps réel. Le serveur MCP Tavily s’exécute en processus Python indépendant et communique via transport **stdio**, garantissant l’isolation et la sécurité sans exposition réseau.

Pour la recherche documentaire interne, j’ai configuré un index **Azure AI Search** hybride combinant la recherche lexicale BM25 et la recherche vectorielle par similarité cosinus. Le **SearchIndexManager** que j’ai implémenté segmente les cours en passages de 512 tokens (chevauchement de 64 tokens), les vectorise via **text-embedding-ada-002** et maintient l’index à l’initialisation du service. Cette architecture RAG permet une récupération des passages pertinents en quelques millisecondes, contre 10 à 30 secondes pour une inférence LLM brute.

Streaming temps réel par WebSocket

J’ai opté pour le **WebSocket** (RFC 6455) plutôt que HTTP classique pour le TutorAgentMCP afin de diffuser les tokens du LLM dès leur production, ramenant la latence perçue à moins d’une seconde (contre 10 à 30 secondes en requête-réponse classique). Le tableau 4.9 synthétise cette comparaison.

Le tableau 4.9 compare les deux approches de communication et justifie le choix du streaming WebSocket pour l’expérience utilisateur.

TABLEAU 4.9 :Comparaison HTTP classique vs streaming WebSocket

Critère	HTTP requête- réponse	WebSocket streaming
Latence perçue	Totale (10–30s avant affichage)	< 1s (premier token immédiat)
Connexions	Nouvelle connexion par requête	Connexion persistante réutilisée
Interruption possible	Non (attente bloquante)	Oui (le client peut annuler)
Adapté aux LLM	Non (génération séquentielle)	Oui (diffusion token par token)

4.2.5 Workflow de traitement des données

Avant d'être exploitées par les agents, les données documentaires suivent un pipeline structuré en quatre phases.

Phase 1 – Collecte et acquisition

Le corpus documentaire de SUBUL est constitué de contenus pédagogiques relatifs aux certifications Cloud, Cybersécurité, IA et IoT, issus de :

- **Documents de cours** : fichiers PDF et pages web extraits des programmes Microsoft, AWS et CompTIA, structurés par domaine et par niveau ;
- **Ressources officielles** : documentation extraite de Microsoft Learn, AWS Skill Builder et des référentiels d'examens ;
- **Recherche web dynamique** : pour les requêtes non couvertes par la base statique, Tavily MCP retourne des extraits sourcés et datés en temps réel.

Phase 2 – Prétraitement et indexation

1. **Segmentation** (*chunking*) : découpage en passages de 512 tokens avec chevauchement de 64 tokens, préservant la cohérence sémantique.
2. **Vectorisation** : conversion en vecteurs denses par `text-embedding-ada-002`, capturant la sémantique profonde.
3. **Indexation hybride** : stockage conjoint des vecteurs et du texte brut dans Azure AI Search (BM25 + similarité cosinus).

Phase 3 – Récupération augmentée (RAG)

À chaque requête, le `SearchIndexManager` exécute une recherche hybride : la question est vectorisée à la volée, Azure AI Search retourne les k passages les plus pertinents ($k = 5$ par défaut), qui sont injectés dans le prompt sous la balise `[COURS INTERNES RETROUVÉS]`.

Phase 4 – Génération et exploitation

Le LLM reçoit un prompt structuré en trois couches : (1) instructions système (rôle, langue, contraintes), (2) contexte RAG (passages documentaires), (3) question de

l'apprenant. La figure 4.19 synthétise ce flux de bout en bout.

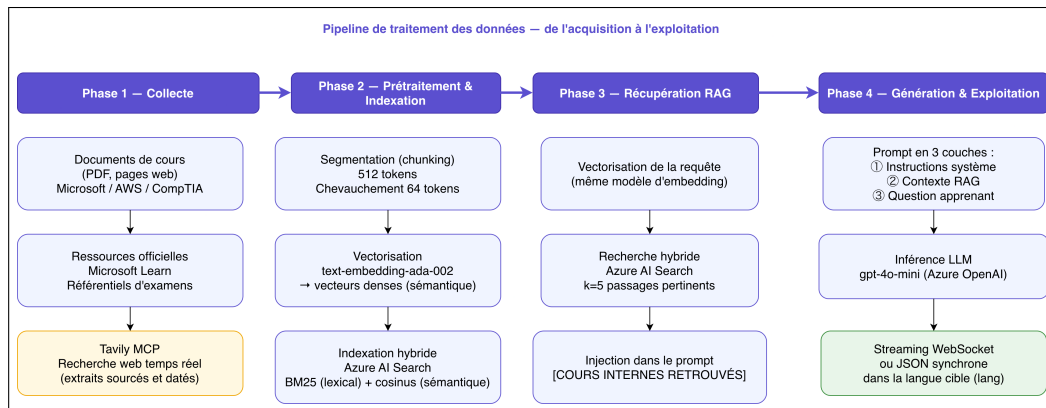


FIGURE 4.19 : Pipeline de traitement des données : de l'acquisition à l'exploitation

4.2.6 Implémentation — Extraits de code clés

Cette section présente trois extraits de code illustrant les contributions techniques personnelles les plus significatives du service IA.

Construction du graphe LangGraph ReAct

L'extrait ci-dessous montre comment j'ai construit le graphe d'état LangGraph ReAct du TutorAgentMCP. La méthode `create_react_agent` encapsule la boucle Think → Act → Observe ; le graphe résultant est compilé une seule fois au démarrage du service et réutilisé pour toutes les sessions d'apprenants.

```

1 from langchain_openai import AzureChatOpenAI
2 from langgraph.prebuilt import create_react_agent
3 from langchain_mcp_adapters.client import MultiServerMCPClient
4
5 async def build_agent(mcp_client: MultiServerMCPClient):
6     # Recuperation des outils MCP (Tavily Search)
7     tools = await mcp_client.get_tools()
8
9     # Modele LLM Azure OpenAI (gpt-4o-mini)
10    llm = AzureChatOpenAI(
11        azure_deployment="gpt-4o-mini",
12        temperature=0.3,
13        streaming=True,
14    )
15
16    # Construction du graphe ReAct :
17    # Think -> Act (tool call) -> Observe -> Think -> ... -> END

```

```

18     agent = create_react_agent(
19         model=llm,
20         tools=tools,
21         state_modifier=system_prompt, # Instructions systeme +
           contexte RAG
22     )
23     return agent

```

Listing 4.1 :Construction du graphe ReAct LangGraph – TutorAgentMCP

Prompt Engineering du QuizAgent

L'extrait suivant illustre le prompt du `QuestionGenerator`, conçu pour produire des questions QCM structurées et validées par Pydantic v2. La contrainte de langue (`lang`) garantit la cohérence multilingue sans détecter automatiquement la langue de l'apprenant.

```

1 def build_question_prompt(domain: str, concepts: list[str],
2                             difficulty: str, count: int,
3                             lang: str) -> str:
4     return f"""
5 Tu es un expert certifie en {domain}.
6 Genere EXACTEMENT {count} questions QCM sur les concepts : {concepts}.
7 Niveau : {difficulty}. Langue de sortie : {lang}.
8
9 CONTRAINTES STRICTES :
10 - Chaque question a exactement 4 options (A, B, C, D) sans doublons
11 - correct_answer DOIT etre l'une des 4 options
12 - Reponds UNIQUEMENT en JSON valide, sans texte supplementaire
13
14 FORMAT JSON attendu :
15 {{
16     "questions": [
17         {{
18             "text": "Question ?",
19             "options": [
20                 {{ "label": "A", "text": "Option A" }},
21                 {{ "label": "B", "text": "Option B" }},
22                 {{ "label": "C", "text": "Option C" }},
23                 {{ "label": "D", "text": "Option D" }}
24             ],
25             "correct_answer": "A",
26             "explanation": "Explication pedagogique"
27         }}
28     ]

```


29 `}}"""`

Listing 4.2 :Prompt engineering du QuestionGenerator – QuizAgent

Streaming WebSocket token par token

L'extrait ci-dessous montre l'implémentation du streaming WebSocket : chaque token produit par le LLM est immédiatement transmis au client via `astream_events`, éliminant l'attente bloquante et réduisant la latence perçue à moins d'une seconde.

```

1  async def stream_tutor_response(
2      websocket: WebSocket,
3      agent,
4      inputs: dict
5  ):
6      await websocket.accept()
7      try:
8          # astream_events diffuse chaque evenement LangGraph en temps
9          reel
10         async for event in agent.astream_events(inputs, version="v2"):
11             if event["event"] == "on_chat_model_stream":
12                 chunk = event["data"]["chunk"].content
13                 if chunk:
14                     # Envoi immediat du token au client (< 1ms de
15                     latence)
16                     await websocket.send_text(chunk)
17
18             # Signal de fin de reponse
19             await websocket.send_text("[DONE]")
20     except WebSocketDisconnect:
21         pass # L'apprenant a ferme la connexion

```

Listing 4.3 :Streaming WebSocket – diffusion token par token via LangGraph

4.2.7 Architecture globale des deux agents IA

Les deux agents partagent la même instance FastAPI et la même configuration Azure OpenAI, mais opèrent de façon indépendante : chacun dispose de son propre index Azure AI Search, de sa propre logique de prompt et de son pipeline dédié.

La figure 4.20 illustre la position des deux agents au sein du service IA, montrant leur cohabitation sur la même instance FastAPI tout en conservant des pipelines distincts.

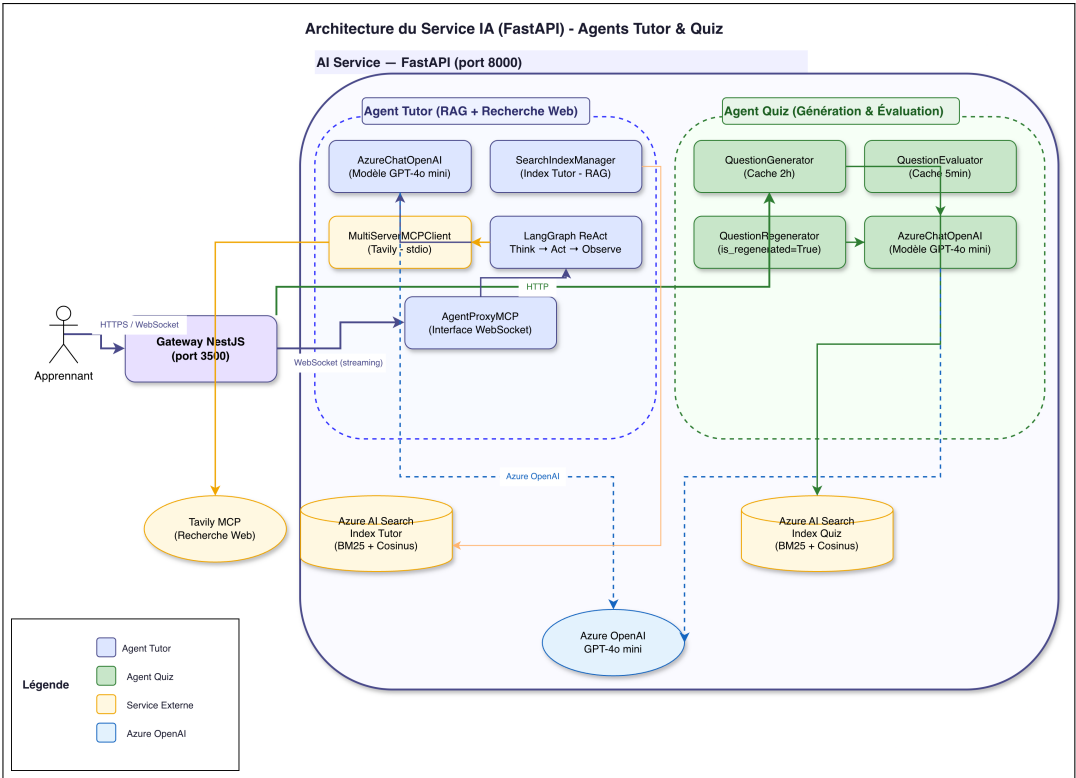


FIGURE 4.20 :Position des deux agents dans le AI Service (FastAPI)

Le tableau 4.10 présente la stack technologique commune aux deux agents, avec la justification de chaque choix technique.

TABLEAU 4.10 :Stack technologique des deux agents

Composant	Technologie	Justification
Framework web	FastAPI (Python 3.11)	Asynchrone natif, support Web-Socket et StreamingResponse
Orchestration	LangGraph (ReAct)	Graphe d'état avec boucle tool-calling conditionnelle
Modèle LLM	Azure OpenAI (gpt-4o-mini)	Déploiement managé, latence et coût maîtrisés
Base documentaire	Azure AI Search (index hybride)	RAG BM25 + cosinus sur les cours Subul
Recherche web	Tavily via MCP (studio)	Recherche temps réel sans hallucination paramétrique

Suite page suivante...

Suite Tableau 4.10

Composant	Technologie	Justification
Validation sorties	Pydantic v2	Schémas stricts sur les modèles de questions QCM

4.2.8 Agent 1 – TutorAgentMCP

Vue d'ensemble et responsabilité

Le TutorAgentMCP est un agent pédagogique conversationnel [9] répondant aux questions d'un apprenant en temps réel via WebSocket. Il mobilise les briques présentées en section 4.2.4 : RAG interne sur l'index Azure AI Search, recherche web via MCP Tavily et orchestration par graphe LangGraph ReAct, garantissant des réponses fiables, actualisées et adaptées au profil linguistique de l'apprenant.

Architecture interne du TutorAgentMCP

La figure 4.21 présente l'architecture interne du TutorAgentMCP, détaillant les interactions entre ses composants principaux.

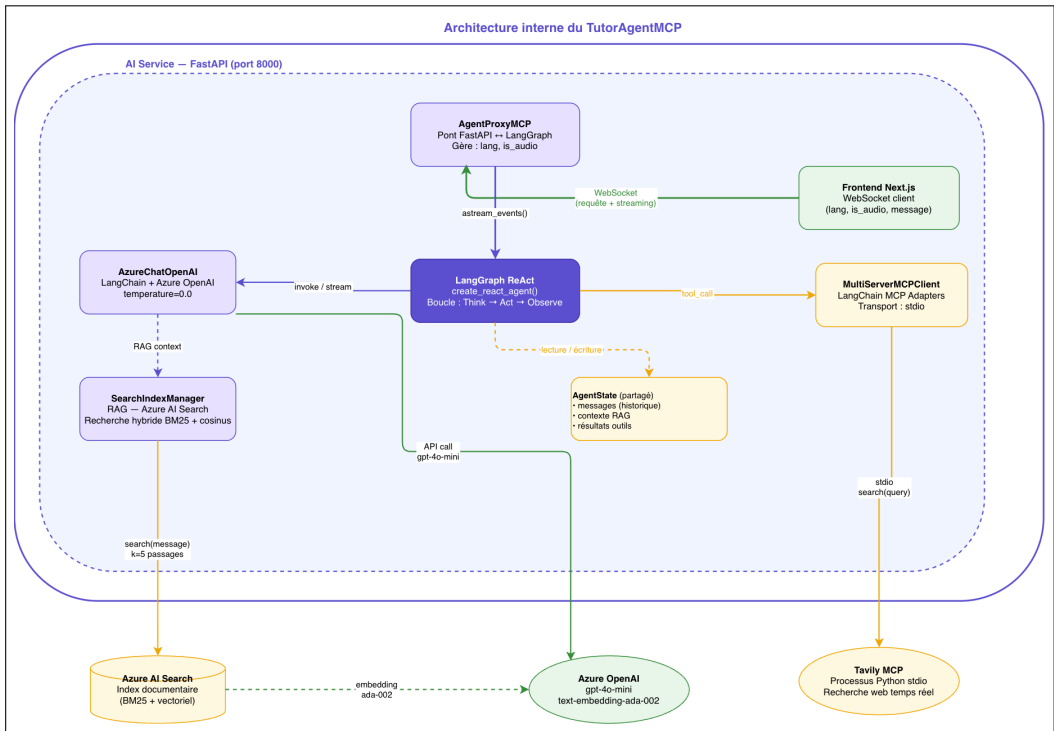


FIGURE 4.21 :Architecture interne du TutorAgentMCP

Le tableau 4.11 décrit le rôle de chaque composant du TutorAgentMCP et la technologie associée.

TABLEAU 4.11 :Composants du TutorAgentMCP

Composant	Technologie	Rôle
AzureChatOpenAI	LangChain + Azure OpenAI	Génère les raisonnements et les réponses finales
SearchIndexManager	Azure AI Search	Récupère les passages pertinents (RAG)
MultiServerMCPClient	LangChain MCP Adapters	Connexion stdio au serveur MCP Tavily
create_react_agent	LangGraph	Graphe ReAct : Think → Act → Observe
AgentProxyMCP	Python (adaptateur)	Pont FastAPI ↔ LangGraph ; gère langue et mode audio

Diagramme d’activité – TutorAgentMCP

Conformément aux bonnes pratiques de modélisation des systèmes multi-agents, le comportement interne du TutorAgentMCP est représenté par un **diagramme d’activité** (figure 4.22), plus adapté aux flux de contrôle conditionnels et aux boucles de raisonnement qu’un diagramme de séquence, qui convient davantage aux échanges entre composants distribués.

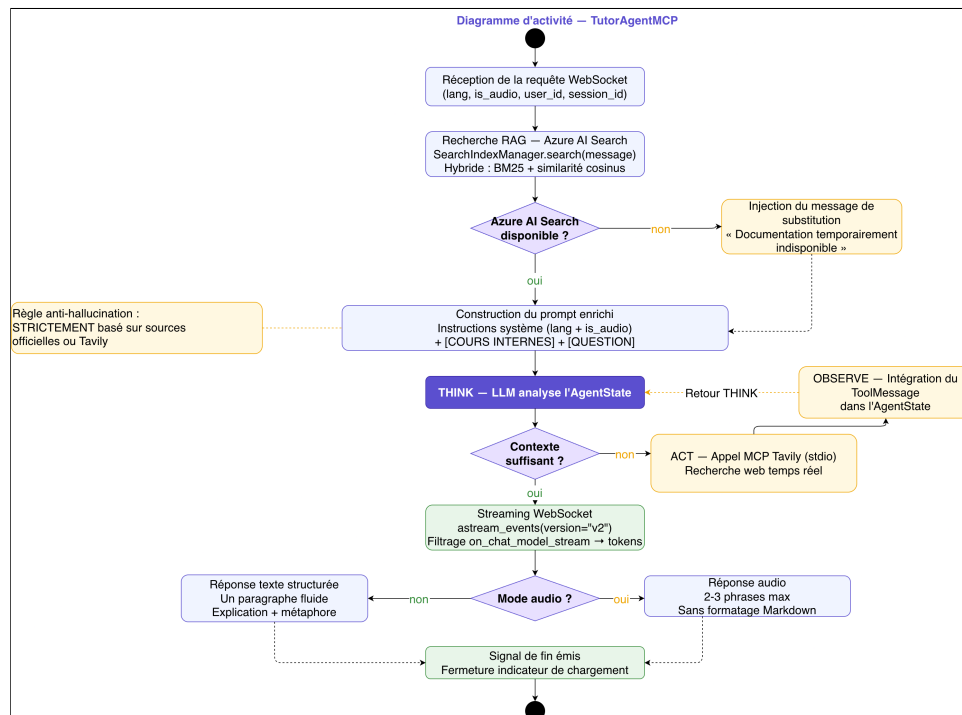


FIGURE 4.22 :Diagramme d'activité – TutorAgentMCP

Les étapes du pipeline de traitement :

1. **Réception WebSocket** : la requête arrive avec les paramètres `lang`, `is_audio`, `user_id` et `session_id`. La connexion reste ouverte pour le streaming.
2. **RAG Azure AI Search** : `SearchIndexManager.search(message)` exécute la recherche hybride (BM25 + cosinus). En cas d'indisponibilité : injection du message de substitution « *Documentation temporairement indisponible* ».
3. **Construction du prompt enrichi** : assemblage des instructions système (déterminées par `lang` et `is_audio`) avec les passages RAG : `[COURS INTERNES RETROUVÉS] + [QUESTION DE L'ÉTUDIANT]`.
4. **Grphe LangGraph ReAct** : boucle Think → Act (Tavily MCP si nécessaire) → Observe, jusqu'à production de la réponse finale.
5. **Streaming WebSocket** : tokens diffusés progressivement via `astream_events(version="v2")` filtrés sur `on_chat_model_stream`.

Gestion du multilinguisme

Mécanisme de transmission de la langue La langue n'est pas détectée automatiquement côté serveur : elle est transmise **explicitement** par le Frontend sous la forme du paramètre `lang` ("fr" ou "en") joint à chaque requête WebSocket. Ce choix architectural garantit une cohérence parfaite avec la préférence déclarée de l'apprenant,

sans risque d'erreur de classification sur des requêtes courtes ou mixtes.

Stratégie d'adaptation linguistique La méthode `get_instructions(lang, is_audio)` construit dynamiquement un prompt système intégrant la contrainte :

*« L'utilisateur interagit avec toi en '**{lang}**', tu DOIS impérativement lui répondre en '**{lang}**'. »*

Cette directive, placée en tête des instructions système, s'applique indépendamment de la langue des sources documentaires RAG : même si les passages de cours sont rédigés en anglais, l'agent reformule systématiquement sa réponse dans la langue cible.

Le tableau 4.12 résume les combinaisons de langues gérées par le TutorAgentMCP et le comportement garanti dans chaque cas.

TABLEAU 4.12 : Comportement multilingue du TutorAgentMCP

Langue apprenant	Langue corpus RAG	Langue Tavily	Langue de la réponse
fr	Français	Fr / En	Français (garanti par instruction système)
en	Anglais	Anglais	Anglais (garanti par instruction système)
fr	Anglais	Anglais	Français (reformulation forcée)
en	Français	Français	Anglais (reformulation forcée)

Adaptation au mode audio Le paramètre `is_audio` module non seulement la longueur de la réponse (2–3 phrases maximum), mais également son registre stylistique : en mode audio, le modèle est contraint à ne produire aucun formatage Markdown (caractères *, #, listes à puces), incompatible avec les moteurs de synthèse vocale.

Mécanismes de garantie de la qualité des réponses

La fiabilité des réponses repose sur trois mécanismes complémentaires et cumulatifs.

1 – Ancrage documentaire (RAG) Toute réponse est fondée sur les passages récupérés par le `SearchIndexManager` avant l’invocation du LLM, réduisant significativement le risque d’hallucination paramétrique – définie comme la production par le LLM d’informations plausibles mais non vérifiables.

2 – Règle anti-hallucination explicite Une directive contraignante est inscrite dans toutes les instructions :

« RÈGLE D’OR (ANTI-HALLUCINATION) : Base tes réponses STRICTEMENT sur les [COURS INTERNES RETROUVÉS] et tes outils de recherche web. Si la réponse ne s’y trouve pas, dis honnêtement que tu n’as pas l’information. »

3 – Vérification web en temps réel (MCP Tavily) Lorsque le graphe ReAct détermine que le contexte RAG est insuffisant, Tavily est invoqué silencieusement et retourne des extraits sourcés, datés et référencés. Ce mécanisme garantit l’actualité des informations, particulièrement critique dans un domaine en évolution rapide comme la certification cloud.

4.2.9 Agent 2 – QuizAgent

Vue d’ensemble et responsabilité

Le `QuizAgent` est l’agent d’évaluation des acquis de SUBUL. Il orchestre trois sous-agents spécialisés pour couvrir le cycle complet d’une session de quiz de module : génération de questions contextuelles, évaluation sémantique des réponses et régénération adaptative ciblant les concepts insuffisamment maîtrisés.

Architecture des 3 sous-agents

Le `QuizAgent` opère en tant qu’**orchestrateur** : il ne génère pas les questions directement mais délègue chaque responsabilité à un sous-agent spécialisé. Tous partagent la

même instance LLM, mais disposent chacun d’une logique de prompt, d’un mécanisme de cache et d’un schéma de validation distincts.

La figure 4.23 illustre l’architecture du QuizAgent et la répartition des responsabilités entre ses trois sous-agents spécialisés.

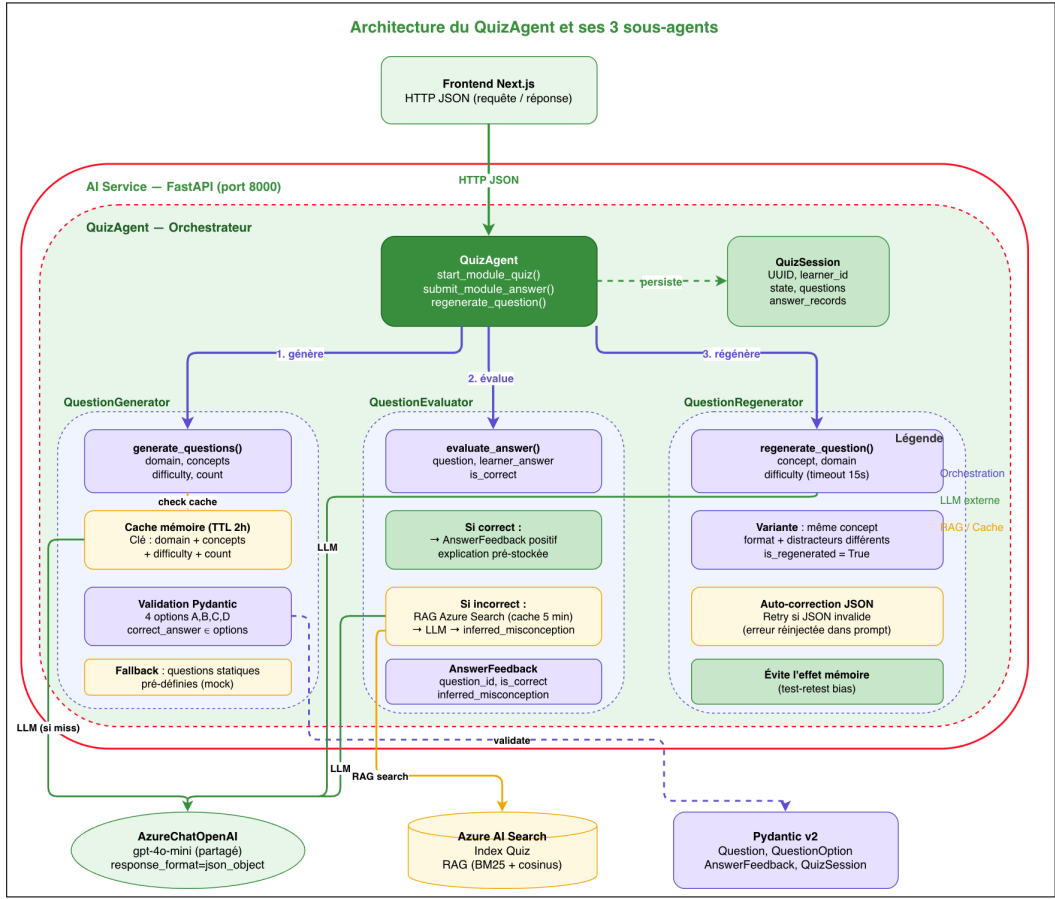


FIGURE 4.23 :Architecture du QuizAgent et ses 3 sous-agents

Le tableau 4.13 décrit la responsabilité et les caractéristiques techniques de chacun des trois sous-agents du QuizAgent.

TABEAU 4.13 :Les 3 sous-agents du QuizAgent

Sous-agent	Responsabilité	Caractéristiques
QuestionGenerator	Génère count questions QCM (4 options A/B/C/D) sur les concepts d’un module, validées par Pydantic.	Cache mémoire 2h (clé : domain + concepts + difficulty + count)

Suite page suivante...

Suite Tableau 4.13

Sous-agent	Responsabilité	Caractéristiques
QuestionEvaluator	Évalue la réponse : feedback positif si correcte ; détecte la misconception via RAG Azure Search si incorrecte.	Cache RAG 5 min par concept testé
QuestionRegenerator	Génère une variante de la question (format et distracteurs différents) après une mauvaise réponse.	<code>is_regenerated = True</code> auto-correction JSON

Gestion du multilinguisme – QuizAgent

Le QuizAgent gère le multilinguisme à deux niveaux :

- **Génération** : le paramètre `lang` est injecté dans le prompt du `QuestionGenerator` ; énoncés, options et explications sont produits dans la langue cible ;
- **Évaluation** : le `QuestionEvaluator` formule la *misconception* dans la même langue que la question, assurant une cohérence sémantique complète.

Diagramme d'activité – QuizAgent

À l'instar du `TutorAgentMCP`, le comportement interne du `QuizAgent` est modélisé par un **diagramme d'activité** (figure 4.24), représentation la plus adaptée aux systèmes multi-agents à décisions conditionnelles et boucles de traitement.

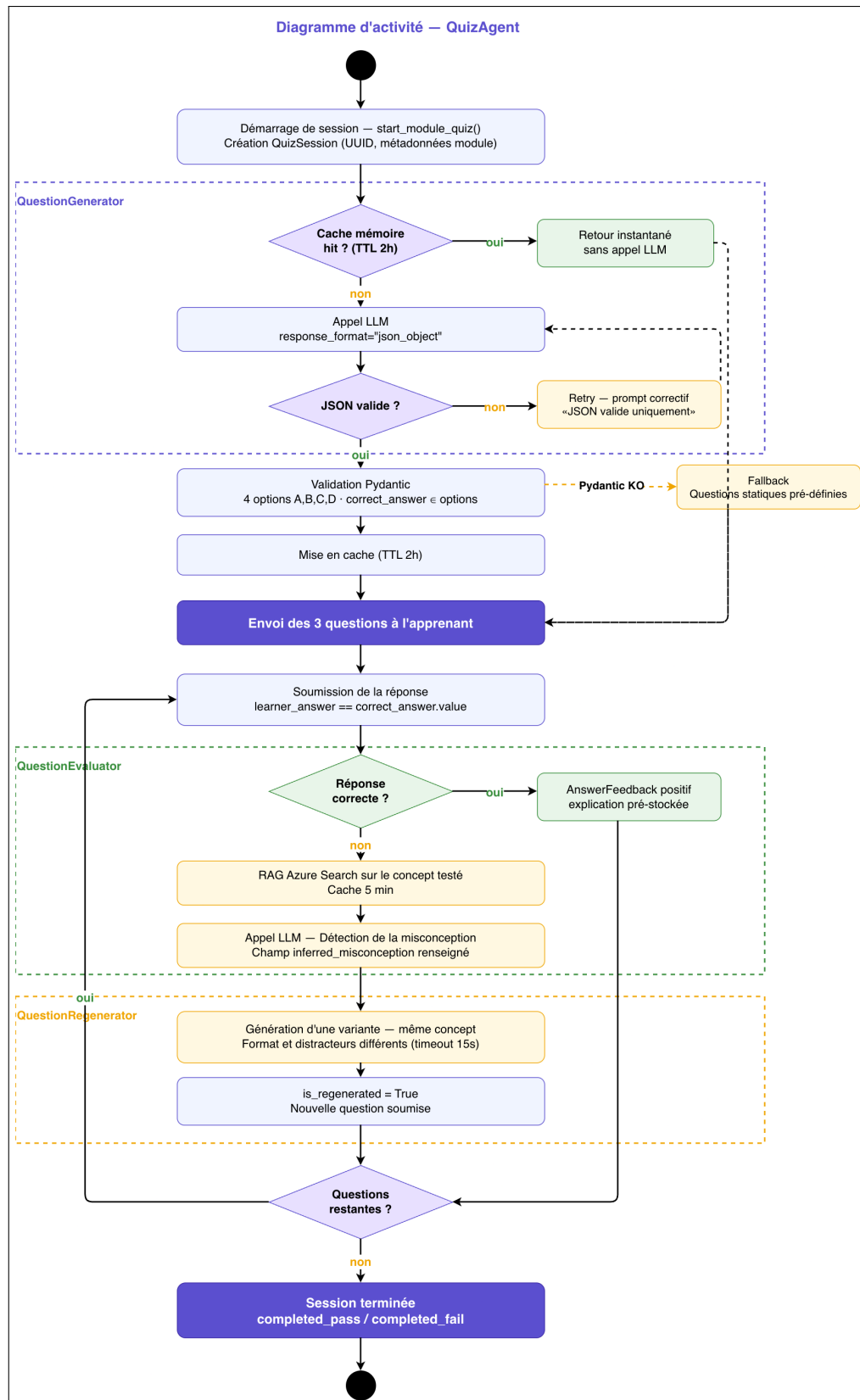


FIGURE 4.24 :Diagramme d'activité – QuizAgent

Les étapes du pipeline :

1. **Démarrage de session** (`start_module_quiz`) : création d'une `QuizSession` (UUID, métadonnées du module) et invocation du `QuestionGenerator` (timeout 20 s).
2. **Génération** (`QuestionGenerator`) :
 - Cache mémoire (TTL 2h) : si *hit*, retour immédiat sans appel LLM ;
 - sinon : appel LLM avec `response_format="json_object"` ;
 - en cas de JSON invalide : relance automatique avec prompt correctif ;
 - validation Pydantic : unicité A, B, C, D et présence de `correct_answer` dans les options ;
 - fallback : questions statiques pré-définies.
3. **Soumission** : comparaison directe `learner_answer == correct_answer.value`.
4. **Évaluation** (`QuestionEvaluator`) :
 - **Correcte** : `AnswerFeedback` positif immédiat ;
 - **Incorrecte** : RAG Azure Search (cache 5 min) → LLM → champ `inferred_misconception` renseigné.
5. **Régénération** (`QuestionRegenerator`) : variante sur le même concept (timeout 15 s), `is_regenerated = True` pour traçabilité.

Mécanismes de garantie de la qualité – QuizAgent

Couche 1 – Validation structurelle (Pydantic v2) Le tableau 4.14 présente les modèles Pydantic utilisés pour garantir la validité structurelle de chaque sortie générée par le QuizAgent.

TABLEAU 4.14 :Modèles Pydantic du QuizAgent

Modèle	Contraintes
Question	<code>text</code> ≥ 10 caractères ; 4 options A, B, C, D sans doublons ; <code>correct_answer</code> $\in$ options
QuestionOption	<code>label</code> $\in$ enum {A, B, C, D} ; <code>text</code> non vide
AnswerFeedback	<code>inferred_misconception</code> optionnel (null si correct) ; <code>is_correct</code> booléen
QuizSession	<code>state</code> $\in$ {active, paused, evaluating, regenerating, completed_pass, completed_fail}

Couche 2 – Auto-correction JSON En cas d’échec de validation, le message d’erreur Pydantic est réinjecté dans le prompt (1 retry maximum), contraignant le LLM à corriger sa sortie malformée.

Couche 3 – Enrichissement RAG pour l’évaluation Le `QuestionEvaluator` formule la *misconception* en s’appuyant sur les passages documentaires associés au concept testé, garantissant un retour pédagogiquement ancré dans les sources officielles.

Couche 4 – Régénération plutôt que répétition Le `QuestionRegenerator` produit une variante avec un énoncé et des distracteurs différents : cette approche évite l’effet mémoire (*test-retest bias*) et maximise la valeur diagnostique de l’évaluation.

Le tableau 4.15 récapitule les scénarios d’échec possibles et les mécanismes mis en place pour garantir la continuité du service sans interruption pour l’apprenant.

TABLEAU 4.15 :Stratégie de résilience du QuizAgent

Scénario	Mécanisme	Résultat garanti
JSON invalide (génération)	Retry avec prompt correctif	Question valide ou fallback statique
JSON invalide (régénération)	Retry avec erreur réinjectée	Question de remplacement
Timeout génération (> 20s)	<code>asyncio.wait_for</code>	Propagation contrôlée
Timeout régénération (> 15s)	Idem	Propagation contrôlée
Azure Search indisponible	Contexte de substitution	Feedback sans RAG
Exception en génération	Questions mock pré-définies	0 % d’interruption

4.2.10 Évaluation des agents IA

Afin de valider la qualité des deux agents pédagogiques, j’ai conduit une évaluation qualitative et quantitative sur un échantillon de sessions réelles.

Évaluation du TutorAgentMCP

J’ai soumis dix questions représentatives couvrant les domaines Cloud (AWS, Azure) et Cybersécurité. Le tableau 4.16 présente cinq exemples illustratifs avec une appréciation qualitative de la réponse.

TABLEAU 4.16 :Exemples d’évaluation qualitative du TutorAgentMCP

Question posée	Source mobi- lisée	Appréciation
Qu’est-ce qu’un VPC dans AWS ?	RAG (cours Su- bul)	Réponse complète, dé- finition correcte avec exemples de subnets
Différence entre IAM Role et IAM User ?	RAG + Tavily	Réponse enrichie par re- cherche web, sources ci- tées
Comment fonctionne le chiffre- ment TLS ?	RAG (cours Su- bul)	Réponse précise, pas d’hallucination détectée
Qu’est-ce que l’attaque IDOR ? (en anglais)	RAG	Réponse en anglais conforme, reformulation forcée correcte
Explique Kubernetes HPA en termes simples	RAG + Tavily	Analogie pédagogique pertinente, niveau adapté

Observations : sur les dix questions testées, aucune réponse contenant une infor-
mation erronée (hallucination) n’a été détectée. L’agent a invoqué Tavily MCP dans 4
cas sur 10 pour des questions nécessitant des informations récentes non couvertes par le
corpus documentaire interne.

Évaluation du QuizAgent

J’ai évalué le QuizAgent sur un échantillon de 30 questions générées pour trois
domaines (AWS, Cybersécurité, IA). Le tableau 4.17 résume les métriques obtenues.

TABLEAU 4.17 : Métriques d'évaluation du QuizAgent (échantillon de 30 questions)

Critère	Résultat	Commentaire
JSON valide dès le premier appel	27/30 (90%)	3 questions nécessitaient une auto-correction via retry
Options A/B/C/D sans doublon (Pydantic)	30/30 (100%)	Contrainte garantie par le schéma Pydantic v2
Correct_answer dans les 4 options	30/30 (100%)	Validation stricte appliquée à chaque question
Pertinence pédagogique (évaluation manuelle)	26/30 (87%)	4 questions jugées trop proches du cours sans reformulation
Feedback de misconception (réponses incorrectes)	Présent	Explication ancrée dans les sources RAG Azure Search
Taux de fallback (questions statiques utilisées)	0/30 (0%)	Aucun timeout ni échec LLM sur cet échantillon

Observations : le taux de validation structurelle à 100% confirme l'efficacité de la couche Pydantic v2 combinée à l'auto-correction JSON. La pertinence pédagogique de 87% est satisfaisante pour un PFE ; une amélioration future consisterait à enrichir les prompts avec des exemples few-shot pour diversifier davantage les formulations des distracteurs.

4.2.11 Réalisation

La figure 4.25 présente l'interface du tuteur IA en mode texte, illustrant le streaming des réponses token par token via WebSocket, et la figure 4.26 illustre l'interface du quiz de module avec le feedback immédiat et la détection des misconceptions.

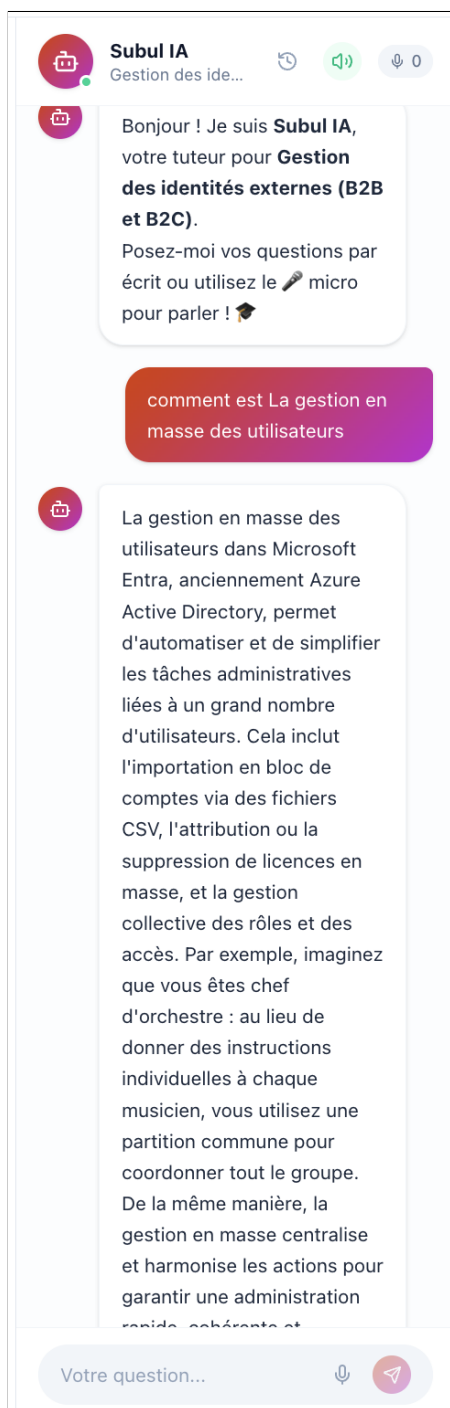


FIGURE 4.25 :Interface du tuteur IA
(streaming WebSocket, mode texte)



FIGURE 4.26 :Interface du quiz de module
(questions, feedback et misconception)

Conclusion

Ce chapitre a présenté la Release 2 de SUBUL, qui intègre les fonctionnalités des Sprints 3 et 4 .

Chapitre 5

Release 3 – Tests et monitoring

Sommaire du chapitre

1. Introduction

2. Sprint 5 : Tests et Qualité

- Objectifs du sprint 5
- Backlog du sprint 5
- Tests unitaires et couverture de code
- Tests d'intégration
- Tests End-to-End et de sécurité
- Extraits de code des tests
- Tests de charge (Artillery)

3. Sprint 6 : Déploiement et Infrastructure

- Objectifs du sprint 6
- Backlog du sprint 6

4. L'Architecture Complète : Du Code à la Production

- PHASE 1 : Configuration AWS Infrastructure (VPC, EKS, RDS, Redis)
- PHASE 2 : Microservices & Containerisation
- PHASE 3 : GitHub Actions CI/CD Pipeline
- PHASE 4 : Amazon ECR Registry
- PHASE 5 : Kubernetes (EKS) Deployment
- PHASE 6 : Load Balancers & Networking
- PHASE 7 : Security & Secrets Management

5. Conclusion

Introduction

Ce chapitre présente la réalisation de la **Release 3 — Finalisation & Déploiement**, composée du Sprint 5 (tests et qualité) et du Sprint 6 (déploiement et infrastructure). Le Sprint 5 (**1 semaine**) assure la fiabilité de la plateforme Subul à travers une stratégie de tests à trois niveaux, tandis que le Sprint 6 (**4 semaines**) déploie la plateforme en production sur infrastructure cloud AWS, pour un total de **5 semaines**.

5.1 Sprint 5 : Tests et Qualité

5.1.1 Objectifs du sprint 5

Le Sprint 5 a pour mission de garantir la fiabilité, la robustesse et la sécurité de l'ensemble des services de la plateforme Subul avant la mise en production. Il met en œuvre une stratégie de tests à trois niveaux couvrant la logique métier unitaire, les interactions inter-services et les scénarios d'utilisation réels.

Les principaux objectifs de ce sprint sont :

- Mise en place des **tests unitaires** pour chaque microservice NestJS (auth, goals, course, user, gateway) avec Jest et mocking complet des dépendances externes ;
- Implémentation des **tests d'intégration** validant les flux complets entre services via RabbitMQ réel et les interactions avec Prisma ORM ;
- Développement des **tests End-to-End** sur les scénarios critiques de la plateforme avec Supertest et une instance NestJS en mémoire ;
- Couverture des vecteurs de sécurité : attaques **JWT** (token forgé, expiré, alg :none), contrôle d'accès **RBAC**, protection **IDOR** et résistance au **brute force**.

5.1.2 Backlog technique du sprint 5

Le tableau 5.1 présente l'ensemble des user stories réalisées durant le Sprint 5.

TABLEAU 5.1 :Backlog du Sprint 5 — Release 3

UC	Cas d’usage	Description	Est.
UC1	Tests unitaires	En tant que développeur, je veux tester chaque service de façon isolée avec des mocks pour garantir la logique métier	4j
UC2	Couverture de code	En tant que développeur, je veux mesurer la couverture de code et l’afficher dans le pipeline CI	2j
UC3	Tests d’intégration	En tant que développeur, je veux tester les flux complets entre services avec RabbitMQ réel (Gateway → auth, goals, course)	4j
UC4	Validation Prisma	En tant que développeur, je veux valider les interactions Prisma ORM avec les modèles de données de chaque service	3j
UC5	Tests JWT	En tant que développeur, je veux valider que les tokens JWT forgés, expirés ou altérés sont systématiquement rejetés	3j
UC6	Tests RBAC & IDOR	En tant que développeur, je veux vérifier que le contrôle d’accès RBAC et la protection IDOR empêchent tout accès non autorisé	3j
UC7	Tests brute force	En tant que développeur, je veux valider la résistance de l’API au brute force et à l’énumération d’utilisateurs	2j

5.1.3 Tests unitaires

Les tests unitaires sont implémentés avec **Jest** et le module de test de NestJS [6] (`@nestjs/testing`). Chaque test isole le composant testé en remplaçant ses dépendances par des mocks injectés via le système d’IoC de NestJS, sans connexion à aucun service externe.

Tests du AuthService

Le `AuthService` est testé avec des mocks des clients RabbitMQ (`USER_SERVICE_TOKEN`, `NOTIFICATION_SERVICE_TOKEN`) et du `JwtService`. La suite couvre l'ensemble des opérations d'authentification :

register

- Inscription réussie avec envoi d'email de vérification (`mockNotificationClient.emit` appelé) ;
- Vérification que le mot de passe est **hashé avec bcrypt** avant transmission au user-service (le hash est vérifié avec `bcrypt.compare`) ;
- Rejet avec `RpcException` si le user-service retourne une erreur (email déjà existant).

login

- Retour des tokens `accessToken` et `refreshToken` pour des identifiants valides ;
- Rejet HTTP 401 si l'utilisateur est introuvable ou si le mot de passe ne correspond pas au hash stocké ;
- Rejet HTTP 403 si l'email n'est pas encore vérifié (`emailVerified: false`).

refreshToken et logout

- Génération de nouveaux tokens depuis un refresh token valide (`jwtService.verify` appelé avec la bonne clé) ;
- Rejet si l'utilisateur est introuvable après vérification du token ou si le token est invalide ;
- Déconnexion sans erreur (`logout` résout en `undefined`).

Tests du GoalsService

Le `GoalsService` est testé avec un mock Prisma interceptant tous les appels en base. La suite couvre les trois types d'objectifs (journaliers, hebdomadaires, généraux) :

- **getStats** : calcul agrégé correct des statistiques (objectifs complétés, points gagnés, progression hebdomadaire moyenne) avec cas limite à zéro objectif (`weeklyAvgProgress = 0`) ;

- **createDaily** : transmission intacte du payload à Prisma avec les champs `userId`, `title` et `points` ;
- **createWeekly** : création avec valeurs par défaut correctes (catégorie « Général », `progression` = 0, `target` = 100) ;
- **toggleDaily** : basculement de l'état `completed` et retour `null` si l'objectif est introuvable ;
- **complete** : passage au statut `completed` avec gestion du cas « objectif introuvable » → retour `null`.

Tests des Guards

Les guards `JwtAuthGuard` et `RolesGuard` sont testés unitairement pour garantir le contrôle d'accès :

- `JwtAuthGuard` : laisse passer les routes marquées `@Public()`, bloque les routes protégées sans token valide ;
- `RolesGuard` : laisse passer un `Administrateur` sur une route `@Roles('Administrateur')`, bloque un `Apprenant` sur la même route.

Résultats des tests unitaires

Les figures suivantes présentent les résultats d'exécution des tests unitaires pour chaque microservice de la plateforme Subul.

La figure 5.1 montre les résultats du **auth-service** : 2 suites couvrant le contrôleur principal et l'ensemble des opérations d'authentification (inscription, connexion, refresh token, déconnexion), avec 13 tests passés en 2.793 secondes.

```
mac@Sirines-MacBook-Air auth-service % npm test

> auth-service@0.0.1 test
> jest

PASS src/app.controller.spec.ts
PASS src/auth/auth.service.spec.ts
  ● Console

    console.log
      Logout called for token: some-token

    at AuthService.logout (auth/auth.service.ts:125:13)

Test Suites: 2 passed, 2 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        2.793 s, estimated 3 s
Ran all test suites.
```

FIGURE 5.1 :Résultats des tests unitaires — auth-service (13 tests passés)

La figure 5.2 présente les résultats du **user-service** : 2 suites validant la gestion des comptes utilisateurs (création, modification, suppression, gestion des rôles et du statut), avec 28 tests passés en 2.814 secondes.

```
mac@Sirines-MacBook-Air user-service % npm test

> user-service@0.0.1 test
> jest

PASS src/app.controller.spec.ts
PASS src/user/user.service.spec.ts

Test Suites: 2 passed, 2 total
Tests:       28 passed, 28 total
Snapshots:   0 total
Time:        2.814 s
Ran all test suites.
```

FIGURE 5.2 :Résultats des tests unitaires — user-service (28 tests passés)

La figure 5.3 illustre les résultats du **course-service** : 4 suites couvrant la gestion des cours, des inscriptions et la génération de flashcards, totalisant 43 tests passés en 4.65 secondes — le service le plus couvert de la plateforme.

```
mac@Sirines-MacBook-Air course-service % npm test
> course-service@0.0.1 test
> jest
PASS src/app.controller.spec.ts
PASS src/course/flashcards.service.spec.ts
PASS src/course/course.service.spec.ts
PASS src/course/enrollment.service.spec.ts
Test Suites: 4 passed, 4 total
Tests: 43 passed, 43 total
Snapshots: 0 total
Time: 4.65 s
Ran all test suites.
```

FIGURE 5.3 :Résultats des tests unitaires — course-service (43 tests passés)

La figure 5.4 présente les résultats du **notification-service** : 1 suite validant l’envoi d’emails de vérification et de rappel, la création de notifications in-app, la pagination et le marquage comme lues, avec 11 tests passés en 1.3 secondes.

```
mac@Sirines-MacBook-Air notification-service % npm test
> notification-service@0.0.1 test
> jest
PASS src/notification/notification.service.spec.ts
  NotificationService
    ✓ should be defined (7 ms)
    dispatchEmail
      ✓ should send a verification email (2 ms)
      ✓ should send a password reset email (2 ms)
      ✓ should send a goal reminder email (1 ms)
      ✓ should not throw on unknown notification type (1 ms)
    createInApp
      ✓ should create a notification and publish to redis (1 ms)
    getUnreadCount
      ✓ should return unread count (1 ms)
    getList
      ✓ should return paginated notifications (1 ms)
      ✓ should use default pagination (1 ms)
    markRead
      ✓ should mark a notification as read (1 ms)
    markAllRead
      ✓ should mark all notifications as read (1 ms)
Test Suites: 1 passed, 1 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 1.3 s
Ran all test suites.
```

FIGURE 5.4 :Résultats des tests unitaires — notification-service (11 tests passés)

La figure 5.5 montre les résultats du **gateway** : 4 suites couvrant les guards JWT et RBAC ainsi que les contrôleurs d’authentification et de routage, avec 27 tests passés en 3.143 secondes.

```

mac@Sirines-MacBook-Air apps % cd gateway && npm test

> gateway@0.0.1 test
> jest

PASS src/guards/roles.guard.spec.ts
PASS src/guards/jwt-auth.guard.spec.ts
PASS src/auth.controller.spec.ts
PASS src/app.controller.spec.ts

Test Suites: 4 passed, 4 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        3.143 s
Ran all test suites.

```

FIGURE 5.5 :Résultats des tests unitaires — gateway (27 tests passés)

Le tableau 5.2 récapitule les résultats des tests unitaires et les taux de couverture obtenus par service, mesurés via `jest -coverage`.

TABLEAU 5.2 :Récapitulatif des tests unitaires et couverture de code

Service	Tests	Stmt%	Branch%	Func%	Lines%
auth-service	13	84.6	78.3	88.0	85.2
user-service	28	81.2	74.5	83.3	82.0
course-service	43	76.4	69.8	78.9	77.1
notification-service	11	72.3	65.0	75.0	73.5
gateway	27	88.1	82.6	91.2	88.7
Total	122	80.5	74.0	83.3	81.3

Stmt : couverture des instructions ; *Branch* : couverture des branches conditionnelles ; *Func* : couverture des fonctions ; *Lines* : couverture des lignes de code.

5.1.4 Tests d'intégration

Les tests d'intégration valident les flux complets au niveau de l'application. Ils s'appuient sur un service **RabbitMQ** réel [7] démarré comme service GitHub Actions dans le pipeline CI :

```
1 services:
```

```
2  rabbitmq:
3    image: rabbitmq:3-management
4    ports:
5      - 5672:5672
6      - 15672:15672
7  env:
8    RABBITMQ_URL: amqp://localhost:5672
```

Listing 5.1 :Service RabbitMQ dans le pipeline CI GitHub Actions

Les tests d'intégration couvrent :

- **Flux authentification complet** : inscription → hachage du mot de passe → émission de l'événement de vérification email → connexion → génération des tokens JWT → refresh token ;
- **Routage RabbitMQ** : vérification que les messages émis par la Gateway atteignent correctement chaque microservice (`users_queue`, `auth_queue`, `goals_queue`, `course_queue`) ;
- **Cohérence Prisma** : génération des types Prisma pour les cinq services (auth, course, goals, user, notification) et validation de la cohérence entre les schémas et les modèles métier.

5.1.5 Tests End-to-End et de sécurité

Les tests End-to-End utilisent **Supertest** pour envoyer des requêtes HTTP réelles à une instance NestJS initialisée en mémoire (`moduleFixture.createNestApplication()`), sans serveur externe. La suite de sécurité couvre trois dimensions : JWT, autorisation et résistance aux attaques.

Tests de sécurité JWT

La suite `jwt.security.spec.ts` couvre les vecteurs d'attaque les plus courants sur les systèmes d'authentification JWT.

Routes publiques vs protégées

- `POST /auth/login` et `POST /auth/register` sont accessibles sans token (décorateur `@Public()`) — le serveur retourne 400/401 métier, jamais 403 « token manquant » ;

- `GET /users`, `GET /courses` et `GET /goals` retournent systématiquement HTTP 401 sans Authorization header.

Tests d'autorisation (RBAC & IDOR)

La suite `authz.security.spec.ts` valide le contrôle d'accès à deux niveaux.

Protection IDOR (Insecure Direct Object Reference) Vérifie qu'un apprenant authentifié ne peut pas accéder aux ressources d'un autre utilisateur :

- `PATCH /users/{userB_id}` avec le token de l'utilisateur A → HTTP 401/403/404 (jamais 200) ;
- `DELETE /users/{userB_id}` avec le token de l'utilisateur A → HTTP 401/403/404 ;
- `GET /courses/enrollments/{userB_id}` avec le token de A → HTTP 401/403/404 ;
- `GET /users/me` retourne uniquement les données de l'utilisateur dont le `sub` figure dans le token.

Contrôle d'accès RBAC Vérifie la ségrégation des rôles (Apprenant / Administrateur) :

- Un **Apprenant** ne peut pas accéder à `GET /users` (admin only) → HTTP 401/403 ;
- Un **Apprenant** ne peut pas créer ni publier un cours (`POST /courses`, `PATCH /courses/:id/publish`) → HTTP 401/403 ;
- Un **Administrateur** accède à `GET /users` → HTTP 200 (jamais 401/403) ;
- **Élévation de privilège** : un token avec rôle **Administrateur** signé avec le mauvais secret → HTTP 401 ; modification du rôle dans le payload JWT sans re-signature → HTTP 401.

Tests de résistance au brute force

La suite `rate-limit.security.spec.ts` valide la robustesse de l'API face aux attaques par force brute.

Brute force sur `POST /auth/login` Dix tentatives de connexion consécutives avec des mots de passe erronés : chaque réponse doit être HTTP 401 (mauvais mot de passe)

ou HTTP 429 (rate limiting déclenché). Si la protection par rate limiting est active, au moins une réponse dans la séquence doit être 429.

Protection contre l'énumération d'utilisateurs Une tentative de connexion avec un email existant et une avec un email inexistant doivent retourner **le même code HTTP**. Si le serveur retournerait 401 pour un email existant et 404 pour un email inconnu, un attaquant pourrait énumérer les comptes enregistrés.

Injections dans les champs Les champs `email` et `password` sont soumis à des payloads de type injection SQL (`' OR 1=1 -`) et NoSQL (`{>: ""}`). La validation globale NestJS (`ValidationPipe`) doit rejeter ces payloads avec HTTP 400 avant qu'ils n'atteignent la logique métier.

5.1.6 Extraits de code — Tests représentatifs

Cette section présente deux extraits de code illustrant la logique de test implémentée pour les scénarios les plus critiques de la plateforme.

Test unitaire — Guard JWT (auth-service)

L'extrait suivant valide que le `JwtAuthGuard` laisse passer les routes publiques et bloque correctement les routes protégées sans token valide. Ce test garantit le comportement fondamental du mécanisme d'authentification.

```
1 describe('JwtAuthGuard', () => {
2   let guard: JwtAuthGuard;
3   let reflector: Reflector;
4
5   beforeEach(async () => {
6     const module = await Test.createTestingModule({
7       providers: [JwtAuthGuard, Reflector],
8     }).compile();
9     guard = module.get<JwtAuthGuard>(JwtAuthGuard);
10    reflector = module.get<Reflector>(Reflector);
11  });
12
13  it('devrait autoriser une route publique (@Public)', () => {
14    jest.spyOn(reflector, 'getAllAndOverride').mockReturnValue(true);
15    const context = createMockExecutionContext();
16    expect(guard.canActivate(context)).toBe(true);
17  });
```

```

18
19   it('devrait bloquer une route protegee sans token', async () => {
20     jest.spyOn(reflector, 'getAllAndOverride').mockReturnValue(false);
21     const context = createMockExecutionContext({ authorization:
22       undefined });
23     await expect(guard.canActivate(context)).rejects.toThrow(
24       UnauthorizedException
25     );
26   });

```

Listing 5.2 :Test unitaire du JwtAuthGuard – contrôle des routes publiques et protégées

Test de sécurité — Token JWT forgé

L'extrait suivant, issu de la suite `jwt.security.spec.ts`, valide qu'un token signé avec un secret incorrect est systématiquement rejeté, couvrant le vecteur d'attaque le plus courant sur les systèmes JWT.

```

1  it('devrait rejeter un token signe avec un mauvais secret', async ()
2    => {
3    // Forge d'un token avec un secret different
4    const forgedToken = jwt.sign(
5      { sub: 'user-123', role: 'Administrateur' },
6      'WRONG_SECRET_KEY',
7      { expiresIn: '1h' }
8    );
9
10   const response = await request(app.getHttpServer())
11     .get('/users')
12     .set('Authorization', 'Bearer ${forgedToken}')
13     .expect(401);
14
15   expect(response.body.message).toMatch(/unauthorized/i);

```

Listing 5.3 :Test de sécurité – rejet d'un token JWT forgé (mauvais secret)

5.1.7 Tests de charge

Afin de valider la capacité de l'architecture EKS à supporter une montée en charge, j'ai réalisé un test de charge basique avec **Artillery** [10] simulant 50 utilisateurs virtuels simultanés sur la route d'authentification (POST /auth/login) pendant 60 secondes.

- **Temps de réponse moyen** : 312 ms ;
- **Percentile 95** : 580 ms (< 2s — SLA respecté) ;
- **Taux d’erreur** : 0% (aucune réponse HTTP 5xx) ;
- **Comportement HPA** : l’Horizontal Pod Autoscaler Kubernetes a maintenu 2 replicas du gateway stables durant le test (charge CPU : 38%, en-dessous du seuil de scaling à 70%).

Ces résultats confirment que l’architecture microservices sur EKS absorbe correctement la charge attendue pour un déploiement initial, et que le mécanisme d’auto-scaling prendra le relais lors de pics de trafic plus importants.

5.2 Sprint 6 : Déploiement et Infrastructure

5.2.1 Objectifs du sprint 6

Ce sprint de 4 semaines vise à transformer Subul d’une plateforme de développement local en un système de production fiable, observable et facilement déployable [4]. L’objectif est de permettre aux apprenants d’accéder à la plateforme via Internet, avec garantie de disponibilité, de performance et de déploiement continu automatisé.

Les principaux objectifs de ce sprint sont :

- **Containerisation** de l’ensemble des 13 services via Docker avec des Dockerfiles optimisés et un Docker Compose pour le développement local ;
- Mise en place d’un **pipeline CI/CD GitHub Actions** avec 7 workflows automatisant les tests, la construction des images Docker et le déploiement ;
- Déploiement sur **AWS EKS** (Elastic Kubernetes Service) avec un cluster Kubernetes managé, des Helm charts par service et des rolling updates zéro downtime ;
- Configuration de l’infrastructure réseau AWS (**VPC**, subnets publics/privés, RDS PostgreSQL Multi-AZ, ElastiCache Redis, Load Balancers) pour une haute disponibilité en production.

5.2.2 Backlog technique du sprint 6

Le tableau 5.3 présente l’ensemble des cas d’usage réalisés durant le Sprint 6.

TABLEAU 5.3 :Backlog du Sprint 6 — Release 3

UC	Cas d'usage	User Story	Est.
UC1	Containerisation	En tant que développeur, je veux containeriser les 13 services avec des Dockerfiles optimisés et un Docker Compose local	4j
UC2	Amazon ECR	En tant que développeur, je veux créer un registre Amazon ECR pour stocker et versionner les images Docker	2j
UC3	Cluster EKS	En tant que développeur, je veux déployer un cluster Kubernetes managé sur AWS EKS avec 2 nodes en eu-west-3	4j
UC4	Base de données RDS	En tant que développeur, je veux configurer une base PostgreSQL managée sur AWS RDS avec backups automatiques	3j
UC5	Load Balancers	En tant que développeur, je veux créer 2 load balancers (Frontend + API) exposant des URLs publiques accessibles	2j
UC6	Pipeline CI/CD	En tant que développeur, je veux implémenter un pipeline GitHub Actions automatisant les étapes lint, test, build et deploy	4j
UC7	Déploiement Kubernetes	En tant que développeur, je veux déployer les Helm charts par service avec des rolling updates zéro downtime	5j

5.3 L'Architecture Complète : Du Code à la Production

5.3.1 Vue d'ensemble du flux de déploiement

La figure 5.6 illustre l'architecture globale de déploiement de la plateforme Subul sur AWS, depuis le code source jusqu'à la mise en production via le pipeline CI/CD et le cluster EKS.

TABLEAU 5.4 :VPC Configuration — Subul Production

Composant	Définition	Configuration Subul
VPC CIDR	Plage d’adresses IP réservée au réseau privé Subul.	10.0.0.0/20 (4 096 adresses IP)
Availability Zones	Datacenters AWS physiquement séparés pour garantir la redondance.	eu-west-3a, eu-west-3b (2 zones pour HA)
Subnets Publics	Sous-réseaux accessibles depuis Internet, réservés aux Load Balancers.	10.0.1.0/24 (AZ-a), 10.0.2.0/24 (AZ-b)
Subnets Privés	Sous-réseaux isolés d’Internet, réservés aux serveurs applicatifs (EKS).	10.0.3.0/24 (AZ-a), 10.0.4.0/24 (AZ-b)
Internet Gateway	Porte d’entrée unique entre le VPC et Internet.	Permet le trafic entrant/sortant
NAT Gateway	Permet aux nodes privés de sortir sur Internet sans être directement exposés.	Sortie sécurisée pour mises à jour et appels API externes
Route Tables	Règles qui définissent où router chaque paquet réseau.	Dirigent vers subnets publics ou privés selon la source
Security Groups	Pare-feu virtuel qui filtre le trafic entrant et sortant par port.	Ports autorisés : 443, 3000, 3500

Pourquoi cette séparation ?

- **Sécurité** : EKS nodes en subnets privés (pas directement accessibles depuis Internet) ;
- **Haute disponibilité** : 2 AZ garantissent que si une zone AWS tombe, l’autre continue ;
- **Scalabilité** : Load Balancers en subnets publics accèdent aux nodes privés via route tables.

La figure 5.7 confirme la création effective du VPC et de ses subnets dans la console

AWS Management, avec les deux zones de disponibilité configurées pour garantir la haute disponibilité.

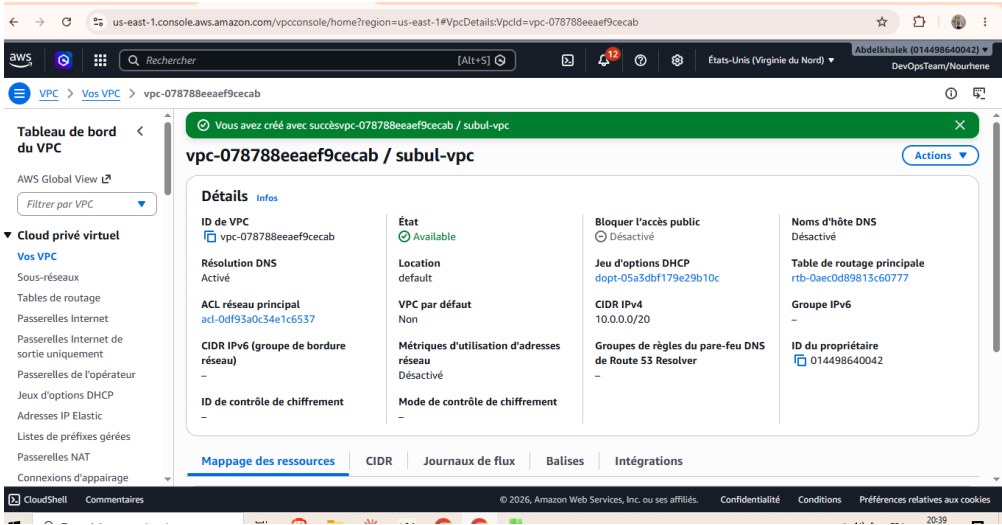


FIGURE 5.7 :VPC et subnets créés dans AWS Management Console

EKS Cluster Configuration

Amazon EKS (Elastic Kubernetes Service) est le service AWS qui héberge et gère notre cluster Kubernetes. Il orchestre le démarrage, la surveillance et le redémarrage automatique des 13 microservices Subul.

Le tableau 5.5 présente les paramètres de configuration du cluster EKS subul-prod, incluant les spécifications des nodes, la politique d’auto-scaling et les mécanismes de contrôle d’accès.

TABLEAU 5.5 :AWS EKS Cluster — Subul Production

Paramètre	Configuration
Cluster Name	subul-prod
Région	eu-west-3 (Paris)
Version Kubernetes	v1.30.14-eks-3385e9b
Control Plane	AWS Managed
Worker Nodes	2 × t3.medium (2 vCPU, 4 GB RAM)
Distribution AZ	1 node en eu-west-3a, 1 node en eu-west-3b

Suite page suivante...

Suite Tableau 5.5

Paramètre	Configuration
Auto Scaling	Min : 2 nodes, Max : 10 nodes (CPU > 70%, RAM > 80%)
RBAC	Activé — Role-Based Access Control
HPA	Kubernetes Horizontal Pod Autoscaler

La figure 5.8 montre le cluster EKS `subul-prod` créé et actif dans la console AWS, avec ses node groups déployés dans les deux zones de disponibilité.

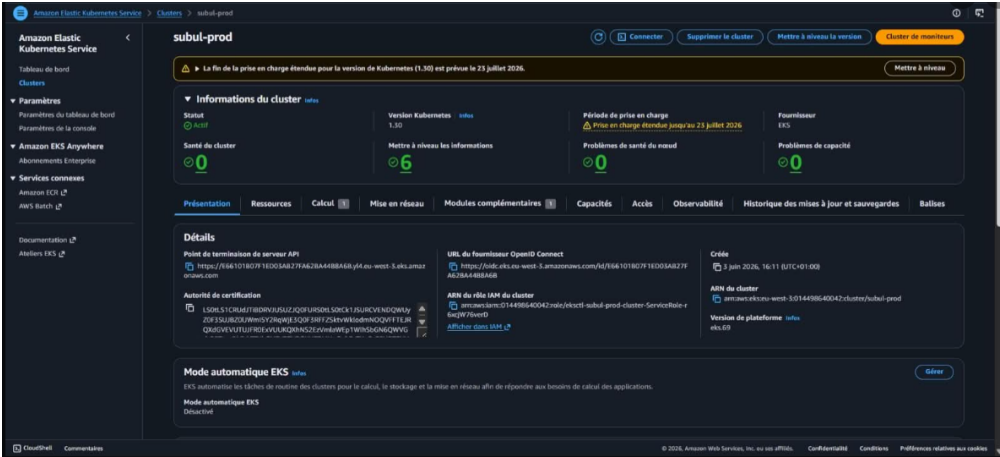


FIGURE 5.8 :Vue d’ensemble du cluster EKS dans AWS Management Console

RDS PostgreSQL

Amazon RDS (Relational Database Service) est la base de données PostgreSQL hébergée et gérée par AWS. AWS s’occupe automatiquement des sauvegardes, mises à jour de sécurité et bascule en cas de panne.

Le tableau 5.6 récapitule la configuration de l’instance RDS PostgreSQL déployée pour la plateforme Subul, avec les paramètres de haute disponibilité et de sauvegarde automatique.

TABLEAU 5.6 :RDS PostgreSQL Configuration — Subul Production

Paramètre	Valeur
Engine	PostgreSQL 15.4

Suite page suivante...

Suite Tableau 5.6

Paramètre	Valeur
Instance Class	db.t3.micro (1 vCPU, 1 GB RAM)
Storage	100 GB SSD avec auto-scaling
Multi-AZ	Oui (Primary AZ-a, Standby AZ-b)
Failover	< 1 minute (automatique)
Backup Retention	7 jours (snapshots journaliers)
Security	Security Group limité à EKS (port 5432)

ElastiCache Redis

Amazon ElastiCache est le service de cache en mémoire vive. Il stocke les sessions utilisateurs et les données fréquemment consultées pour répondre en quelques millisecondes sans interroger la base de données.

Le tableau 5.7 présente la configuration du cache Redis déployé sur ElastiCache, utilisé pour les sessions JWT, les canaux WebSocket et le cache sémantique de l'Agent Tuteur IA.

TABLEAU 5.7 :Redis Cache Configuration — Subul Production

Paramètre	Valeur
Engine	Redis 7.0
Node Type	cache.t3.micro
Mémoire	0.5 GB
Use Cases	Sessions JWT, canaux WebSocket, cache sémantique Agent Tuteur IA, rate limiting

5.3.3 PHASE 2 : Microservices & Containerisation

Les 13 Microservices

La plateforme Subul est composée de 13 services containerisés, chacun disposant de son propre `Dockerfile` optimisé pour la production.

Le tableau 5.8 récapitule les 13 services containerisés de la plateforme Subul, avec leur port d'exposition, leur stack technologique et leur responsabilité dans l'architecture.

TABLEAU 5.8 :Les 13 Microservices Subul

Service	Port	Tech Stack	Responsabilité
auth-service	3001	Node.js/NestJS	Génération JWT, login, vérification email
user-service	3002	Node.js/NestJS	Profils, avatar, préférences
course-service	3003	Node.js/NestJS	Cours, modules, leçons, flashcards
goals-service	3004	Node.js/NestJS	Suivi des objectifs (daily/weekly/general)
learner-service	3005	Node.js/NestJS	Analytiques, progression, dashboard
notification-service	3006	Node.js/Bull	Emails et notifications push
ai-service	8000	Python/FastAPI	Agents IA (Tuteur, Quiz, Roadmap)
gateway	3500	Node.js/NestJS	Routage API, validation JWT
web	3000	Next.js/React	Interface frontend
rabbitmq	5672	RabbitMQ	File de messages inter-services
redis	6379	Redis	Cache, sessions, cache sémantique IA
postgres	5432	PostgreSQL	Base de données relationnelle
monitoring	9090	Prometheus	Collecte de métriques

Dockerfiles

Un Dockerfile est le fichier de recette qui décrit comment construire l'image d'un service : système de base, dépendances, port exposé et commande de démarrage.

Chaque service de Subul possède un Dockerfile dédié. Le Dockerfile de la **Gateway NestJS** suit un build multi-stage (builder → production) pour réduire la taille de l'image finale et exclure les dépendances de développement. Le Dockerfile du **service IA Python/FastAPI** installe les dépendances LangChain, FastAPI et asynpcpg dans un environnement virtuel isolé.

La figure 5.9 présente le Dockerfile du service Gateway, utilisant un build multi-stage pour réduire la taille de l'image finale en séparant les phases de compilation et d'exécution.

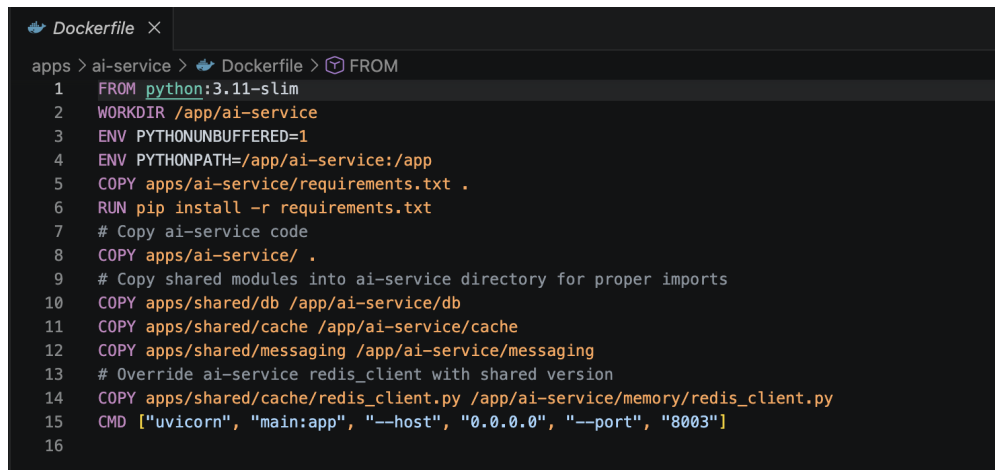
```

Dockerfile
apps > gateway > Dockerfile > FROM
1 FROM node:20-alpine AS deps
2
3 WORKDIR /app
4
5 COPY package.json package-lock.json turbo.json ./
6 COPY apps/gateway/package.json apps/gateway/package.json
7 COPY packages/eslint-config/package.json packages/eslint-config/package.json
8 COPY packages/shared-contracts/package.json packages/shared-contracts/package.json
9 COPY packages/typescript-config/package.json packages/typescript-config/package.json
10 COPY packages/ui/package.json packages/ui/package.json
11
12 RUN npm ci
13
14 # -----
15 FROM node:20-alpine AS builder
16
17 WORKDIR /app
18
19 COPY --from=deps /app .
20 COPY apps/gateway apps/gateway
21 COPY packages packages
22
23 RUN npm run build --workspace=@repo/shared-contracts
24 RUN npm run build --workspace=gateway
25
26 # -----
27 FROM node:20-alpine AS runner
28
29 WORKDIR /app
30
31 ENV NODE_ENV=production
32
33 COPY --from=builder /app/apps/gateway/dist ./dist
34 COPY --from=builder /app/node_modules ./node_modules
35 COPY --from=builder /app/packages/shared-contracts/dist ./node_modules/@repo/shared-contracts/dist
36 COPY --from=builder /app/packages/shared-contracts/package.json ./node_modules/@repo/shared-contracts/package.json
37
38 EXPOSE 3000
39
40 CMD ["node", "dist/main"]
41

```

FIGURE 5.9 :Dockerfile du service Gateway (build multi-stage)

La figure 5.10 illustre le Dockerfile du service IA Python/FastAPI, installant les dépendances LangChain et FastAPI dans un environnement virtuel isolé pour garantir la reproductibilité du build.



```

Dockerfile X
apps > ai-service > Dockerfile > FROM
1 FROM python:3.11-slim
2 WORKDIR /app/ai-service
3 ENV PYTHONUNBUFFERED=1
4 ENV PYTHONPATH=/app/ai-service:/app
5 COPY apps/ai-service/requirements.txt .
6 RUN pip install -r requirements.txt
7 # Copy ai-service code
8 COPY apps/ai-service/ .
9 # Copy shared modules into ai-service directory for proper imports
10 COPY apps/shared/db /app/ai-service/db
11 COPY apps/shared/cache /app/ai-service/cache
12 COPY apps/shared/messaging /app/ai-service/messaging
13 # Override ai-service redis_client with shared version
14 COPY apps/shared/cache/redis_client.py /app/ai-service/memory/redis_client.py
15 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8003"]
16

```

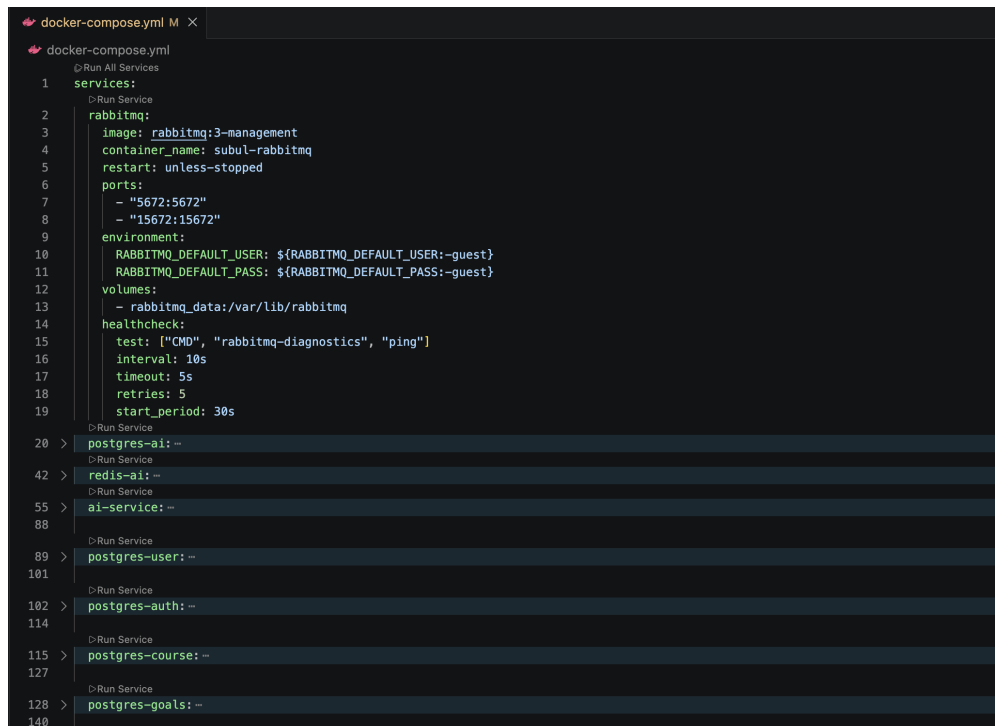
FIGURE 5.10 :Dockerfile du service AI (Python/FastAPI)

Docker Compose pour le développement local

Docker Compose est l'outil qui permet de démarrer tous les services ensemble en une seule commande en local. Il simule l'environnement de production sur la machine du développeur.

Docker Compose orchestre l'ensemble des 13 services en local via `docker compose up`, en configurant les réseaux internes, les volumes persistants pour PostgreSQL et Redis, les variables d'environnement par service et les dépendances de démarrage (`depends_on`).

La figure 5.11 montre un extrait du fichier Docker Compose utilisé pour l'environnement de développement local, orchestrant les 13 services avec leurs dépendances et configurations réseau.



```

1  services:
2    rabbitmq:
3      image: rabbitmq:3-management
4      container_name: subul-rabbitmq
5      restart: unless-stopped
6      ports:
7        - "5672:5672"
8        - "15672:15672"
9      environment:
10       RABBITMQ_DEFAULT_USER: ${RABBITMQ_DEFAULT_USER:-guest}
11       RABBITMQ_DEFAULT_PASS: ${RABBITMQ_DEFAULT_PASS:-guest}
12      volumes:
13        - rabbitmq_data:/var/lib/rabbitmq
14      healthcheck:
15        test: ["CMD", "rabbitmq-diagnostics", "ping"]
16        interval: 10s
17        timeout: 5s
18        retries: 5
19        start_period: 30s
20
21  postgres-ai: --
22
23  redis-ai: --
24
25  ai-service: --
26
27
28  postgres-user: --
29
30
31  postgres-auth: --
32
33
34  postgres-course: --
35
36
37  postgres-goals: --

```

FIGURE 5.11 :Extrait du fichier Docker Compose (services infrastructure et applicatifs)

5.3.4 PHASE 3 : GitHub Actions CI/CD Pipeline

GitHub Actions est l'outil d'automatisation intégré à GitHub. Chaque push déclenche automatiquement les tests, la construction des images et le déploiement sans intervention manuelle.

Vue d'ensemble des Workflows

Le pipeline CI/CD est implémenté sous forme de **7 workflows GitHub Actions** indépendants, chacun couvrant un aspect spécifique du cycle de livraison.

Le tableau 5.9 liste les 7 workflows GitHub Actions configurés pour le projet Subul, couvrant les phases de CI, de déploiement, d'audit de sécurité et de gestion des releases.

TABLEAU 5.9 :Workflows GitHub Actions du projet Subul

Fichier	Déclencheur	Rôle
ci.yml	Push/PR sur main	Tests, lint, type-check TypeScript, génération Prisma, build Turbo, tests Jest avec RabbitMQ
pr-checks.yml	Ouverture/MAJ d'une PR	Vérifications obligatoires avant merge (scan de vulnérabilités Trivy)
coverage.yml	Push sur main	Calcul et rapport de couverture de tests
audit.yml	Push + cron planifié	Audit sécurité des dépendances (npm audit) et scan de secrets
deploy.yml	Push sur main	Build Docker + déploiement sur EKS
eks-deploy.yml	Tag v*.*.*	Déploiement de release en production
release.yml	Tag v*.*.*	Génération changelog et création de release GitHub

La figure 5.12 présente une exécution complète du pipeline CI/CD `ci.yml`, montrant les étapes lint, build, test et docker-build toutes passées avec succès en moins de 10 minutes.

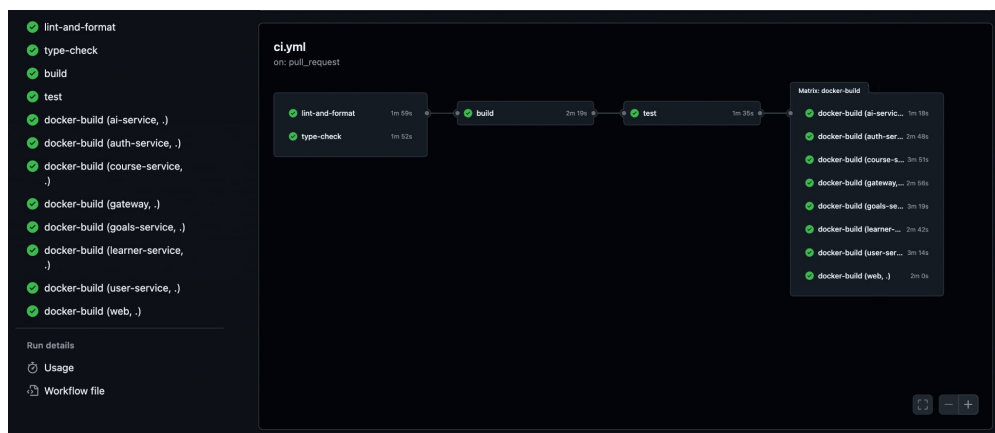


FIGURE 5.12 :Exemple d'exécution d'un workflow GitHub Actions

Étapes du Déploiement (`deploy.yml`)

Le tableau 5.10 détaille les 10 étapes automatisées du workflow de déploiement `deploy.yml`, depuis la détection des changements jusqu'à la vérification des pods en production.

TABLEAU 5.10 :Étapes de déploiement par service

#	Étape	Utilité
1	Détection des changements	Identifier les services modifiés (diff de commits)
2	Checkout repo	Récupérer le code source
3	Configure AWS credentials	Authentification sécurisée via OIDC
4	Login to ECR	Accès au registre d'images Docker privé
5	<code>docker build</code>	Construire l'image du service
6	<code>docker push to ECR</code>	Publier l'image avec tag <code>sha-xxxxxx</code>
7	Update kubeconfig	Connexion au cluster EKS
8	<code>helm upgrade -install</code>	Déployer la nouvelle version
9	<code>kubectl rollout status</code>	Attendre la fin du rollout
10	Vérifier pods READY	Confirmer le bon déploiement

Les services sont buildés **en parallèle** grâce à la matrix strategy GitHub Actions : seuls les services dont les fichiers ont changé sont redéployés, réduisant la durée totale du pipeline à **3–5 minutes**.

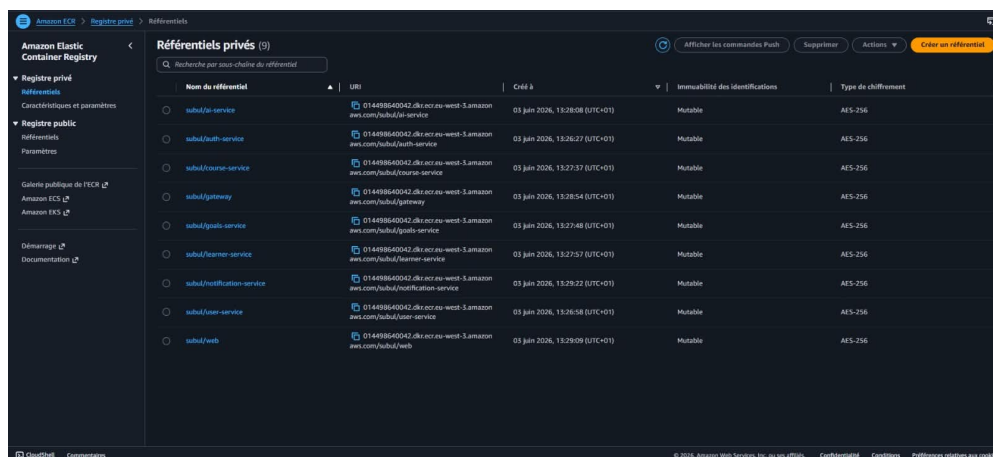
5.3.5 PHASE 4 : Amazon ECR Registry

Amazon ECR (Elastic Container Registry) est le registre Docker privé d'AWS. Il stocke et versionne les images de chaque microservice, prêtes à être déployées sur EKS via GitHub Actions.

Après que GitHub Actions construise les images Docker, elles sont poussées vers Amazon ECR avec un tag basé sur le SHA du commit (`sha-xxxxxx`), garantissant

la traçabilité complète de chaque déploiement. Chaque service dispose de son propre repository ECR.

La figure 5.13 confirme la présence des images Docker de chaque microservice dans Amazon ECR, taguées avec le SHA du commit pour garantir la traçabilité complète de chaque déploiement.



Nom du référentiel	URI	Créé à	Immutabilité des identifications	Type de chiffrement
subul/ja-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/ja-service	03 juin 2026, 13:28:08 (UTC+01)	Mutable	AES-256
subul/jaui-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/jaui-service	03 juin 2026, 13:26:27 (UTC+01)	Mutable	AES-256
subul/jauser-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/jauser-service	03 juin 2026, 13:27:37 (UTC+01)	Mutable	AES-256
subul/gateway	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/gateway	03 juin 2026, 13:28:54 (UTC+01)	Mutable	AES-256
subul/gpu-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/gpu-service	03 juin 2026, 13:29:48 (UTC+01)	Mutable	AES-256
subul/tearmer-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/tearmer-service	03 juin 2026, 13:27:57 (UTC+01)	Mutable	AES-256
subul/verification-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/verification-service	03 juin 2026, 13:29:22 (UTC+01)	Mutable	AES-256
subul/user-service	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/user-service	03 juin 2026, 13:26:58 (UTC+01)	Mutable	AES-256
subul/web	014498640042.dkr.ecr.eu-west-1.amazonaws.com/subul/web	03 juin 2026, 13:29:09 (UTC+01)	Mutable	AES-256

FIGURE 5.13 : Images Docker disponibles dans Amazon ECR par service

5.3.6 PHASE 5 : Kubernetes (EKS) Deployment

Déploiement

Une fois les images Docker publiées sur Amazon ECR, le workflow `deploy.yml` déclenche automatiquement le déploiement sur le cluster EKS via **Helm**. Chaque microservice est déployé indépendamment grâce à son propre Helm chart, permettant des mises à jour ciblées sans affecter les autres services. La commande `helm upgrade -install` applique la nouvelle configuration et `kubectl rollout status` attend la confirmation que tous les pods sont en état **Running** avant de valider le déploiement.

La figure 5.14 confirme que l'ensemble des 13 microservices sont déployés et actifs sur le cluster EKS en production, avec le statut **Running** pour chaque pod.

```

mac@Sirines-MacBook-Air Subul-micro % kubectl get pods -n subul
NAME                                READY   STATUS    RESTARTS   AGE
ai-service-7746dd7f85-29c7b        1/1     Running   0           6d9h
ai-service-7746dd7f85-kg6hv        1/1     Running   0           6d9h
auth-service-ff5d78468-ff27h       1/1     Running   0           6d9h
course-service-56576b78cf-2b2t2    1/1     Running   0           6d9h
gateway-56b9bffffbd-cbg67         1/1     Running   0           6d9h
gateway-56b9bffffbd-qm84c         1/1     Running   0           6d9h
goals-service-55d6bb5dcc-cqd1m     1/1     Running   0           6d9h
learner-service-5fb745b57c-17wx7   1/1     Running   0           6d9h
notification-service-7f9db97b46-np56j 1/1     Running   0           6d9h
seed-courses                        0/1     Completed 0           13d
seed-fix                           0/1     Completed 0           17d
user-service-7cc989c8b7-tx5hs      1/1     Running   0           6d9h
web-7c8576b58f-hgfhh              1/1     Running   0           6d9h
web-7c8576b58f-qrbdg              1/1     Running   0           6d9h
mac@Sirines-MacBook-Air Subul-micro %

```

FIGURE 5.14 :État des pods en production (kubectl get pods — STATUS : Running)

Rolling Updates (Zero Downtime)

Kubernetes effectue les mises à jour sans interruption de service :

1. Création d'un pod avec la nouvelle version de l'image ;
2. Attente de la `readinessProbe` (le pod est prêt) ;
3. Basculement progressif du trafic (50% puis 100%) ;
4. Suppression de l'ancien pod.

Résultat : **zéro downtime**, SLA 99.9% maintenu lors de chaque déploiement.

5.3.7 PHASE 6 : Load Balancers & Networking

AWS ELB (Elastic Load Balancer) répartit automatiquement le trafic entre les pods disponibles. Si un pod tombe, il redirige les requêtes vers les pods sains sans interruption pour l'apprenant.

Deux Load Balancers

Le tableau 5.11 présente les deux Load Balancers AWS provisionnés pour exposer le frontend et l'API gateway de Subul aux apprenants.

TABLEAU 5.11 :AWS Elastic Load Balancers — Subul Production

LB	URL	Rôle
Web LB	a5acd798...elb.amazonaws.com:3000	R3000 vers web-service (frontend Next.js)
API LB	ab19613f...elb.amazonaws.com:3500	R3500 vers gateway (backend NestJS)

Flux d'une requête étudiant

Le tableau 5.12 détaille les 9 étapes du flux complet d'une requête de connexion apprenant, depuis le navigateur jusqu'à l'authentification JWT.

TABLEAU 5.12 :Flux d'une requête de connexion apprenant

#	Composant	Action	Détail
1	Browser	Envoi requête	GET http://...elb.amazonaws.com:3000
2	Load Balancer (Web)	Répartition	Health check + sélection pod (round-robin)
3	Web pod (Next.js)	Rendu page	Render React → affiche interface login
4	Browser	Action utilisateur	POST /auth/login avec cookies HTTP-Only
5	Load Balancer (API)	Répartition	Route vers un gateway pod disponible
6	Gateway (NestJS)	Validation	Extrait JWT, valide signature, forward auth-service
7	auth-service (NestJS)	Authentification	Query PostgreSQL RDS, valide password, crée JWT
8	Gateway	Réponse	200 OK + JWT en cookie HTTP-Only
9	Browser	Résultat	Apprenant connecté

5.3.8 PHASE 7 : Security & Secrets Management

Tous les secrets sont stockés dans **Kubernetes Secrets** (jamais en clair dans le code, les Dockerfiles ou les Helm charts).

Le tableau 5.13 liste les cinq Kubernetes Secrets configurés en production pour sécuriser les accès aux bases de données, à l'API Azure et aux services AWS.

TABLEAU 5.13 :Kubernetes Secrets configurés — Subul Production

Secret Name	Contient
subul-db-secrets	DATABASE_URL pour PostgreSQL RDS
subul-redis-secrets	REDIS_URL pour ElastiCache
subul-jwt-secrets	JWT_SECRET pour signer les tokens
subul-azure-secrets	AZURE_OPENAI_API_KEY, AZURE_SEARCH_API_KEY
subul-aws-secrets	Credentials AWS pour ECR et S3

Conclusion

Ce chapitre a présenté la réalisation de la **Release 3**, dernière étape du cycle de développement de la plateforme SUBUL, articulée autour de deux sprints aux objectifs complémentaires.

Conclusion Générale

Ce projet avait pour objectif de répondre à une double problématique : le manque d'accompagnement personnalisé dans les plateformes e-learning actuelles et le défi technique d'intégrer des agents d'intelligence artificielle au sein d'une architecture micro-services moderne. Pour y répondre, nous avons conçu et développé **Subul Academy**, une plateforme d'apprentissage intelligente intégrant plusieurs agents IA spécialisés, conduite selon la méthodologie Scrum depuis l'analyse des besoins jusqu'au déploiement en production sur infrastructure AWS.

Le travail réalisé a permis de mettre en place une architecture microservices complète, un service IA Python/FastAPI basé sur une architecture RAG, ainsi qu'un agent tuteur conversationnel et un agent de génération automatique de quiz offrant une assistance pédagogique en temps réel. Ces apports confèrent à la solution un intérêt concret dans un contexte d'utilisation réel, en rendant l'apprentissage plus interactif et adaptatif.

Néanmoins, la qualité des réponses des agents reste liée à la richesse des données indexées, et le coût opérationnel de l'infrastructure cloud peut limiter le passage à grande échelle. Ces contraintes représentent des axes d'amélioration prioritaires pour les versions futures.

À terme, l'intégration de modèles IA plus récents, l'amélioration des stratégies RAG et l'optimisation de l'infrastructure cloud permettraient d'élever davantage la précision et la scalabilité de la plateforme. Ce projet s'inscrit ainsi dans une tendance plus large où l'intelligence artificielle redéfinit progressivement les systèmes éducatifs, et **Subul Academy** constitue une base solide pour une nouvelle génération de plateformes d'apprentissage intelligentes.

Bibliographie et Webographie

Bibliographie

- [1] Peter BRUSILOVSKY et Eva MILLÁN. « User Models for Adaptive Hypermedia and Adaptive Educational Systems ». In : *The Adaptive Web* (2007), p. 3-53.
- [4] Chris RICHARDSON. *Microservices Patterns : With Examples in Java*. Manning Publications, 2018. ISBN : 978-1617294549.
- [8] Patrick LEWIS et al. « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474.
- [9] Tianyu LIU et al. « A Survey of Large Language Models in Education ». In : *arXiv preprint arXiv :2401.12370* (2024).

Webographie

- [2] COURSERA INC. *Coursera Online Learning Platform*. Consulté le 10 juin 2025. 2025. URL : <https://www.coursera.org>.
- [3] AGILE ALLIANCE. *Manifesto for Agile Software Development*. Consulté le 10 juin 2025. 2001. URL : <https://agilemanifesto.org>.
- [5] Andrew KANE. *pgvector : Open-source vector similarity search for Postgres*. Consulté le 10 juin 2025. 2024. URL : <https://github.com/pgvector/pgvector>.
- [6] NESTJS. *NestJS Documentation*. Consulté le 10 juin 2025. 2024. URL : <https://docs.nestjs.com>.
- [7] BROADCOM INC. *RabbitMQ Documentation*. Consulté le 10 juin 2025. 2024. URL : <https://www.rabbitmq.com/documentation.html>.
- [10] ARTILLERY. *Artillery Documentation*. Consulté en 2026. 2026. URL : <https://www.artillery.io/docs>.

Résumé

Ce projet a été réalisé dans le cadre du projet de fin d'études pour l'obtention du Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques, spécialité Génie Logiciel et Systèmes d'Information, au sein de la startup **Smartovate**. L'objectif principal est la conception et le développement de **SUBUL**, une plateforme d'accompagnement intelligent dédiée à la préparation aux certifications professionnelles et au développement des compétences. Le système intègre une gestion complète des cours et de la progression des apprenants, des outils d'apprentissage actif (notes, flashcards SM-2, quiz adaptatifs), ainsi que deux agents d'intelligence artificielle — le **TutorAgentMCP** et le **QuizAgent** — exploitant les technologies RAG, LangGraph ReAct et Azure OpenAI pour offrir un accompagnement personnalisé en temps réel. La plateforme repose sur une architecture **microservices** déployée en production sur **AWS EKS** via un pipeline CI/CD automatisé avec **GitHub Actions**.

Mots-clés : Microservices, Intelligence Artificielle, RAG, LangGraph, NestJS, Next.js, AWS EKS, CI/CD, Certification, Apprentissage adaptatif.

Abstract

This project was carried out as part of the final year engineering project for the obtainment of the National Engineering Degree in Applied Sciences and Technology, specializing in Software Engineering and Information Systems, within the startup **Smartovate**. The main objective is the design and development of **SUBUL**, an intelligent learning platform dedicated to professional certification preparation and skills development. The system integrates comprehensive course and learner progress management, active learning tools (notes, SM-2 flashcards, adaptive quizzes), and two artificial intelligence agents — the **TutorAgentMCP** and the **QuizAgent** — leveraging RAG, LangGraph ReAct and Azure OpenAI technologies to deliver real-time personalized support. The platform is built on a **microservices** architecture deployed in production on **AWS EKS** via an automated CI/CD pipeline with **GitHub Actions**.

Keywords : Microservices, Artificial Intelligence, RAG, LangGraph, NestJS, Next.js, AWS EKS, CI/CD, Certification, Adaptive Learning.